



OSTBAYERISCHE
TECHNISCHE HOCHSCHULE
REGENSBURG

Artificial Intelligence-Based Anomaly Detection for Electric Vehicle Supply Equipment Using Long Short-Term Memory Architecture

An der Fakultät für Informatik und Mathematik der
Ostbayerischen Technischen Hochschule Regensburg
im Studiengang
Wirtschaftsinformatik

eingereichte

Bachelorarbeit

zur Erlangung des akademischen Grades des
Bachelor of Science (B.Sc.)

Vorgelegt von: Yannic Hanel
Matrikelnummer: 3296954

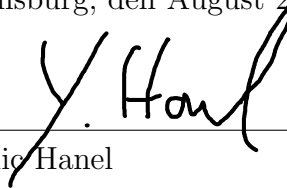
Erstgutachter: Prof. Dr. Rudolf Hackenberg
Zweitgutachter: Prof. Dr. Sebastian Fischer

Abgabedatum: 22.08.2025

Erklärung zur Bachelorarbeit

1. Mir ist bekannt, dass dieses Exemplar der Abschlussarbeit als Prüfungsleistung in das Eigentum der Ostbayerischen Technischen Hochschule Regensburg übergeht.
2. Ich erkläre hiermit, dass ich diese Abschlussarbeit selbständig verfasst, noch nicht anderweitig für Prüfungszwecke vorgelegt, keine anderen als die angegebenen Quellen und Hilfsmittel benutzt sowie wörtliche und sinngemäße Zitate als solche gekennzeichnet habe.

Regensburg, den August 21, 2025



Yannic Hanel

Abstract

This thesis explores the use of artificial intelligence for anomaly detection in electric vehicle supply equipment (EVSE), with a particular focus on wallbox charging units in private environments. The objective is to reliably detect cyberattacks and abnormal behavior by analyzing hardware and kernel event data using machine learning techniques, specifically, Long Short-Term Memory (LSTM) networks.

After reviewing established approaches, both publicly available datasets (hereafter referred to as CICEVSE2024) and self-collected data are analyzed, with decision trees applied for feature selection before training the LSTM models.

The implemented LSTM-based detection system is evaluated on both datasets and achieves high detection accuracy, especially for flooding attacks. The results demonstrate that careful feature selection significantly reduces model complexity and resource consumption, enabling real-time deployment on embedded devices. Nonetheless, limitations remain, particularly regarding cross-platform transferability and the black-box nature of LSTM models. Future work will focus on broadening the coverage of attack scenarios and extending detection to distributed, multi-station setups. Another priority will be ensuring that the detection algorithm always operates within a reserved portion of system resources. This would guarantee that Prometheus queries and anomaly detection remain functional even under severe flooding attacks.

Overall, this research contributes to enhancing the security and robustness of monitoring solutions for critical electric vehicle charging infrastructure.

Contents

I	List of Abbreviations	XI
1	Introduction	1
1.1	Motivation and Relevance of Anomaly Detection in Electric Vehicle Supply Equipment	1
1.2	Overview of Artificial Intelligence-based Approaches	4
1.3	Research Questions	5
1.4	Structure of the Thesis	6
2	Theoretical Background and Core Concepts	7
2.1	Fundamentals of Anomaly Detection	7
2.2	Data Processing and Visualization Tools	8
2.3	Introduction to Decision Trees, Isolation Forest and Long Short-Term Memory .	9
2.4	Overview of Long Short-Term Memory Architecture	11
3	Related Work	13
3.1	Recent Advances in Cybersecurity and Anomaly Detection for Electric Vehicle Supply Equipment	13
3.2	General Anomaly Detection Methods and Foundations	13
3.3	Performance Comparison of Deep Learning Approaches for Electric Vehicle Supply Equipment Anomaly Detection	14
4	Data Acquisition	16
4.1	Data Sources for Electric Vehicle Supply Equipment Anomaly Detection	16
4.2	Integration of Prometheus	17
5	Exploratory Data Analysis	19
5.1	Holistic Dataset Comparison	19
5.2	Feature Comparison of CICEVSE2024 and Self-Collected Datasets	24
6	Feature Selection	31
6.1	Data Preprocessing Steps	31
6.2	Feature Selection Using Decision-Tree-Classifier	32
7	LSTM Architecture: Structure, Training, and Optimization	41
7.1	LSTM Architecture and Preprocessing	41
7.2	Parameter Configuration and Hyperparameter Optimization	42
8	Analysis and Evaluation	45
8.1	Experimental Setup and Evaluation Metrics	45

8.2 Results of Anomaly Detection Using the Machine Learning Model	46
9 Discussion	52
10 Future Work	55
Anhang	XLIII

List of Figures

1	Availability of public charging points and market share of electric cars in the European Union [4].	1
2	Electric Vehicle Supply Equipment as an access node to critical data [6].	2
3	Interior view of the wallbox [12].	3
4	An illustration of the position of Machine Learning and Deep Learning within the area of Artificial Intelligence [15]	4
5	Data distribution classified by Type 1: Outlier Clustering [17].	7
6	Z-score with critical region [18]	8
7	Decision Tree [22]	10
8	Setup of the Electric Vehicles Charging Station [26]	20
9	Selected Hardware Performance Counter and Kernel Features from the paper "Enhancing EV Charging Station Security Using a Multi-dimensional Dataset" [26]	21
10	Boxplot of the CICEVSE2024 Dataset Features.	22
11	Boxplot of the self-collected Dataset Features.	23
12	L1-icache-loads timeline (CICEVSE2024).	24
13	L1-icache-loads timeline (self-collected).	24
14	Instruction timeline.	25
15	Instruction timeline (CICEVSE2024).	25
16	Cache-misses timeline.	26
17	Cache-misses timeline (CICEVSE2024).	26
18	Exc-taken timeline.	27
19	Exc-taken timeline (CICEVSE2024).	27
20	Cpu-migrations timeline.	28
21	Cpu-migrations timeline (CICEVSE2024).	28
22	dTLB-store-misses timeline.	29
23	dTLB-store-misses timeline (CICEVSE2024).	29
24	Comparison of feature selection methods for Long Short-Term Memory accuracy [29].	33
25	Top 10 most important features selected by the Decision-Tree-Classifier.	34
26	Correlation matrix top Features with Decision-Tree-Classifier.	35
27	Correlation matrix of top features selected by the Decision-Tree-Classifier on the CICEVSE2024 dataset.	36
28	Correlation between <code>skb_kfree_skb</code> and <code>fib_fib_table_lookup</code> features.	37
29	Timeline behavior of the <code>node-stores</code> feature, showing distinguishable patterns during flooding attacks.	38
30	Timeline behavior of the <code>node-stores</code> feature from the CICEVSE2024 dataset.	38

31	Timeline behavior of the <code>sys_calls_sys_enter_kill</code> feature, highlighting its usefulness in detecting Portscan attacks.	39
32	Timeline behavior of the <code>sys_calls_sys_enter_kill</code> feature, from the CICEVSE2024 dataset. The lack of variation in the feature within the CICEVSE2024 dataset may be due to a measurement error or the system call not being invoked during the entire monitoring period. Unfortunately, neither the paper [26] nor the accompanying dataset provides a definitive explanation.	39
33	Timeline behavior of the <code>unaligned_st_spec</code> feature, showing little variation between benign and attack data.	40
34	Timeline behavior of the <code>unaligned_st_spec</code> feature, from the CICEVSE2024 dataset.	40
35	Confusion matrix for the self-collected dataset	48
36	Confusion matrix for the CEVSE2024 dataset	48
37	ROC curve for the self-collected dataset	49
38	ROC curve for the CEVSE2024 dataset	49
39	Precision-Recall curve for the self-collected dataset	50
40	Precision-Recall curve for the CEVSE2024 dataset	50
41	Recall statistics for the self-collected dataset	51
42	Recall statistics for the CEVSE2024 dataset	51
43	Live data classification results	52
44	<code>l1d.cache_wr</code> timeline	VIII
45	<code>l1d.cache_wr</code> timeline (CICEVSE2024)	IX
46	<code>instruction</code> timeline	IX
47	<code>instruction</code> timeline (CICEVSE2024)	X
48	<code>l2d.cache_rd</code> timeline	X
49	<code>l2d.cache_rd</code> timeline (CICEVSE2024)	XI
50	<code>mem.access_rd</code> timeline	XI
51	<code>mem.access_rd</code> timeline (CICEVSE2024)	XII
52	<code>mem.access_wr</code> timeline	XII
53	<code>mem.access_wr</code> timeline (CICEVSE2024)	XIII
54	<code>kmem.kfree</code> timeline	XIII
55	<code>kmem.kfree</code> timeline (CICEVSE2024)	XIV
56	<code>net_dev_xmit</code> timeline	XIV
57	<code>net_dev_xmit</code> timeline (CICEVSE2024)	XV
58	<code>qdisc.dequeue</code> timeline	XV
59	<code>qdisc.dequeue</code> timeline (CICEVSE2024)	XVI
60	<code>raw_syscalls_sys_enter</code> timeline	XVI
61	<code>raw_syscalls_sys_enter</code> timeline (CICEVSE2024)	XVII
62	<code>irq_softirq_raise</code> timeline	XVII
63	<code>irq_softirq_raise</code> timeline (CICEVSE2024)	XVIII

64	sched_migrate_task timeline	XVIII
65	sched_migrate_task timeline (CICEVSE2024)	XIX
66	sched_switch timeline	XIX
67	sched_switch timeline (CICEVSE2024)	XX
68	syscalls_sys_enter_close timeline	XX
69	syscalls_sys_enter_close timeline (CICEVSE2024)	XXI
70	syscalls_sys_enter_read timeline	XXI
71	syscalls_sys_enter_read timeline (CICEVSE2024)	XXII
72	syscalls_sys_enter_write timeline	XXII
73	syscalls_sys_enter_write timeline (CICEVSE2024)	XXIII
74	node-stores timeline	XXIII
75	node-stores timeline (CICEVSE2024)	XXIV
76	bus_access timeline	XXIV
77	bus_access timeline (CICEVSE2024)	XXV
78	skb_kfree_skb timeline	XXV
79	skb_kfree_skb timeline (CICEVSE2024)	XXVI
80	cpu_cycles timeline	XXVI
81	cpu_cycles timeline (CICEVSE2024)	XXVII
82	tcp_tcp_probe timeline	XXVII
83	tcp_tcp_probe timeline (CICEVSE2024)	XXVIII
84	l2d_cache_wb_victim timeline	XXVIII
85	l2d_cache_wb_victim timeline (CICEVSE2024)	XXIX
86	syscalls_sys_enter_fcntl timeline	XXIX
87	syscalls_sys_enter_fcntl timeline (CICEVSE2024)	XXX
88	syscalls_sys_exit_fcntl timeline	XXX
89	syscalls_sys_exit_fcntl timeline (CICEVSE2024)	XXXI
90	writeback_global_dirty_state timeline	XXXI
91	writeback_global_dirty_state timeline (CICEVSE2024)	XXXII
92	sched_sched_stat_runtime timeline	XXXII
93	sched_sched_stat_runtime timeline (CICEVSE2024)	XXXIII
94	st_spec timeline	XXXIII
95	st_spec timeline (CICEVSE2024)	XXXIV
96	syscalls_sys_enter_kill timeline	XXXIV
97	syscalls_sys_enter_kill timeline (CICEVSE2024)	XXXV
98	fib_fib_table_lookup timeline	XXXV
99	fib_fib_table_lookup timeline (CICEVSE2024)	XXXVI
100	kmem_mm_page_alloc_zone_locked timeline	XXXVI
101	kmem_mm_page_alloc_zone_locked timeline (CICEVSE2024)	XXXVII
102	sock_inet_sock_set_state timeline	XXXVII
103	sock_inet_sock_set_state timeline (CICEVSE2024)	XXXVIII

104	l2d_cache_wb timeline	XXXVIII
105	l2d_cache_wb timeline (CICEVSE2024)	XXXIX
106	unaligned_st_spec timeline	XXXIX
107	unaligned_st_spec timeline (CICEVSE2024)	XL
108	mem_access_rd timeline	XL
109	mem_access_rd timeline (CICEVSE2024)	XLI
110	l2d_cache_refill_wr timeline	XLI
111	l2d_cache_refill_wr timeline (CICEVSE2024)	XLII
112	mmc_mmc_request_start timeline	XLII
113	mmc_mmc_request_start timeline (CICEVSE2024)	XLIII

List of Tables

1	Comparison of machine learning approaches with examples, advantages, and disadvantages [15].	5
2	Comparison of supervised learning models with advantages and disadvantages [22][23][24][25].	11
3	Advantages and disadvantages of LSTM [25].	12
4	Overview of executed attacks.	31
5	Attack intervals during data collection.	32
6	Explanation of key components and layers used in the LSTM model configuration [47][48][49][50].	43
7	Classification Metrics for Self-Collected Dataset	46
8	Classification Metrics for the CEVSE2024 Dataset	46
9	Misclassified attacks, Self-collected Dataset	48
10	Misclassified attacks, CEVSE2024 Dataset	48

I List of Abbreviations

Electric Vehicle Supply Equipment (EVSE)
Electric Vehicles (EVs)
Charging Stations (CS)
Datenschutz-Grundverordnung (DSGVO)
General Data Protection Regulation (GDPR)
Artificial Intelligence (AI)
Isolation-Forest (iForest)
Recurrent Neural Network (RNN)
Exploratory Data Analysis (EDA)
Deep Neural Network (DNN)
Application Programming Interface(s)(APIs)
Information Technology (IT)
Denial-of-Service (DoS)
Principal Component Analysis (PCA)
Man-in-the-Middle (MitM)
Prometheus Query Language (PromQL)
Hardware Performance Counter (HPC)
Long Short-Term Memory (LSTM)
Decision tree (DT)
Not a Number (NaN)

1 Introduction

1.1 Motivation and Relevance of Anomaly Detection in Electric Vehicle Supply Equipment

Electric Vehicle Supply Equipment (EVSE) has gained significant importance in recent years [1]. Analyzing the market sales of electric vehicles (EVs), a remarkable growth can be observed. In 2012, only 120,000 EVs were sold, whereas by 2021, sales had surged to nearly 6.6 million. To achieve net-zero CO₂ emissions by 2050, as planned by the European Union [2], the number of EVs must continue to rise substantially, requiring an 11-fold increase from current levels by 2030 [3].

To manage the much higher demand for EVs on public roads, it is expected that the market for EVSE will grow similarly to the sales of EVs [4]. The correlation between the number of EVs and the number of EVSE is high. Market data from the last few years show that the top four countries in the European Union, such as Germany, the Netherlands, France, and Belgium, with the best EVSE infrastructure, are also the top four with the largest market share of EVs [4].

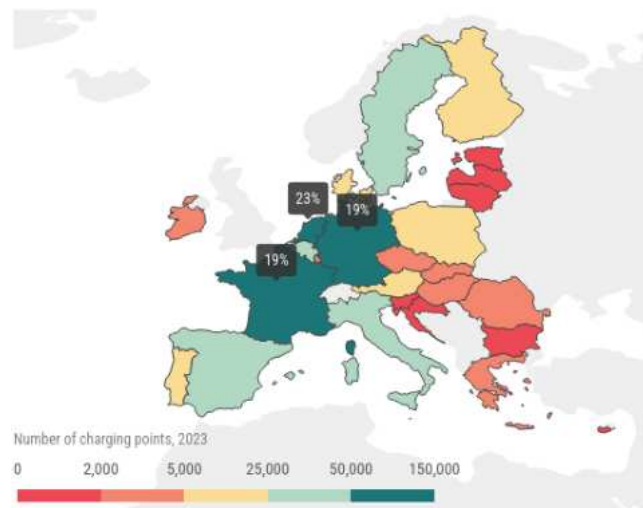


Figure 1: Availability of public charging points and market share of electric cars in the European Union [4].

With the growing number of EVSE, particularly Charging Stations (CS), the interest of cyber-criminal groups in attacking these systems is also increasing [5]. At first glance, EVSE systems may not appear to be direct targets for accessing critical data or manipulation. Upon closer examination, it turns out that they can serve as entry points to gain access to more sensitive or private information [6]. The following figure illustrates how EVSE components, such as charging stations, can act as access nodes to critical or private data.

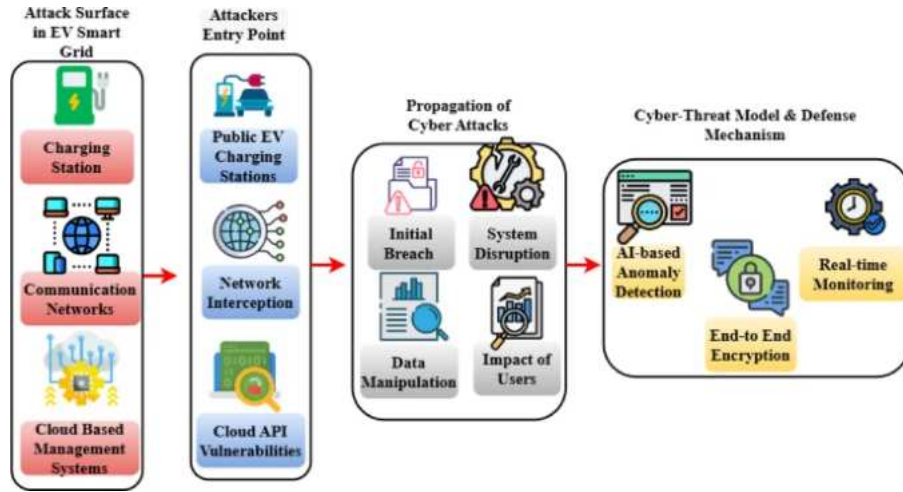


Figure 2: Electric Vehicle Supply Equipment as an access node to critical data [6].

With access to these kinds of data, critical attacks, such as data manipulations, data theft, disruption of service, or even complete system failure, are possible [6]. This does not only lead to a high financial risk, but it also affects the trust of users in EVSE systems and potentially slows down the adoption of EVs in general.

If end users are provided with a stable and secure EVSE infrastructure, consumer confidence in switching to electric vehicles will be significantly higher. This ensures, in turn, consistent growth in the adoption of both EVs and EVSE systems [7].

As outlined in the first paragraphs, the rapid growth of EVs and EVSE infrastructure also requires a robust monitoring and anomaly detection system to safeguard the security of the EVSE infrastructure. Only by guaranteeing a secure EVSE environment can user trust in the energy transition be maintained and major cybersecurity incidents in the future be avoided.

Ensuring high security standards for EVSE systems is of critical importance, but difficult to achieve. The EVSE infrastructure constitutes a complex system that includes not only charging stations but also sophisticated backend systems and communication protocols. These components and software solutions are typically developed, published, and maintained by various vendors, which makes securing the entire system a challenging task.

Combining all the different software from vendors into a secure infrastructure is challenging enough, but the EVSE infrastructure, especially charging stations (CS), is also regularly installed in public places. Even when not in a public setting, these installations are often outdoors, where security measures are significantly lower than in buildings protected by walls, doors, and other physical barriers.

In this paper, the focus is on wallboxes, which are generally smaller than CS and are typically used in private settings. They are often installed directly on house walls or in garages, and thus still form part of the overall EVSE infrastructure.

The fact that they are installed in private areas initially suggests a higher level of security.

However, since these areas are still outdoors, it must be assumed that there are no additional physical security measures beyond the wallbox itself. It is crucial to develop a system that not only protects against software-based attacks, such as Denial-of-Service (DoS) or Distributed Denial-of-Service (DDoS), but also considers other potential attack vectors, such as firmware manipulation and unauthorized data interception. In addition, the system has to secure the hardware against physical threats, like tampering, theft, or other intrusions.

In this project, wallboxes from the *ReSiLENT* [8] project, provided by *Stöhr Mobility GmbH* [9], are used. The software is developed by *ChargeIQ GmbH* [10] and already includes basic security measures to protect against external threats. This project is also carried out in co-operation with *ASVIN GmbH* [11]. In the following chapter, the internal components of the wallbox are examined more closely.



Figure 3: Interior view of the wallbox [12].

In this figure, the internal components of the wallbox are visible. The wallbox is equipped with a Raspberry Pi Zero W, which is responsible for communication with the backend, including the transmission of user data and handling of payment processes from the charging process. This component is secured and programmed by *ChargeIQ GmbH* [10].

The sensitive hardware components are enclosed in an aluminum casing designed by *Stöhr Mobility GmbH* [9], which provides a degree of protection against physical attacks.

As already mentioned, the wallbox also processes personal data of users, such as their user ID and payment information, in communication with the backend server. This requires special attention in the context of the DSGVO / GDPR, as additional requirements for the protection of personal data, such as data minimization, storage limitation [13], and legal obligation [14], must be taken into account throughout the entire EVSE infrastructure.

Security requires significant computational resources, particularly in the domain of anomaly detection. As shown in Figure 3, the wallbox is equipped with a Raspberry Pi Zero W, which offers sufficient resources for a single-board computer. However, when processing and analyzing over 900 hardware events to detect potential anomalies, the system quickly reaches the limits of its hardware capabilities. This is just a small example of the challenges faced in securing EVSE infrastructure.

Due to the combination of digital and physical threats, the involvement of multiple vendors, and strict data protection requirements, securing Electric Vehicle Supply Equipment (EVSE) infrastructure is a complex task and entails numerous challenges.

1.2 Overview of Artificial Intelligence-based Approaches

Artificial intelligence is often used in different contexts; therefore, it is necessary to define what is meant by AI-based approaches and which different approaches exist. It is a leading technology in the current digital transformation and is crucial for advancing the fourth industrial revolution [15].

The goal of AI is to create machines that can perform tasks that typically require human intelligence, such as understanding natural language and recognizing patterns in data. In particular, the detection of patterns is of relevance in this work. The different AI-based approaches can be divided into two subcategories [15]:

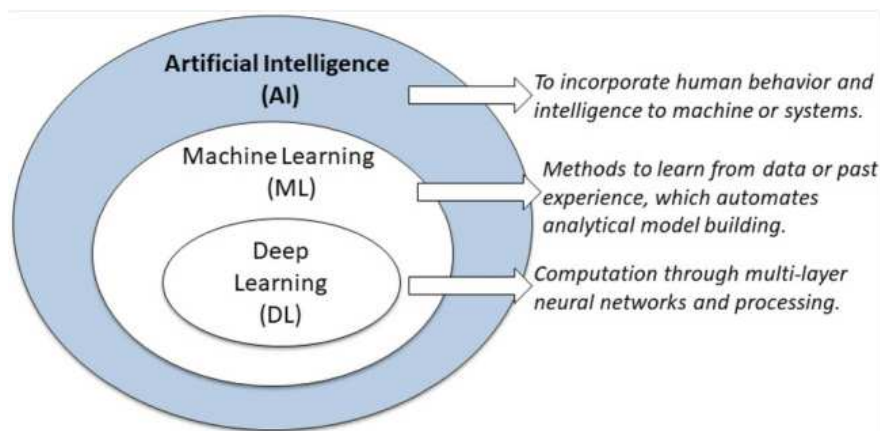


Figure 4: An illustration of the position of Machine Learning and Deep Learning within the area of Artificial Intelligence [15]

These can be further divided into three categories:

- **Supervised Learning:** This approach involves learning from labeled data to identify patterns and make predictions. It is commonly used in tasks such as email spam classification or price prediction, where the algorithm is trained on known input-output pairs.
- **Unsupervised Learning:** This data-driven method is used to uncover hidden patterns in unlabeled data. Techniques such as clustering and dimensionality reduction are applied

to identify structures within the data, for example, to segment customer groups or discover business models.

- **Semi-Supervised Learning:** This approach combines aspects of both supervised and unsupervised learning by utilizing a small amount of labeled data alongside a larger pool of unlabeled data. It is particularly useful when labeling data is expensive or time-consuming, but large amounts of unlabeled data are available.

[15]

In the following table, the different approaches are categorized alongside their corresponding machine learning architectures. This allows identification of the advantages and disadvantages of each method and provides a clearer understanding of which approach is most suitable for producing actionable results in real-world deployments.

Approach	Examples	Advantages	Disadvantages
Supervised Learning	Decision Trees, Random-Forest, LSTM	High prediction accuracy, Clear objectives	Requires a lot of labeled data, Risk of overfitting
Unsupervised Learning	K-Means, Isolation-Forest	Finds patterns without labels, Useful for unlabeled data	Hard to interpret, No clear target
Semi-Supervised Learning	Self-training	Improves performance with few labeled data points	Complex to implement

Table 1: Comparison of machine learning approaches with examples, advantages, and disadvantages [15].

After providing an overview of the different machine learning approaches, the following chapter delves deeper into the field by examining various methods for anomaly detection and identifying the most effective models for detecting typical anomalies.

1.3 Research Questions

The focus of this thesis is on the development and application of "Artificial Intelligence-based Anomaly Detection for Electric Vehicle Supply Equipment using Long Short-Term Memory Architecture". To provide a structured approach to this investigation, the following research questions have been formulated:

- How can data acquisition and preprocessing, including event selection strategies, sampling intervals, and feature aggregation, be optimized to enable high detection accuracy and

real-time anomaly detection in resource-constrained embedded EVSE systems?

- What are the key limitations, uncertainties, and open challenges in transferring, deploying, and operating AI-based anomaly detection models in heterogeneous and distributed electric vehicle charging infrastructures?
- How can the explainability and transparency of LSTM-based anomaly detection systems for Electric Vehicle Supply Equipment (EVSE) be enhanced to increase trust and acceptance among users and operators?

By addressing these questions, this thesis aims to provide insights into the technical implementation and practical deployment of anomaly detection systems in EVSE environments, establishing a foundation for the subsequent analysis and discussion chapters.

1.4 Structure of the Thesis

This thesis is divided into ten chapters, each building upon the previous one to explore the use of AI-based methods—specifically Long Short-Term Memory (LSTM) architectures—for anomaly detection in Electric Vehicle Supply Equipment (EVSE).

The introduction presents the motivation and relevance of the topic, outlines the role of anomaly detection in the context of EV infrastructure, and formulates the central research questions. Additionally, it provides an overview of prior AI-based approaches and concludes with the structural outline of the work. The theoretical foundations necessary for the thesis are summarized, including the basics of anomaly detection, a short introduction to tree-based models such as Decision Trees and Isolation-Forest, and a focused explanation of the LSTM architecture. Related work is presented, covering research on security aspects and anomaly detection mechanisms specific to EVSE, as well as a broader review of AI-based detection methods applied in similar domains. The dataset creation and acquisition process is described, detailing both the CICEVSE2024 datasets and custom setups involving Prometheus monitoring tools. An exploratory data analysis (EDA) follows, intended to uncover key patterns and differences between the CICEVSE2024 and self-generated datasets. The process of data preprocessing and feature selection is discussed, including the identification of relevant features and the use of feature importance methods such as Decision Trees. The LSTM model is described, including its architecture, the training process, and the chosen strategies for hyperparameter tuning. The results of the approach are evaluated, outlining the experimental setup, evaluation metrics, and key outcomes of the applied anomaly detection system. A detailed discussion of the findings in relation to the research questions highlights limitations and potential practical applications. Finally, the work concludes with a reflection on future directions of this research.

This structure ensures a logical and methodical development of the topic, from theoretical foundations to implementation, evaluation, and critical interpretation.

2 Theoretical Background and Core Concepts

2.1 Fundamentals of Anomaly Detection

In the previous chapter, a closer look was taken at what Artificial Intelligence is and how useful it can be in the field of EVSEs. In this chapter, the focus is on the area of Anomaly Detection and its fundamental concepts.

Anomaly Detection is the process of analyzing data patterns to identify outliers. Data points deviate significantly from the norm. This technique can be applied in a wide range of domains, including credit card fraud detection, insurance and healthcare monitoring, intrusion detection in cybersecurity, fault detection in safety-critical systems, military surveillance, and also in the field of EVSEs [16].

Outliers are data points that differ significantly from the majority of the data. In the paper by Hodge and Austin, they define outliers as "data points that are inconsistent with the rest of the data". The paper also includes some informative figures that illustrate the different types of outliers and how they can be detected [17].

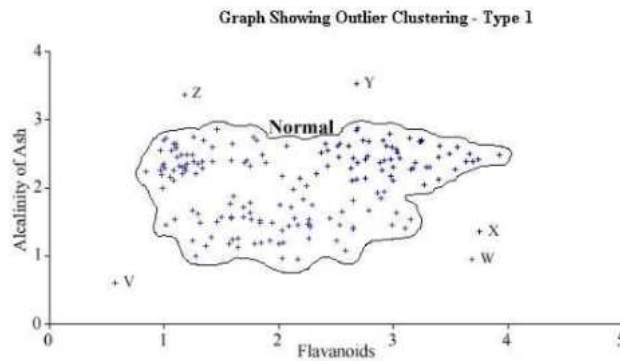


Figure 5: Data distribution classified by Type 1: Outlier Clustering [17].

Type 1 is the most common form of outlier, data points are generally clustered around a central value, but a few points differ significantly from the rest. This can be seen in the Figure 5, where most data is centered, yet a few outliers deviate clearly.

Such outliers can be detected using a simple Z-score algorithm, where the mean μ and the standard deviation σ of the data are calculated. The Z-score is computed using the formula: $Z = \frac{X - \mu}{\sigma}$. Estimated Z-scores greater than 3 or less than -3 can be considered anomalies. These anomalies should be given increased attention during data analysis. Additionally, anomalies can be contextualized by parameters such as time or location to improve their identification [18].

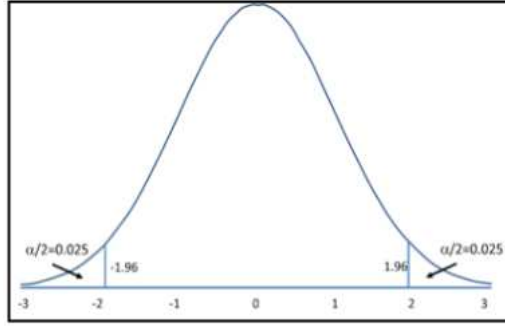


Figure 6: Z-score with critical region [18]

Before the Z-score can be calculated, extensive data collection and preprocessing are required. This forms the first step in the Anomaly Detection process. The data must be cleaned, normalized, and transformed to ensure that it is suitable for analysis. This may involve removing duplicates, handling missing values, and scaling the data to a common range.

After the data is preprocessed, feature engineering is performed to extract relevant features from the data. Feature engineering involves selecting and transforming the data and can sometimes also include the creation of new features that improve the performance of the anomaly detection model. Data reduction is also a relevant step in this process, where the data is reduced to a smaller set of features that are most relevant for the anomaly detection task [19].

Once data collection and preprocessing are completed, a suitable model must be selected for the specific anomaly detection case. Before analyzing the different models, it is important to examine the typical challenges associated with anomaly detection.

As previously mentioned, detecting anomalies requires analyzing data patterns to identify outliers. Determining the correct boundary between normal and abnormal data is challenging and highly dependent on the context.

The EVSE infrastructure changes rapidly. This happens not only due to new technologies, charging stations also change with every new charging process and through communication with other stations in the network. This variability makes it difficult to define a static outlier threshold. Therefore, it is essential to adapt the anomaly detection model to the current state of the EVSE network and the ongoing charging processes. This enables dynamic detection capabilities that remain effective under all conditions in a highly dynamic EVSE environment [16].

2.2 Data Processing and Visualization Tools

This adaptive approach to anomaly detection ensures that detection mechanisms remain robust and reliable, even as EVSE infrastructures and their operational conditions continue to evolve. This chapter introduces the data processing and visualization tools used to implement such adaptable anomaly detection strategies.

To convert the data into more useful formats, *Pandas* was used, a fast, powerful, and easy-to-use open-source data analysis tool for Python. It provides expressive data structures such as

DataFrame and Series for working with structured and time series data [20].

`pandas` was primarily used to convert the data, which was originally stored in CSV files, into PARQUET files. Additionally, it allowed manipulation of the dataframes by dropping cells, adding information such as attack or benign tags to almost 7000 rows. This preprocessing step was essential to clean the data before applying machine learning algorithms [20].

To create diagrams that help better understand the data, a popular plotting library in Python called *Matplotlib* was used. Specifically, the `matplotlib` module was employed, which is a state-based interface to the `matplotlib` library and provides a MATLAB-like way of creating static, animated, and interactive visualizations in Python.

`matplotlib` is well-suited for quickly prototyping visualizations thanks to its simple and readable syntax. This allowed the creation of a wide range of plots, including Line Plots, Scatter Plots, Bar Charts, Histograms, and Heatmaps.

These visualizations enabled a better understanding of the data structure, identification of patterns, and preparation of the dataset for further processing.

In addition to `matplotlib` and `pandas`, the open-source Python library *Seaborn* was also used. Seaborn is especially known for creating statistical graphics and is built on top of the `matplotlib` library. It provides a high-level interface for drawing informative and attractive statistical plots.

Seaborn is closely integrated with `pandas` data structures, which facilitates the visualization of entire datasets. This integration enables the creation of complex visualizations with minimal code. Key features of Seaborn include:

- Plotting of complex relationships between multiple variables.
- Automatic aggregation and statistical estimation (e.g., means, confidence intervals).
- Thematic styling and enhanced color palettes for better aesthetics.

For these reasons, Seaborn was a perfect addition to the `pandas` and `matplotlib` libraries in the data analysis workflow [21].

2.3 Introduction to Decision Trees, Isolation Forest and Long Short-Term Memory

This chapter provides a brief introduction to several machine learning models that are categorized as supervised learning models. These models are also the most relevant for the project. The Decision-Tree-Classifer is one of the most well-known and powerful models in various fields. It works by creating a tree-like structure with nodes and branches, where each node represents an attribute or feature, and each branch represents the characteristic of the attribute [22].

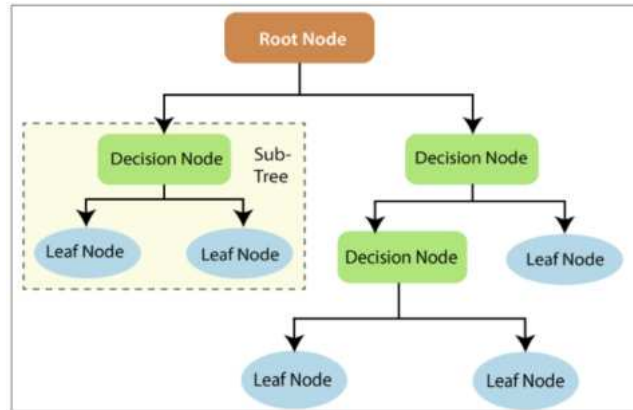


Figure 7: Decision Tree [22]

The decision rule is not a black box and can be easily interpreted. It compares the value of the feature with a threshold and then decides which branch to follow. It is not only used in Machine Learning, but is also common in fields such as image processing and pattern recognition, due to its simple analysis and high accuracy. Despite these advantages, it has a high risk of overfitting, especially with small datasets [22].

The **Random-Forest** is an ensemble method that combines multiple decision trees that work independently of each other. Each decision tree is created using a random vector to improve variance and reduce overfitting.

The final decision is made by a majority vote of all the trees together. This method is more robust against overfitting and can handle large datasets with high dimensionality. Two key factors influence the performance of the Random-Forest: the correlation between the trees and the strength of the individual trees [23].

The **Isolation-Forest** is an innovative approach for anomaly detection. While conventional methods typically create a model of "normal" behavior and classify deviations from it as anomalies, the Isolation-Forest follows the principle of directly isolating anomalies [24].

The explanation for this is that anomalies are characterized by their rarity and by being significantly different in their attributes. Therefore, they are easier to separate from the majority. This model brings advantages such as robustness with regard to dimensionality and irrelevant features. On the other hand, swamping is one of the disadvantages, where normal instances that are too close to anomalies can prevent anomalies from being isolated and thus make them harder to detect [24].

A **Long Short-Term Memory (LSTM)** is a special type of Recurrent Neural Network (RNN) designed to model sequential data and long-term dependencies. Especially the long-term de-

dependencies have high priority in this project, where the focus is on detecting anomalies across extended periods. LSTMs use memory cells and gating mechanisms to store information over long time steps reliably. Despite their many advantages, they come with many different parameters, making them difficult to tune. Additionally, they represent a black-box model, meaning it is hard to understand why the model made a certain decision [25].

Machine Learning Model	Advantages	Disadvantages
Decision Tree	Easy to understand and interpret	High possibility of overfitting
Random-Forest	Less overfitting than a single decision tree; handles high-dimensional data	Not well-suited for small datasets
Isolation-Forest	Robust to high-dimensional and irrelevant features	Swamping effect in the presence of nearby normal instances
Long Short-Term Memory (LSTM)	Effective at modeling long-term dependencies	Black-box nature and complex parameter tuning

Table 2: Comparison of supervised learning models with advantages and disadvantages [22][23][24][25].

2.4 Overview of Long Short-Term Memory Architecture

In this chapter, I want to provide a detailed overview of the LSTM architecture, as it is the most frequently used and most complex model in my work.

Long Short-Term Memory (LSTM) was developed in 1997 by Sepp Hochreiter and Jürgen Schmidhuber as a solution to the vanishing gradient problem that occurs in traditional Recurrent Neural Networks (RNNs) [25]. It allows the network to learn dependencies over much longer time sequences, which is particularly important in my datasets, where attacks can be insidious and not immediately obvious.

This addresses one of the core limitations of traditional RNNs, which struggle to learn long-term dependencies due to the vanishing gradient problem. When training these networks, the error signals either explode, causing oscillating weights and unstable learning or vanish, preventing the network from learning dependencies over extended time periods. Standard RNNs can typically bridge about 5 to 10 time steps, whereas LSTMs are capable of learning dependencies over 1,000 or more time steps [25], LSTMs process inputs sequentially, with a constant runtime of $\mathcal{O}(1)$ per time step.

The LSTM architecture consists of six different internal states that are processed in a specific sequence to achieve this goal.

- **Forget Gate:** The forget gate calculates a dynamic value between 0 and 1 that scales the previous cell state. A value of 0 means "forget everything" and a value of 1 means

”keep everything”. This allows the network to selectively forget information that is no longer relevant.

- **Input Gate:** The input gate decides how much of the new information should be added to the new cell state at the current time step. It is a kind of protective mechanism that tries to prevent irrelevant inputs from being added to the cell state.
- **Candidate Values:** The candidate values are the new information that get in the form of squashed values that get written into the memory cell.
- **Cell State Update:** The cell state is updated by simultaneously scaling down the existing state via the forget gate and adding in the gated candidate values. This allows the network to maintain a stable memory while also incorporating new information.
- **Output Gate:** The output gate determines which parts of the cell state should really become the output of the LSTM and protects the network from sharing unnecessary information too early.
- **Hidden State:** The hidden state is the final output of the LSTM for the current time step and what the output gate allows to be shared with the next time step.

[25]

Machine Learning Model	Advantages	Disadvantages
Long Short-Term Memory	Excellent long-term memory capabilities	Fixed network topology – cannot dynamically adjust memory capacity
	Selective information processing through gates	Complexity – more parameters than simple RNNs
	Stable learning without gradient explosion/vanishing	Modularization challenges – unclear how to optimally structure networks

Table 3: Advantages and disadvantages of LSTM [25].

3 Related Work

3.1 Recent Advances in Cybersecurity and Anomaly Detection for Electric Vehicle Supply Equipment

In the emerging field of EVSE, numerous studies focus either on electric vehicle supply equipment itself or on anomaly detection methods. However, the specific combination of anomaly detection applied to EVSE is not yet widely explored. Therefore, the paper [26], titled "Enhancing EV Charging Station Security Using a Multi-dimensional Dataset", is highly relevant to this research. The authors present a dataset used for anomaly detection that includes similar features such as hardware events, power consumption, and other HPC-related data. The paper focuses on various attack scenarios and attempts to detect them using several machine learning models.

Another relevant contribution in this field is the paper [1], titled "Regression-Based Anomaly Detection in Electric Vehicle State of Charge Fluctuations Through Analysis of EVCI Data" which not only highlights the importance of anomaly detection in EVSE but also provides a comprehensive overview of a realistic charging station scenario. The setup includes four different charging boards, six charging ports, and a battery energy storage system (BESS) handling various types of EVs. The study monitors the state-of-charge (SoC) values of EVs, compares them with expected values, and detects outliers that may indicate anomalies. These detections are performed using machine learning methods within a dynamic environment.

In the context of EVSE as part of critical infrastructure, recent research highlights its role as a potential attack vector within complex smart grid environments. Studies report threats such as Denial-of-Service (DoS), Man-in-the-Middle (MitM), spoofing, and data manipulation. Modern approaches increasingly integrate anomaly detection methods using machine learning to identify and mitigate such attacks at an early stage. While not all technical implementation details are relevant here, the findings emphasize the need for advanced security mechanisms to ensure the integrity and availability of EVSE systems [6].

3.2 General Anomaly Detection Methods and Foundations

In the field of anomaly detection, the paper "Anomaly Detection: A Survey" [16] serves as a central methodological foundation. It provides a systematic classification of approaches, including cluster-based and statistical methods, and explains their respective strengths, weaknesses, and typical use cases. Furthermore, it differentiates between various types of anomalies, such as point anomalies and contextual anomalies, and discusses the most suitable detection strategies for each category. This makes it an indispensable reference for modern anomaly detection procedures.

Building on this established methodological framework, a solid basis is created upon which

more recent approaches, particularly deep learning models and methods tailored specifically to EVSE, can be developed. This also includes the LSTM-based anomaly detection presented in this work. Therefore, the paper highlights both the breadth and depth of anomaly detection research and ensures that specialized techniques are embedded within a well-founded conceptual context [16].

Another relevant contribution in the broader context of cybersecurity is presented by Rahman et al. (2024), titled "ChronoCTI: Temporal Pattern Extraction from Cyber Threat Intelligence Reports" [27]. The authors propose an automated pipeline that leverages modern machine learning techniques and natural language processing to extract recurring, time-based sequences of malicious activities from unstructured reports. In contrast to traditional approaches that primarily focus on static indicators such as IP addresses or malware signatures, this work emphasizes the analysis of how an attack evolves over time.

The findings demonstrate that incorporating the temporal progression of malicious activity into detection processes can significantly enhance threat identification. For EVSE cybersecurity, this perspective is particularly valuable, as certain attacks on charging infrastructure may unfold gradually, with patterns becoming apparent only when their temporal dimension is considered. Such an approach aligns closely with the strengths of LSTM-based anomaly detection, which is well-suited to capturing dependencies over time and can thereby improve the detection of subtle, long-running attacks.

3.3 Performance Comparison of Deep Learning Approaches for Electric Vehicle Supply Equipment Anomaly Detection

In the following, it is important to summarize which machine learning and deep learning algorithms have shown promising performance in the field of anomaly detection for EVSE. Since the data involves time series and hardware-related measurements, the paper [28], titled "Deep Learning-based Intrusion Detection System for Electric Vehicle Charging Station", provides a valuable overview of suitable deep learning models in this context.

The authors compare a Deep Neural Network (DNN) with a Long Short-Term Memory (LSTM) model and conclude that the LSTM architecture achieves high accuracy over 99%, after only a few training epochs. This supports the later decision in this thesis to select the LSTM model for better performance and anomaly detection in EVSE environments.

However, the LSTM is only as strong as the significance of its features. Therefore, a good feature selection for the LSTM algorithm is essential to achieve an effective deep learning approach for anomaly detection. The paper by Kottilingal (2024), titled "Comparative analysis of feature selection techniques for lstm based network intrusion detection models" [29] offers a systematic comparison of different feature selection methods in combination with an LSTM network for intrusion detection. The authors evaluate both the detection accuracy and the efficiency of the LSTM model when applying various feature selection strategies. The central

finding is that reduced feature sets, such as those selected using tree-based methods, can achieve almost the same detection accuracy as larger sets, while significantly lowering model complexity and computational requirements. This aspect is particularly relevant for resource-constrained environments such as embedded EVSE systems.

In the context of EVSE, this means that implementing a practical and high-performance anomaly detection system requires not only choosing a suitable deep learning model (e.g., LSTM) but also effectively reducing the input dimensionality through feature selection. As already shown in [26], even with EVSE-specific data, selecting a smaller but relevant subset of hardware and kernel events can maintain detection accuracy while significantly reducing system load. The work by Kottilingal supports this approach with experimental results and provides practical recommendations for combining deep learning architectures with feature selection, making it a valuable reference for objective method comparisons in smart charging anomaly detection.

4 Data Acquisition

4.1 Data Sources for Electric Vehicle Supply Equipment Anomaly Detection

In line with the approach described in the paper [26] "Enhancing EV Charging Station Security Using A Multi-dimensional Dataset : CICEVSE2024", Hardware Performance Counter (HPC) kernel events and tracepoint events were recorded to classify anomalies in Electric Vehicle Supply Equipment (EVSE) data. For efficient monitoring, the `perf` tool was utilized, a robust Linux performance analysis utility capable of capturing a variety of performance data, including hardware events, software events, and tracepoints [30]. This tool offers flexible and effective monitoring of system performance and is instrumental in diagnosing issues efficiently.

To perform the data acquisition on a Raspberry Pi Zero W, the `perf_exporter`[31] was used to record up to 900 tracepoint events, which were selected based on the CICEVSE2024 dataset. Initially, a Python script was developed to capture kernel and HPC events. When expanding to about 110 events, the runtime of the developed script increased to approximately 9 seconds, which proved insufficient for real-time analysis. Consequently, the internal workings of the `perf` tool were investigated to understand how it manages specialized events and optimizes recording.

As detailed in "Linux `perf` event Features and Overhead" [32], `perf` supports multiplexing, allowing the measurement of more events than there are available hardware counters, though the cost of accuracy loss and increased system overhead. Recent updates can result in resource usage exceeding 20%, compared to approximately 2% in older tool versions [32]. To minimize these inaccuracies, events were grouped per CPU, exploiting up to 8 available counter registers and limiting each group to a maximum of 6 events.

For the monitoring interval, recommendations from the literature [33] addressing multiplexing and accuracy trade-offs were followed. The suggested minimum monitoring interval is around 10 ms for reliable results. Balancing the number of counter registers and striving for good measurement accuracy, groups of 6 hardware events per group were configured, with a monitoring interval of 0.1 seconds. This configuration yielded data with sufficient accuracy to detect anomalies in the EVSE dataset [33].

To further optimize efficiency and accuracy, the data acquisition script was re-implemented in Go. The Go programming language offers low-level hardware access and, with available libraries, integrates more seamlessly with the `perf` tool compared to Python.

Additionally, the script exports all monitored hardware events from `perf` into a Prometheus database, enabling centralized and convenient access to all collected data. This setup streamlines subsequent analysis, visualization, and anomaly detection activities.

Instead of including the full script, its functionality can be summarized as follows. The script collects system performance metrics using the Linux `perf` tool and filters the relevant events specified in a YAML configuration file. It then formats these metrics in a way that is compatible with Prometheus and makes them available through an HTTP metrics endpoint. This allows the performance data to be monitored continuously and integrated into observability platforms in a structured manner as shown in Listing 1.

4.2 Integration of Prometheus

In the research on which database would be suitable for centralized data storage and analysis, Prometheus played a central role. Not only is it an open-source project [34], but it is also well known in the industry as a reliable monitoring solution for complex systems. It has a large community, along with comprehensive documentation and strong support, which are essential for maintaining a stable IT environment.

Another reason for choosing Prometheus is its seamless integration with Grafana for visualization, as well as its powerful PromQL query language. Additionally, Prometheus can store metrics in a time series database, which is particularly important for the detection systems [35].

Prometheus consists of several components that are required for its operation. The core of the system is the Prometheus server, which collects and stores metrics and provides access to them through the Prometheus Query Language (PromQL).

Client libraries are directly integrated into the application code and expose a client-server interface. The Prometheus server periodically fetches data from them, by default every 15 seconds, this interval is configurable [36].

As explained in the previous subsection, Prometheus collects data via metrics exporters 1. After the data is collected and stored centrally in the Prometheus time series database, a Python script was developed that allows exporting the data within a defined time range and with specified time steps.

This export functionality is essential for enabling further analysis, including exploratory data analysis, using external tools such as Python or statistical software. The following paragraph presents the Python script developed to export Prometheus data into a structured CSV format, enabling its direct use in subsequent analytical workflows.

The Python script is designed to systematically retrieve and organize time-series metrics from a Prometheus monitoring server, facilitating subsequent analysis and interpretation. The workflow begins by defining a specific time range for the data of interest and partitioning this interval into smaller, manageable chunks, which helps prevent overwhelming the server with excessively

large queries and ensures efficient data retrieval. For each feature specified in an external configuration file, the script constructs appropriate Prometheus queries, attempting first to obtain the standard metric and, if unavailable, a performance-prefixed variant. The retrieved data are then parsed and organized according to their respective timestamps, forming a structured representation of the system's behavior over time. Finally, the script outputs these metrics in a wide-format CSV file, where each row corresponds to a timestamp and each column corresponds to a specific metric. This approach not only allows for straightforward integration with downstream data analysis or visualization workflows but also ensures reproducibility, scalability, and clarity in the presentation of high-dimensional monitoring data.

5 Exploratory Data Analysis

5.1 Holistic Dataset Comparison

So far, an overview of the relevance and challenges involved in securing EVSE environments has been provided. The fundamental knowledge required before developing an anomaly detection system has been discussed, insight into related work in this area has been gained, and the ways in which it can inform the current approach have been explored.

The acquisition of data from EVSE environments and the use of Prometheus to collect and store this data have also been examined.

All of these chapters and subchapters are essential for understanding the context of this work and for showing how it is possible to develop an anomaly detection system for EVSE environments based on the analysis of hardware events and the collected data.

In this chapter, the focus is on the Exploratory Data Analysis (EDA) of the collected data. EDA is a crucial step in understanding the data, identifying patterns, and preparing for further analysis or modeling. Python and its data analysis libraries are used within a Jupyter Notebook environment to visualize and analyze the collected data. The Python libraries utilized for data processing and visualization are explained in more detail in chapter 2.2, providing a foundational understanding of the tools applied in this analysis. Establishing this methodological basis is crucial, as the quality of the results in the following chapters significantly influences how well anomaly detection using machine learning models will perform and how quickly they will learn.

All results will be compared with the dataset CICEVSE2024 to guarantee a meaningful outcome and to allow the transfer of results to other charging station setups.

The framework conditions of both datasets, the dataset CICEVSE2024 and the self-collected dataset, are analyzed first.

The paper "Enhancing EV Charging Station Security Using a Multi-dimensional Dataset" [26] and this work pursue the same goal: detecting anomalies through the analysis of HPC and kernel events. Therefore, it is essential to compare the underlying datasets on which the machine learning algorithms are trained. Such a comparison enables validation of the respective approaches and helps determine whether the results can be generalized.

An attempt was made to get as close as possible to the environment of the paper "Enhancing EV Charging Station Security Using a Multi-dimensional Dataset".

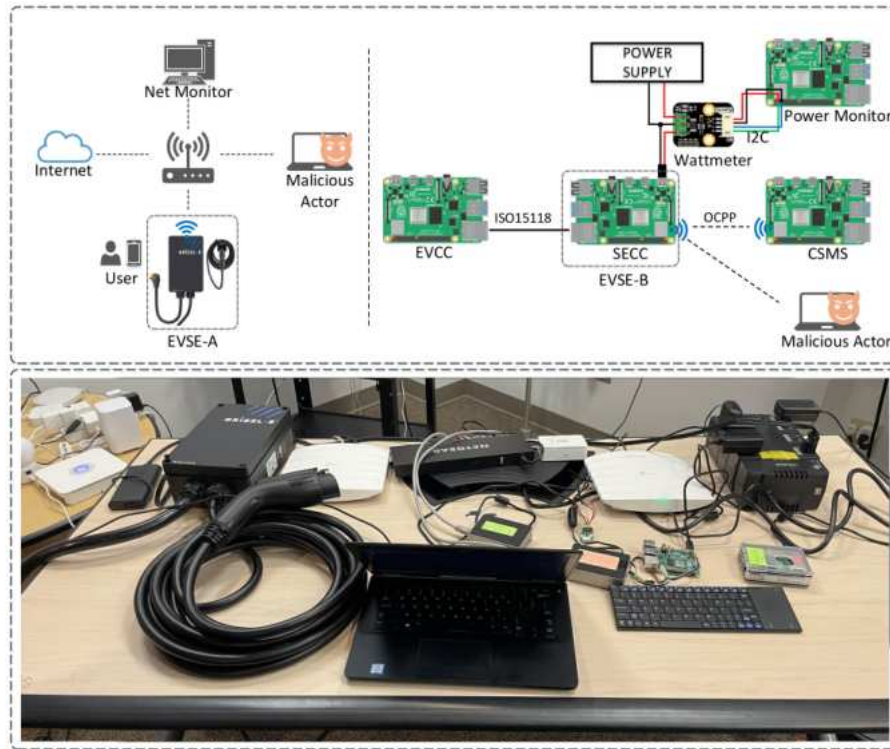


Figure 8: Setup of the Electric Vehicles Charging Station [26]

A Raspberry Pi was also used to represent the EVSE-B, and a wattmeter was employed to monitor power consumption. The combination of HPC events, kernel events, and power consumption measurements will be discussed in future work. As mentioned in chapter 4.1, the `perf` tool was also used to monitor the various events.

Therefore, approximately 900 events can be observed in the dataset, which is comparable to the reference dataset. The complete observation period spans from 2025-07-22T18:00:00 to 2025-07-23T12:52:10, with a sampling interval of 5 seconds, identical to the compared dataset [26]. This results in approximately 8,000 data rows and 905 columns. Importantly, no missing (NaN) values are present, and all measurements were collected using the `perf` tool, producing logically consistent outputs as described in the chapter 4.1.

To ensure comparability, features distributed across multiple CPU cores (e.g., `cpu=0,1,2,3`) were aggregated using appropriate `perf` flags. This not only prevents data loss but also reduces the number of distinct features, thereby aligning the dataset structure more closely with the reference dataset CICEVSE2024.

To establish a clear starting point for the later feature analysis, the selected features presented in the paper "Enhancing EV Charging Station Security Using a Multi-dimensional Dataset:

CICEVSE2024” [26] are first referred to. These selected features provide a foundation for comparison and help guide the own selection process.

Event	Description	Backdoor	Crypto	Dos	Recon
instructions	Number of instructions executed	214.19	943.54	316.65	130.69
cache-misses	Number of cache misses	39.92	397.06	112.83	64.70
exc_taken	Exception taken	86.82	61.42	169.62	44.55
cpu-migrations	CPU Migrations	842.43	-100.00	1425.70	110.05
dTLB-store-misses	Data TLB - Write Misses	39.56	1068.57	101.48	72.97
l1d.cache_wr	Level 1 data cache access - Write	143.87	121.56	321.24	113.14
L1-icache-loads	Level 1 instruction cache access - Read	163.10	538.63	283.92	107.18
l2d.cache_rd	Level 2 data cache - Read	51.47	422.95	151.50	70.75
mem_access_rd	Data memory access - read	118.19	387.56	253.17	91.42
mem_access_wr	Data memory access - write	142.94	119.96	322.17	114.86
kmem_kfree	Kernel memory freeing event	149.06	67.66	2087.07	187.25
net_dev_xmit	Network device transmission event	27.42	33.79	172.88	59.64
qdisc.dequeue	Dequeue event	31.47	42.66	261.16	88.42
raw_syscalls_sys_enter	System call entry (raw) event	107.66	79.70	2.35	44.13
irq_softirq_raise	Software interrupt - Raised	90.29	65.72	1573.65	109.45
sched_migrate_task	Task migration event in the scheduler	670.49	-100.00	1091.74	78.18
sched_switch	Task switch event in the scheduler	42.10	39.22	332.82	24.59
syscalls_sys_enter_close	System call entry for close syscall	421.15	273.72	15.58	231.63
syscalls_sys_enter_read	System call entry for read syscall	44.32	30.19	37.09	56.21
syscalls_sys_enter_write	System call entry for write syscall	119.30	35.73	57.34	76.92

Figure 9: Selected Hardware Performance Counter and Kernel Features from the paper ”Enhancing EV Charging Station Security Using a Multi-dimensional Dataset” [26]

The CICEVSE2024 dataset itself consists of the same 905 event types and comprises approximately 8,500 rows. Timestamps are recorded as floating-point values representing the time of each measurement. Like the self-collected dataset, it was collected using the **perf** tool, ensuring high precision and consistency across all recorded events [26]. Out of the entire dataset, only four rows contain NaN values across all features.

At first glance, one might assume a high level of noise when considering the Z-Score. However, in this case, the values do not represent random noise or errors but instead correspond to peak values during attacks. Thus, the actual noise level in both datasets is very low, which can also be observed later in the direct feature comparison. Such rows with high peaks can therefore be considered meaningful signals rather than noise.

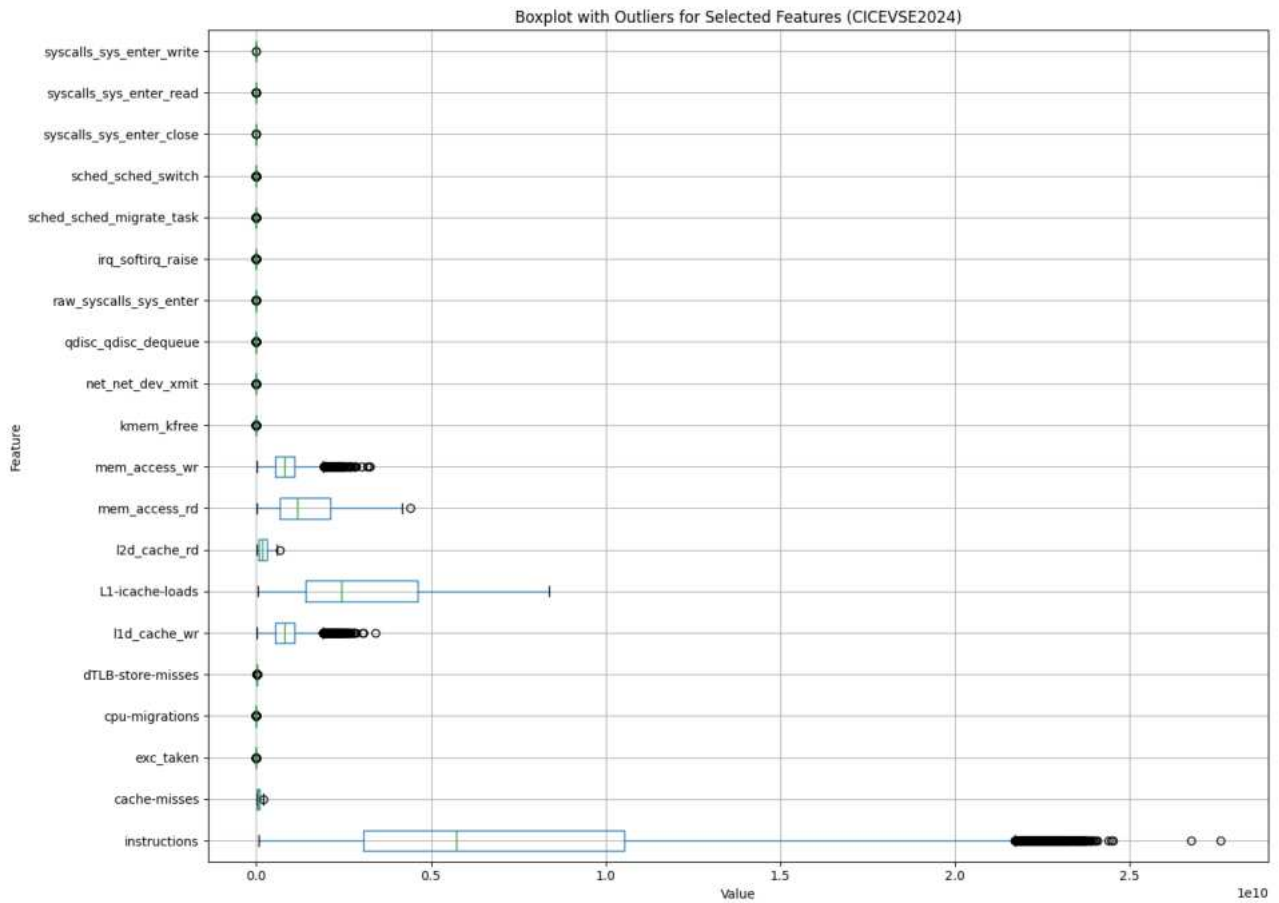


Figure 10: Boxplot of the CICEVSE2024 Dataset Features.

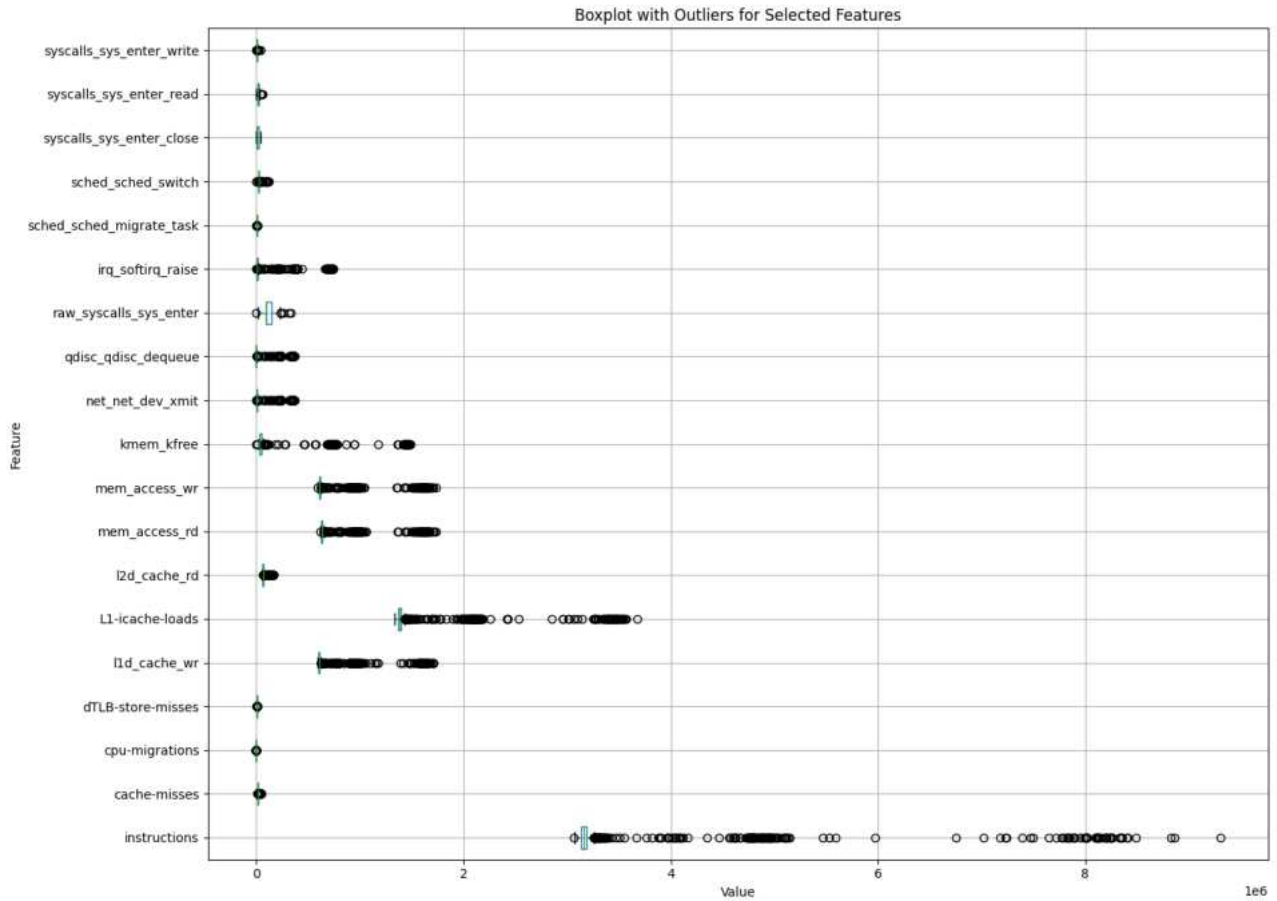


Figure 11: Boxplot of the self-collected Dataset Features.

When comparing the two datasets, it becomes apparent that the self-collected dataset exhibits more outliers than the CICEVSE2024 dataset. This is primarily due to the fact that it contains a significantly larger amount of benign data, which makes the attack data stand out more clearly as outliers. Furthermore, the benign data in the self-collected dataset is much quieter and less variable than in the CICEVSE2024 dataset. This reduced variability becomes especially evident in the analysis of individual features, as can be observed in the following graphs using the L1-icache-loads feature as an example.

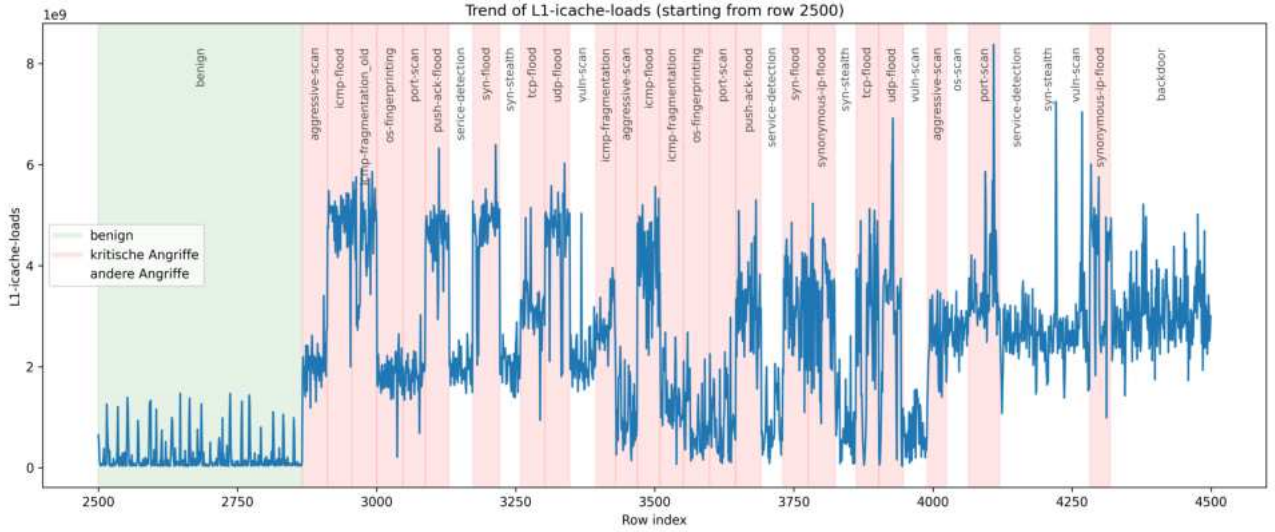


Figure 12: L1-icache-loads timeline (CICEVSE2024).

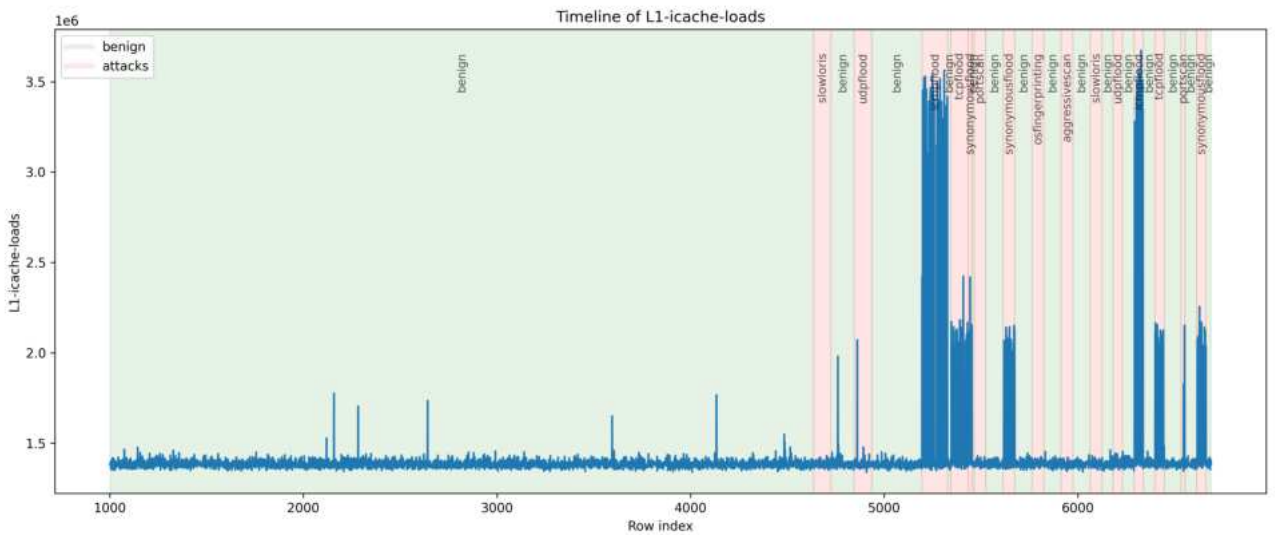


Figure 13: L1-icache-loads timeline (self-collected).

5.2 Feature Comparison of CICEVSE2024 and Self-Collected Datasets

For clarity, we will start as before with the selected features from the paper: "Enhancing EV Charging Station Security Using A Multi-dimensional Dataset: CICEVSE2024" [26]. The remaining features can be found in the appendix, where similar results to those shown here are presented.

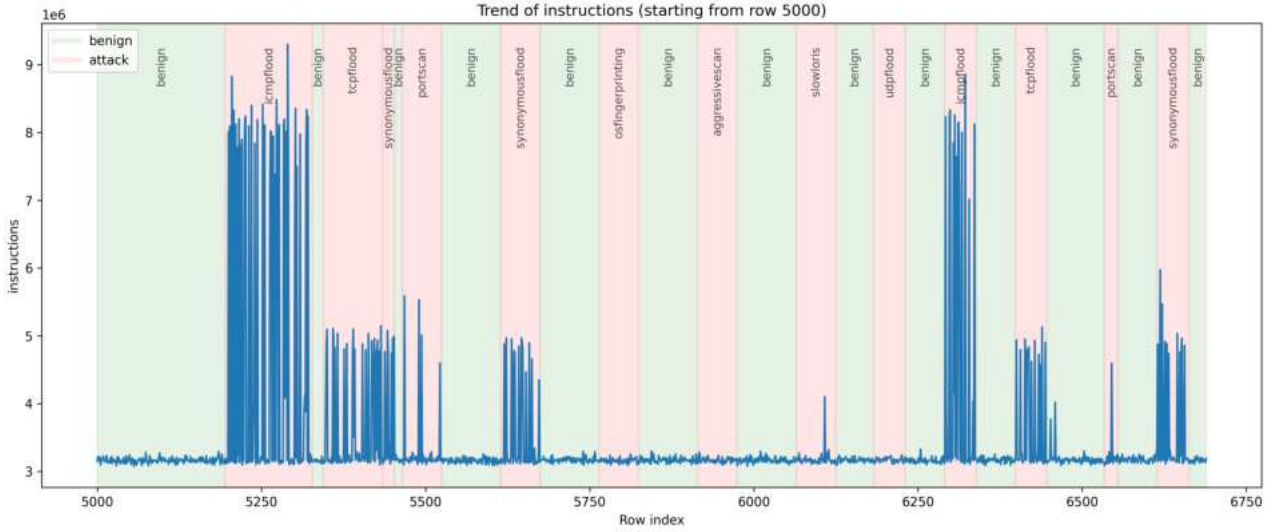


Figure 14: Instruction timeline.

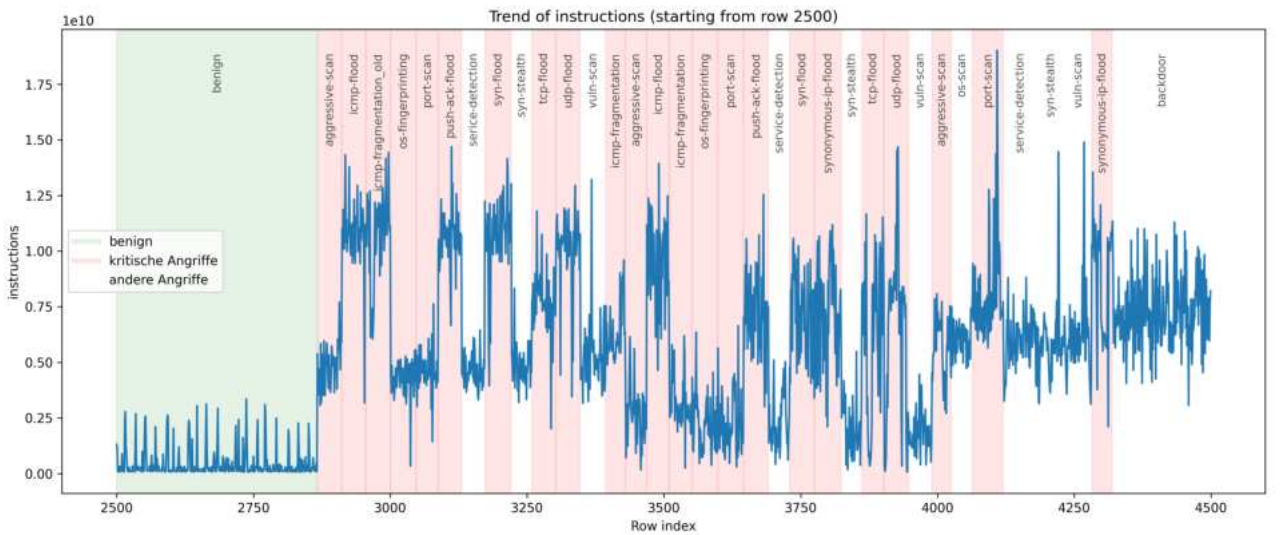


Figure 15: Instruction timeline (CICEVSE2024).

The kernel events `instruction` is used to signal or record noteworthy occurrences within the kernel, such as system calls, interrupts, or state changes, often for the purposes of tracing [37]. In the CICEVSE2024 dataset, this feature exhibits a relatively stable and low level in benign scenarios. However, in flooding attacks such as ICMP-Flood, SYN-Flood, TCP-Flood, and UDP-Flood, a substantial increase in the kernel events metric can be observed. This increase can be attributed to the fact that such attacks overwhelm the target system, resulting in a significantly higher number of interrupts, state changes, and similar operations compared to benign states or lower-impact attacks such as OS-Fingerprinting.

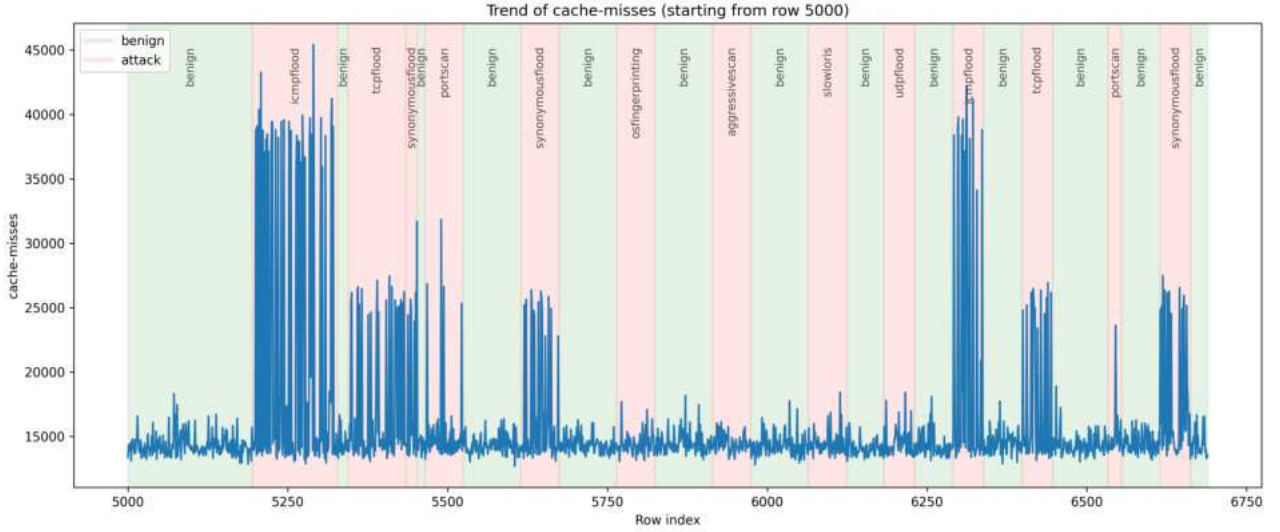


Figure 16: Cache-misses timeline.

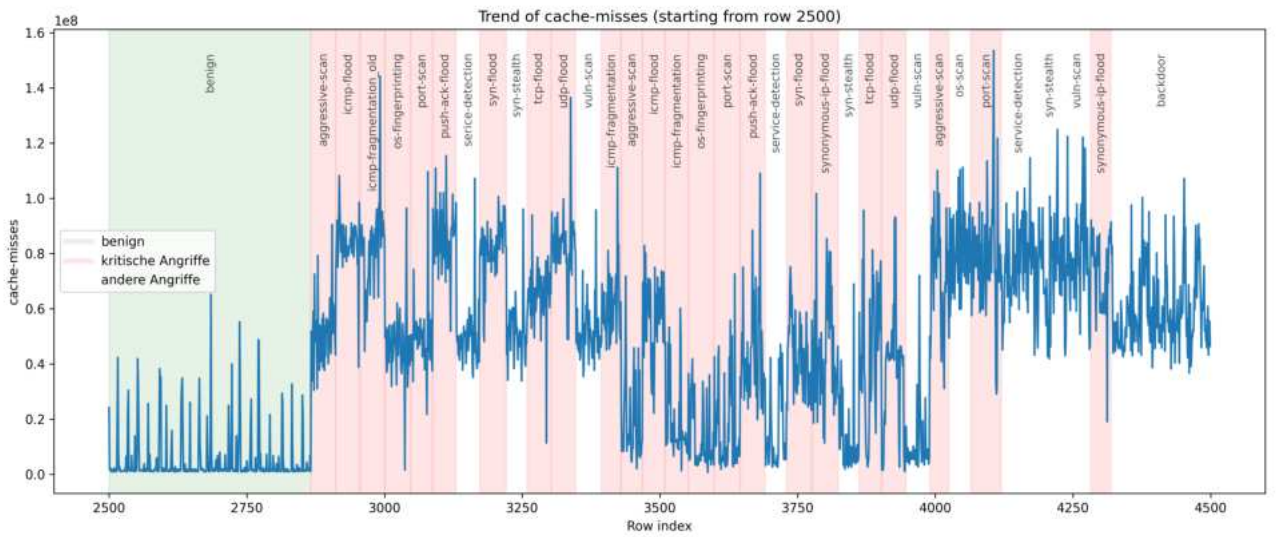


Figure 17: Cache-misses timeline (CICEVSE2024).

The **cache-misses** metric refers to the number of times the CPU is unable to locate the requested data in its cache, necessitating a fetch from a slower memory hierarchy level [38]. In the analyzed dataset, this feature exhibits a very low and stable level in benign scenarios of our measurements, while the benign baseline of the CICEVSE2024 dataset shows considerably higher variability. For both datasets, the highest spikes in **cache-misses** occur during flooding attacks such as ICMP-Flood, SYN-Flood, TCP-Flood, and UDP-Flood, which can be explained by the increased processing demands and the associated stress on the memory hierarchy under these high-throughput conditions. In the CICEVSE2024 dataset, the Aggressive-Scan attack towards the end of the observation period also displays a markedly high value, while the two

preceding Aggressive-Scan instances show values either comparable to the benign baseline or moderately higher. This irregular pattern may be due to varying background processes or system activity during the respective attack executions. However, this remains speculative, as no comprehensive record of all concurrently running programs in the CICEVSE2024 dataset is available.

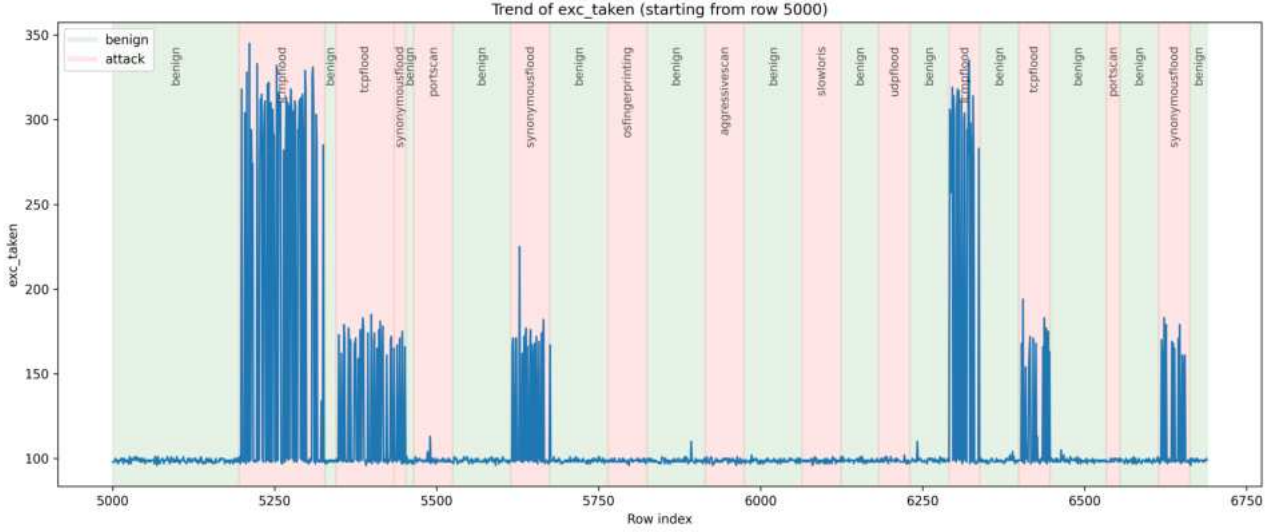


Figure 18: Exc-taken timeline.

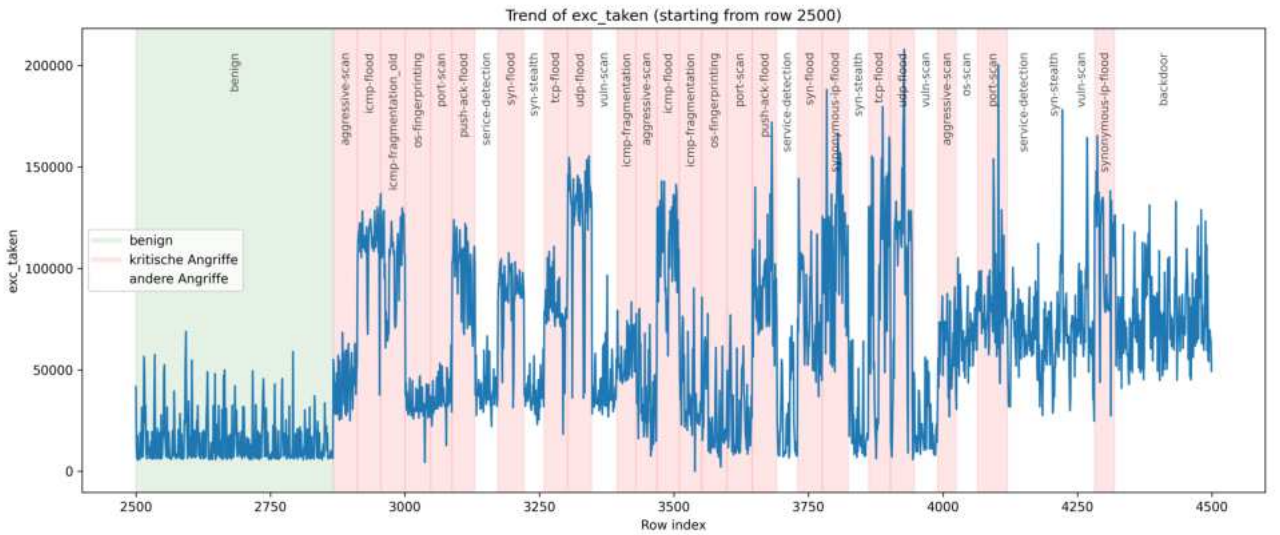


Figure 19: Exc-taken timeline (CICEVSE2024).

The **exc-taken** metric represents the number of **exceptions taken** by the CPU, indicating transitions from normal execution flow to an exception handling routine [39]. In our measurements, benign scenarios show very low and stable values, while the benign baseline of the

CICEVSE2024 dataset exhibits significantly higher variability. In both datasets, the most pronounced peaks occur during flooding attacks such as ICMP-Flood, SYN-Flood, TCP-Flood, and UDP-Flood, reflecting the increased occurrence of exceptions under high-load conditions.

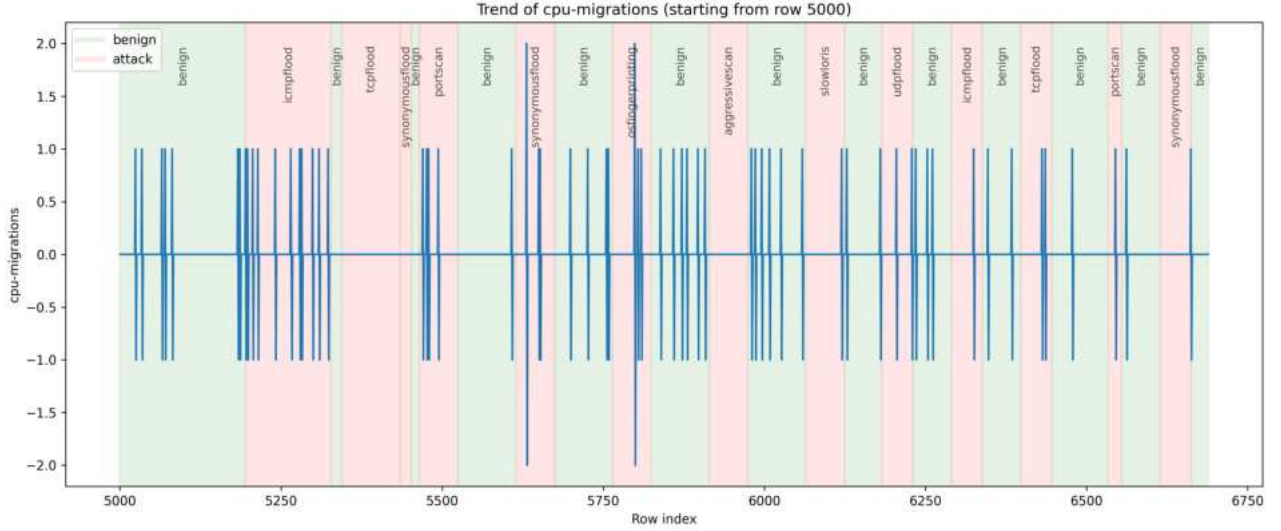


Figure 20: Cpu-migrations timeline.

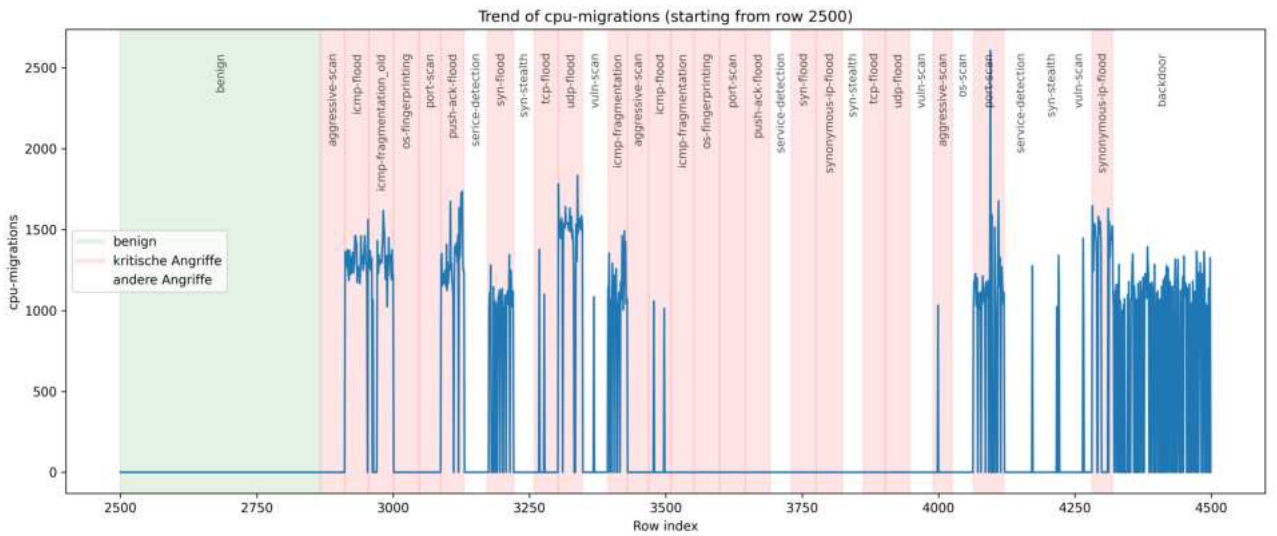


Figure 21: Cpu-migrations timeline (CICEVSE2024).

The `cpu-migrations` metric counts the number of times a process is moved from one CPU core to another, which can occur for load balancing or scheduling purposes [40]. In our measurements, this feature shows limited significance, with only occasional peaks during ICMP-Flood attacks, an effect that is not particularly distinct. In contrast, the CICEVSE2024 dataset reports constant zero values in benign scenarios and very pronounced peaks during ICMP-Flood,

SYN-Flood, and Portscan attacks, making it a potentially valuable indicator for these events. In our data, `cpu-migrations` appears to behave as a continuously increasing counter, with its reset conditions unclear. Our analysis relies on the difference between consecutive readings, which may contribute to the low discriminative power observed. It is possible, though purely speculative, that the CICEVSE2024 dataset employed a different measurement approach, such as directly logging event counts per sampling interval instead of cumulative totals. It is worth considering that our system might exhibit lower load distribution or load-balancing, leading to scenarios where during attacks typically only one CPU core is fully utilized while the others remain relatively idle, resulting in very few migrations between different cores.

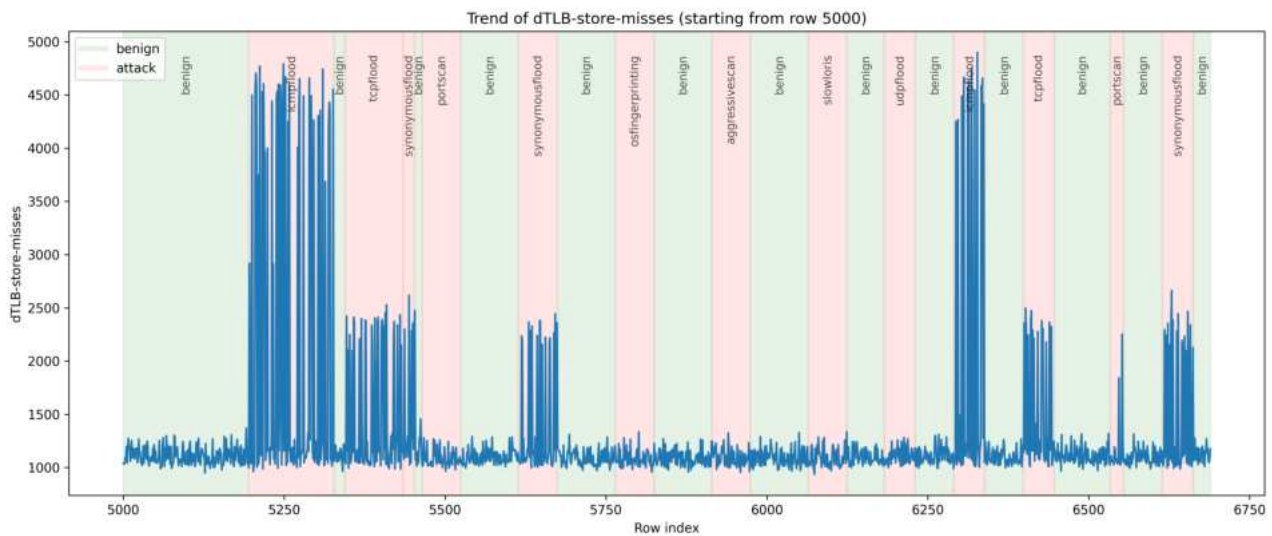


Figure 22: dTLB-store-misses timeline.

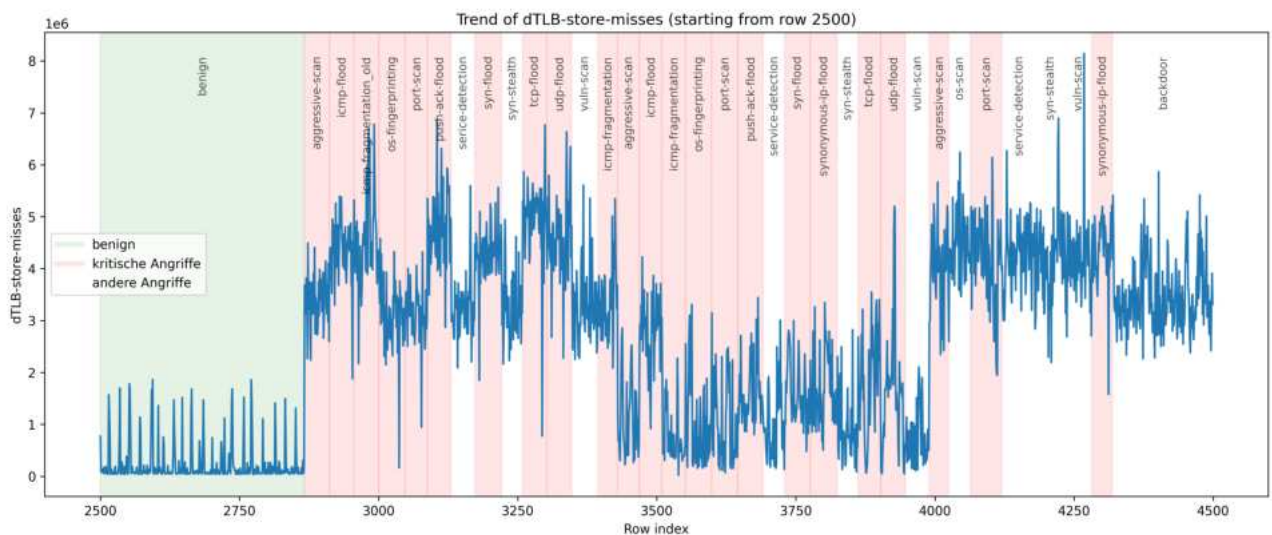


Figure 23: dTLB-store-misses timeline (CICEVSE2024).

The `dTLB-store-misses` metric measures the number of store (write) operations that miss in the data Translation Lookaside Buffer (dTLB), requiring address translation from slower memory structures [41]. In our measurements, benign scenarios exhibit a very stable and low baseline, whereas the benign baseline of the CICEVSE2024 dataset shows greater variability. In both datasets, the highest peaks occur during flooding attacks such as ICMP-Flood, SYN-Flood, TCP-Flood, and UDP-Flood. In the CICEVSE2024 dataset, Aggressive-Scan and Portscan attacks show varying peak intensities: initially strong, then close to benign levels, and later extremely high. This variation may be due to slightly different variants of these attack types being used, although this remains speculative.

It is noteworthy that flooding attacks such as ICMP-Flood, SYN-Flood, TCP-Flood, and UDP-Flood consistently produce the highest peaks across multiple performance metrics, including kernel events, `cache-misses`, `exceptions taken`, `cpu-migrations`, and `dTLB-store-misses`, as well as in the additional analyzed features. These attacks impose a substantial processing load on the target system, resulting in more frequent interrupts, exceptions, cache lookups, and memory translations, alongside increased process scheduling activity. Among these, the ICMP-Flood often exhibits the most pronounced impact, which can be attributed to the fact that each ICMP Echo Request must be fully processed and answered, thereby generating significant CPU and memory subsystem activity.

In contrast, benign scenarios, especially in our own measurements, remain largely stable and low across these features, whereas the CICEVSE2024 dataset displays higher variability in benign baselines. For certain attacks, such as Aggressive-Scan or Portscan, the CICEVSE2024 dataset reveals fluctuating peak intensities, which may be due to slight variations in the specific attack implementations used; however, this remains speculative.

6 Feature Selection

6.1 Data Preprocessing Steps

As already mentioned, for the CICEVSE2024 dataset, it was only necessary to remove four rows containing NaN values to obtain clean and usable data for the initial processing step.

For the dataset generated from the experimental environment, the aim was first to include as much benign-status data as possible. Therefore, a PromQL request was made to retrieve monitoring data from a night when the environment was not used for other tests, as shown in Listing 2.

Subsequently, the attacks were initiated to monitor the behavior of the HPC and kernel events in the environment under attack conditions, which can be seen in Listing 3. Table 4 provides an overview of all attacks carried out during the experiment.

Attack Type	Tool	Description / Parameters
Slowloris	Python script	500 sockets against target host
UDP-Flood	hping3	--flood --udp -p 80
ICMP-Flood	hping3	--flood -1
TCP-Flood	hping3	--flood -p 80
SYN-Flood	hping3	--flood --rand-source -p 80
Portscan	nmap	1000 iterations, -sT scan
OS-Fingerprinting	nmap	1000 iterations, -O scan
Aggressive-Scan	nmap	1000 iterations, -A scan

Table 4: Overview of executed attacks.

Using the generated log file, the correct attack intervals could be assigned to the dataset file for machine learning training and data analysis. Table 5 shows the exact time periods used.

Start Time	End Time	Attack Type
2025-07-23 06:52:35	2025-07-23 07:07:35	Slowloris
2025-07-23 07:27:35	2025-07-23 07:42:35	UDP-Flood
2025-07-23 08:25:45	2025-07-23 08:47:50	ICMP-Flood
2025-07-23 08:50:49	2025-07-23 09:05:49	TCP-Flood
2025-07-23 09:05:49	2025-07-23 09:08:49	SYN-Flood
2025-07-23 09:10:44	2025-07-23 09:20:44	Portscan
2025-07-23 09:35:44	2025-07-23 09:45:44	SYN-Flood
2025-07-23 10:00:44	2025-07-23 10:10:44	OS-Fingerprinting
2025-07-23 10:25:44	2025-07-23 10:35:44	Aggressive-Scan
2025-07-23 10:50:44	2025-07-23 11:00:44	Slowloris
2025-07-23 11:10:21	2025-07-23 11:18:21	UDP-Flood
2025-07-23 11:28:21	2025-07-23 11:36:21	ICMP-Flood
2025-07-23 11:46:21	2025-07-23 11:54:21	TCP-Flood
2025-07-23 12:08:55	2025-07-23 12:12:21	Portscan
2025-07-23 12:22:21	2025-07-23 12:30:21	SYN-Flood

Table 5: Attack intervals during data collection.

These preprocessing steps enabled a meaningful exploratory data analysis, which allowed for the identification of key features for training machine learning models to achieve optimal results. This step is essential, as most models do not perform well with 900 or more features. This is also demonstrated in source [26], where the best results were achieved using only 11 to 20 key features.

Moreover, using too many features would lead to performance issues on the Raspberry Pi in the live system, as it would be required to query and process values for up to 900 features every 5 to 60 seconds.

6.2 Feature Selection Using Decision-Tree-Classifer

Feature selection is a crucial aspect in the preparation of modern deep learning algorithms. However, identifying the most effective method that also performs well in a live environment, such as an EVSE system with limited hardware resources, remains a challenge.

The paper "Enhancing EV Charging Station Security Using a Multi-dimensional Dataset" [26] employed Principal Component Analysis (PCA) to reduce 900 input features for use in an LSTM model. PCA was also experimented with in this study, but due to the nature of this method, all 900 original features must still be requested in the live environment in order to compute the transformed principal components. Executing the required PromQL queries for all 900 features caused the CPU usage on the Raspberry Pi setup to average nearly 50%, which

makes this approach unsuitable for real-time deployment.

Following this limitation, it was necessary to explore alternative feature selection techniques that could offer high performance without requiring access to the full feature set. The paper "Comparative Analysis of Feature Selection Techniques for LSTM-Based Network Intrusion Detection Models" [29] evaluated multiple feature selection strategies to determine the most effective combinations for LSTM-based algorithms.

Model Configuration	Accuracy	F1 Score
LSTM + Without Feature selection	85%	0.872
LSTM + PCA	76%	0.821
LSTM + Univariate Feature Selection	78%	0.836
LSTM + Recursive Feature Elimination	78%	0.837
LSTM + L1- Based Feature Selection	77%	0.829
LSTM + Select Best Feature Selection	83%	0.871
LSTM + Tree Based Feature Selection	82%	0.857
LSTM + Sequential Feature Selection	69%	0.736

Figure 24: Comparison of feature selection methods for Long Short-Term Memory accuracy [29].

According to their findings, the LSTM model without any feature selection achieved the highest accuracy at 85%. However, as previously mentioned, using all 900 features is not feasible in the hardware-constrained environment. The next best accuracies were achieved using LSTM combined with either the "Select Best Feature Selection method" or "Tree-Based feature selection," with only a 1% accuracy drop and an F1-score difference of just 0.014.

Given this background, the Decision Tree algorithm was adopted for feature selection. This allows identification of the most relevant features while maintaining a high level of accuracy and keeping resource usage within system constraints.

With this information, a Decision Tree was created to select the most relevant features for real-time detection in the EVSE system. The Decision-Tree-Classifer from the `sklearn.tree` module was used, as it is easy to use and integrates well with common data science workflows.

In this step, the preprocessed dataset was used to train a Decision-Tree-Classifer for feature selection. First, the categorical labels were converted into a binary format, where benign was mapped to 0 and attack to 1. To prevent data leakage, the columns label, timestamp, and attack were dropped from the feature set before training. The Decision-Tree-Classifer was then trained on the remaining features using a fixed random seed (`random.state=42`). This model primarily served to identify the most relevant features for subsequent analysis and classification tasks.

This approach allowed significant reduction of the number of input features while preserving high detection accuracy and runtime performance.

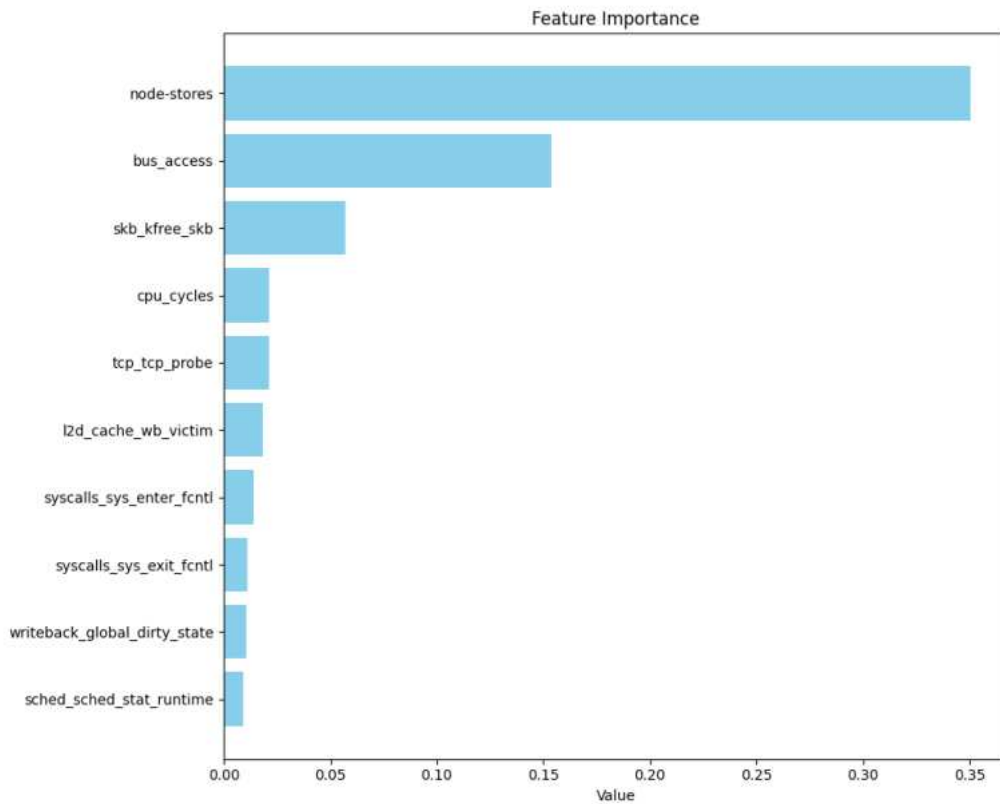


Figure 25: Top 10 most important features selected by the Decision-Tree-Classifer.

node-stores: This event refers to memory store operations targeting specific NUMA nodes in a multi-node system. Monitoring **node-stores** events help analyze memory access patterns and data locality [42], which can be important to detect unusual memory write activity often seen during attacks like UDP-Flood or ICMP-Flood where large volumes of data are rapidly written.

bus-access: This event tracks accesses on the system bus, responsible for communication between CPUs, memory, and peripheral devices [43]. Elevated or abnormal **bus-access** patterns can indicate intensive network or port scanning activities such as Portscans or Aggressive-Scan that generate many small requests to different system resources.

skb_kfree_skb: This tracepoint event in the Linux kernel occurs when socket buffers (SKBs) managing network packets are freed [44]. Tracking this event is crucial for detecting attacks that flood or manipulate network connections, including Slowloris (many half-open connections), SYN-Flood (half-open TCP connections), UDP-Flood, and ICMP-Flood, because these

attacks cause unusually high packet processing and freeing activity.

Using the 20 most important features identified by the Decision-Tree-Classifer, a correlation matrix was created to examine feature redundancy.

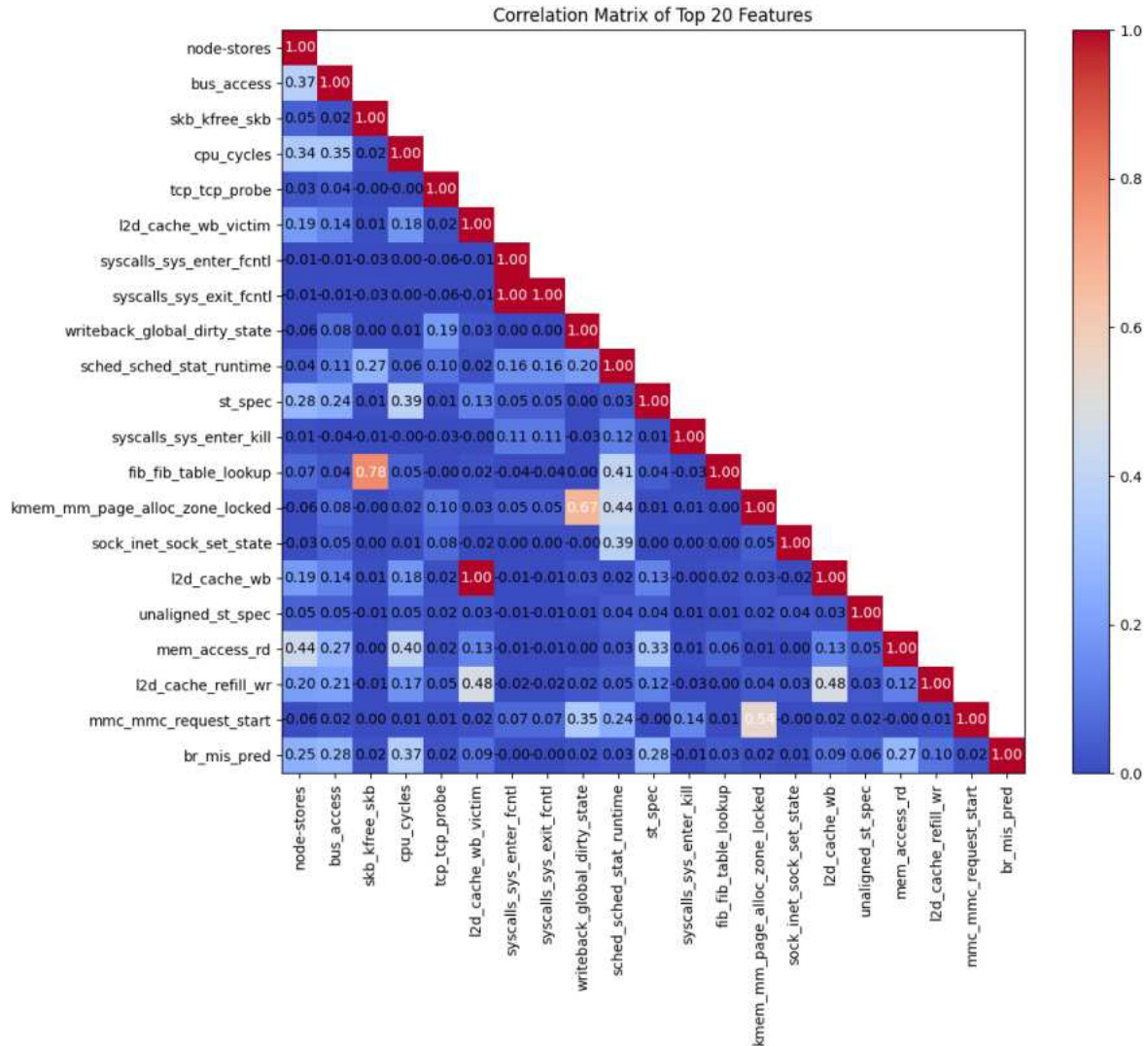


Figure 26: Correlation matrix top Features with Decision-Tree-Classifer.

The correlation matrix in Figure 26 shows that there is almost no correlation between the top 20 features selected by the Decision-Tree-Classifer. This result is highly desirable, as it indicates that the chosen features are largely independent from one another and therefore provide complementary information rather than redundant signals. In practice, this means that the model can learn from a broad spectrum of system behaviors instead of being biased toward multiple highly correlated indicators of the same phenomenon.

For the LSTM-based detection model later, this independence between features is particularly

beneficial. LSTMs are designed to capture temporal dependencies and patterns in sequential data, and providing them with a diverse set of non-redundant input signals increases their ability to generalize across different attack types and benign conditions [45]. Furthermore, reducing feature correlation improves computational efficiency, as the system does not waste resources processing multiple overlapping signals. Overall, the low correlation among the selected features represents an optimal foundation for training the LSTM model in a resource-constrained environment.

Applying the same Decision-Tree-Classifier to the CICEVSE2024 dataset naturally produces different results, as the features exhibit different peaks and benign data patterns, which were already observed earlier.

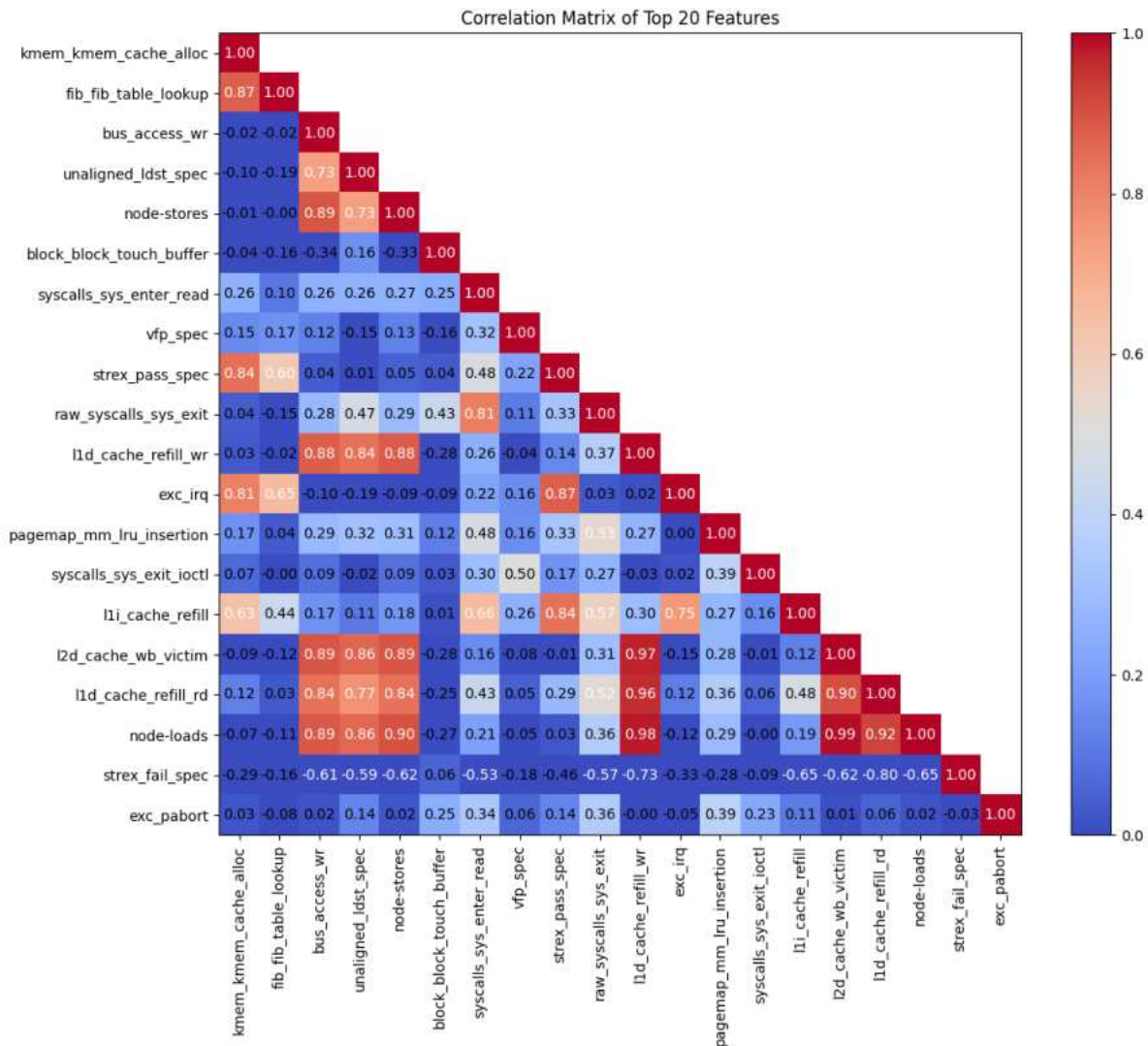


Figure 27: Correlation matrix of top features selected by the Decision-Tree-Classifier on the CICEVSE2024 dataset.

Nevertheless, some correlations among the selected features can still be observed. For example, in both datasets, features such as `node-stores`, `l2d_cache_wb_victim`, and `fib_fib_table_lookup` were consistently selected. The `l2d_cache_wb_victim` event represents write-back operations of the Level 2 data cache for evicted “victim” lines, indicating high memory traffic due to cache evictions under intensive load conditions, such as flooding attacks. The `fib_fib_table_lookup` event is triggered during routing table lookups in the kernel’s forwarding information base, reflecting increased network routing activity common in scanning or flood attacks. Similarly, the `skb_kfree_skb` and `fib_fib_table_lookup` features also show a strong correlation, which is particularly evident in Figure 28. The `skb_kfree_skb` event occurs when socket buffers managing network packets are freed, a critical indicator of network connection flooding or manipulation seen in attacks such as Slowloris, SYN-Floods, UDP-Flood, and ICMP-Floods. These interrelations highlight complementary aspects of network and memory behavior under attack scenarios.

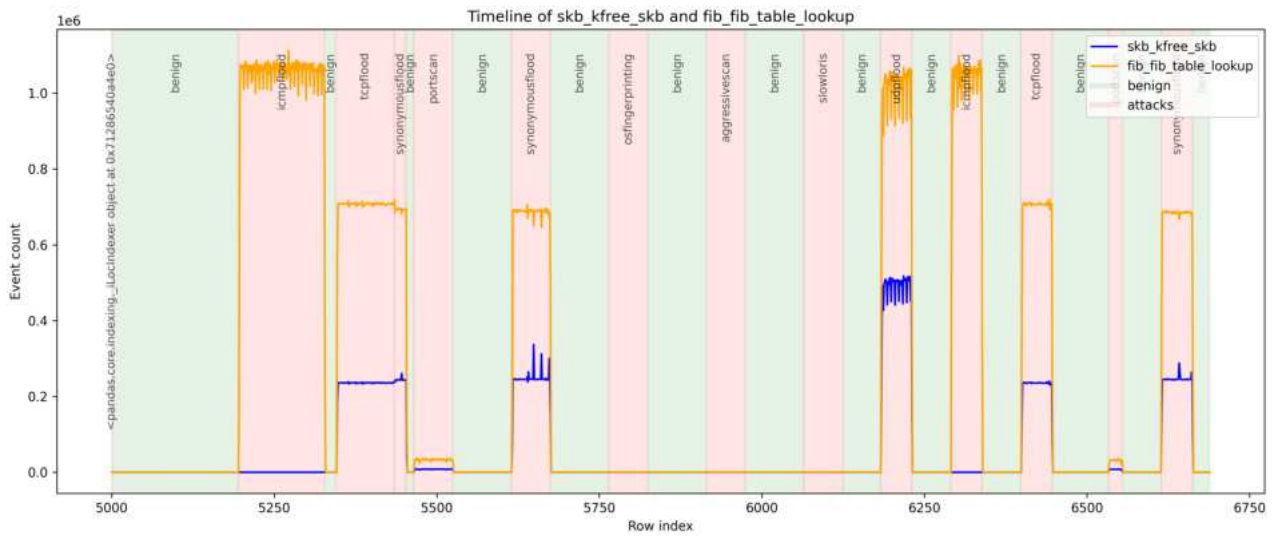


Figure 28: Correlation between `skb_kfree_skb` and `fib_fib_table_lookup` features.

So far, it has been observed that the Decision-Tree-Classifier selects similar features in both datasets. This indicates that it is likely to select comparable features in other, similarly structured setups, even when more than 900 potential features are available. However, the strongest predictive power is concentrated in only a small subset of these features.

It is essential to examine the selected features in the context of the timeline in order to better understand their behavior and patterns over time and to assess whether they are useful for the LSTM algorithm in detecting attacks. In this work, a detailed discussion of every individual feature is not provided; instead, all corresponding figures can be found in the appendix.

As already observed, the `node-stores` feature shows a clear pattern that helps in detecting flooding attacks, as described before, while the `sys_calls_sys_enter_kill` feature records the number of times the Linux kernel processes the “kill” system call (i.e., `sys_kill`), which is used by processes to send signals to other processes. During a Portscan attack, tools such as `nmap` or similar scanners attempt to rapidly open and close many connections across a wide range of ports. This aggressive probing leads to the frequent creation and termination of network sockets and, in some cases, the spawning of short-lived processes for each connection attempt. In summary, the high peaks in `sys_calls_sys_enter_kill` during Portscan attacks reflect the system’s intensive signaling behavior as it deals with the rapid creation and destruction of connections and processes, making it a strong indicator for detecting such attacks.

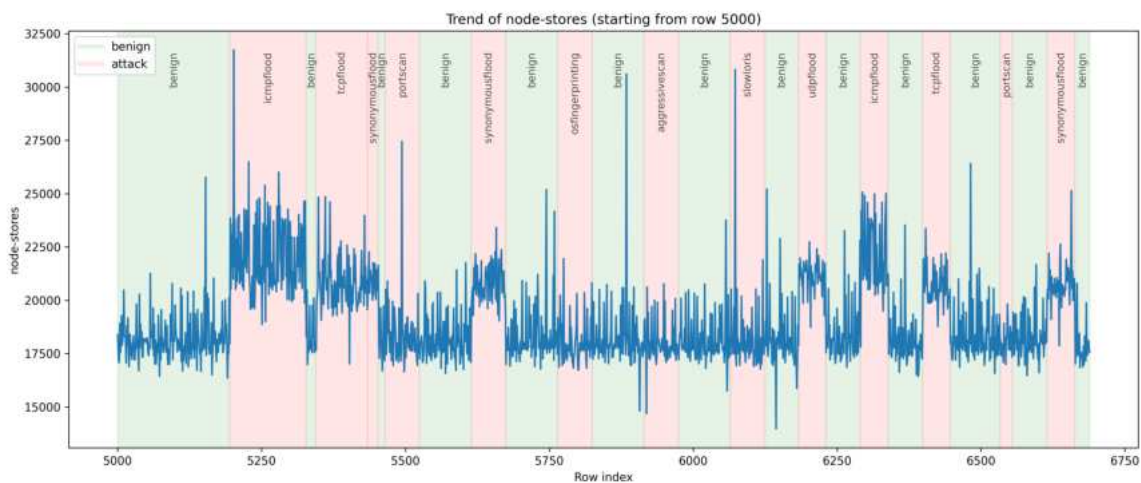


Figure 29: Timeline behavior of the `node-stores` feature, showing distinguishable patterns during flooding attacks.

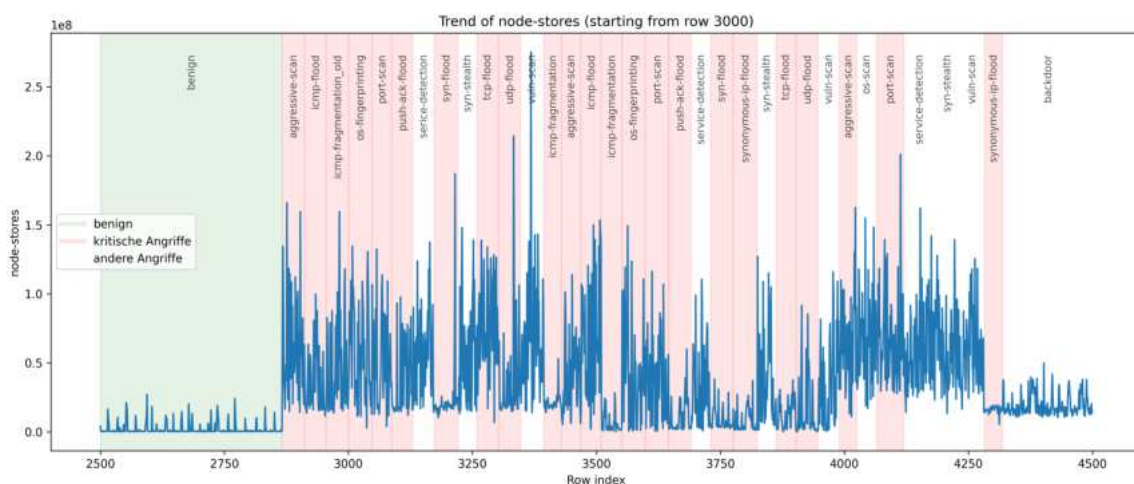


Figure 30: Timeline behavior of the `node-stores` feature from the CICEVSE2024 dataset.

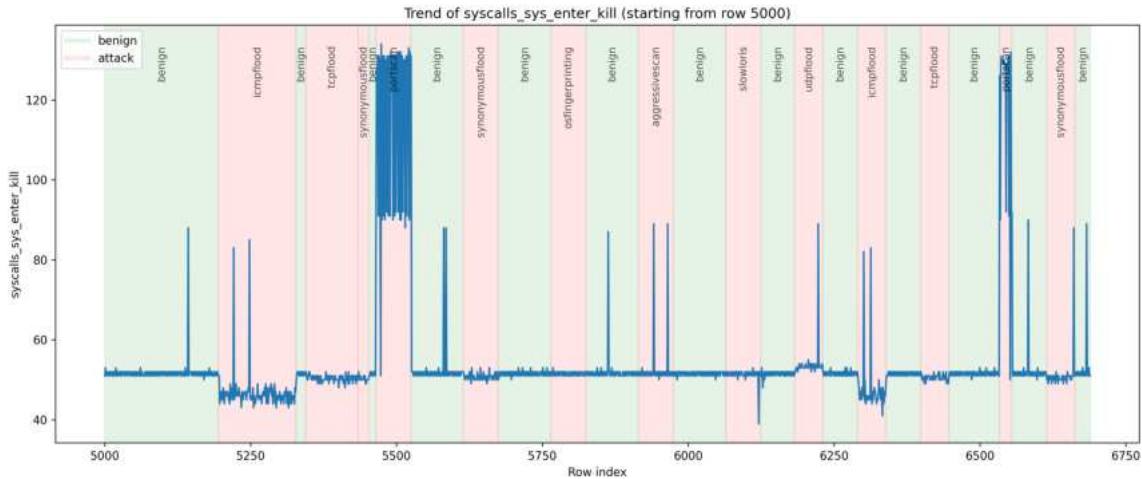


Figure 31: Timeline behavior of the `syscalls_sys_enter_kill` feature, highlighting its usefulness in detecting Portscan attacks.

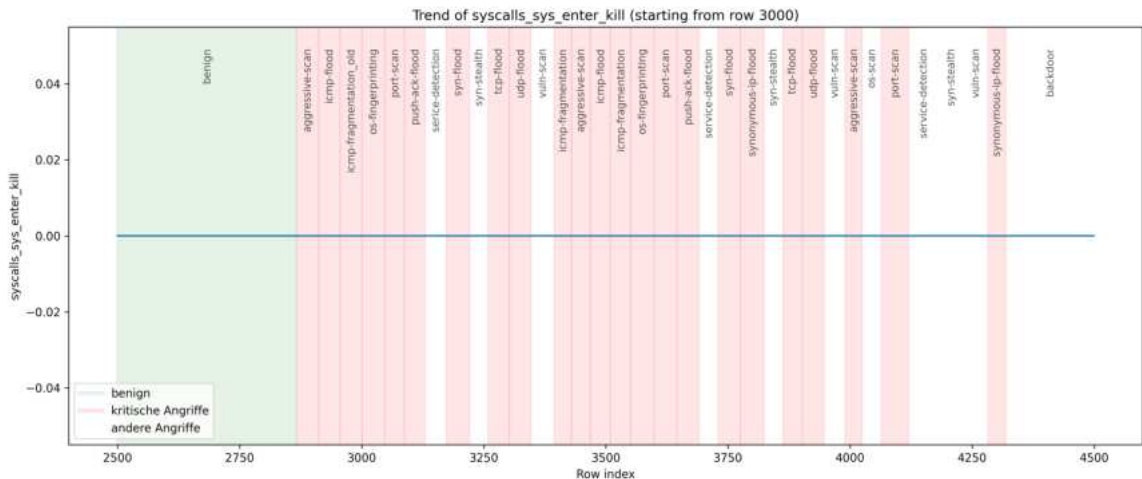


Figure 32: Timeline behavior of the `syscalls_sys_enter_kill` feature, from the CICEVSE2024 dataset. The lack of variation in the feature within the CICEVSE2024 dataset may be due to a measurement error or the system call not being invoked during the entire monitoring period. Unfortunately, neither the paper [26] nor the accompanying dataset provides a definitive explanation.

On the other hand, some features are unfortunately not very useful for attack detection, as they show almost no distinguishable peaks between benign and attack data. This is the case for the feature `unaligned_st_spec`, as illustrated in the figure below.

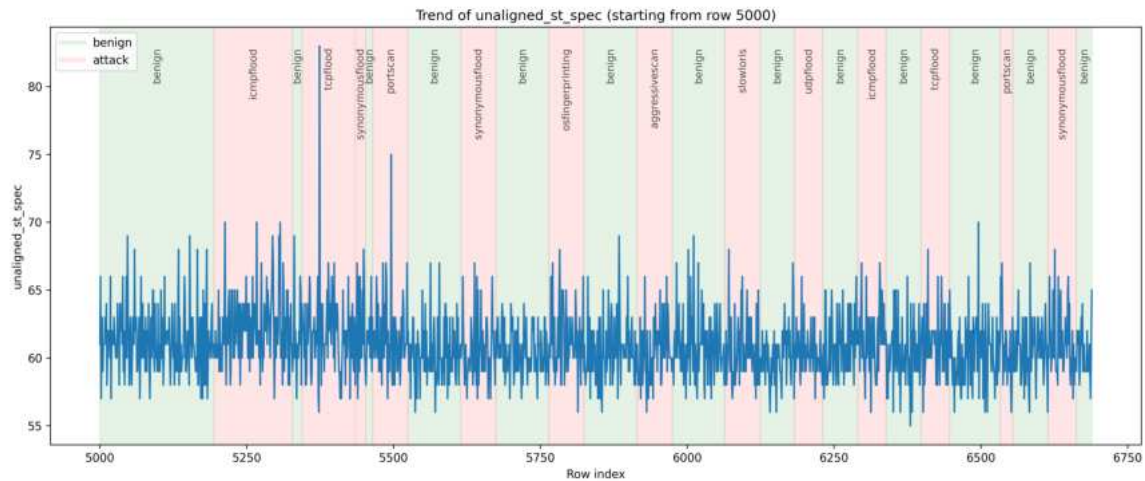


Figure 33: Timeline behavior of the `unaligned_st_spec` feature, showing little variation between benign and attack data.

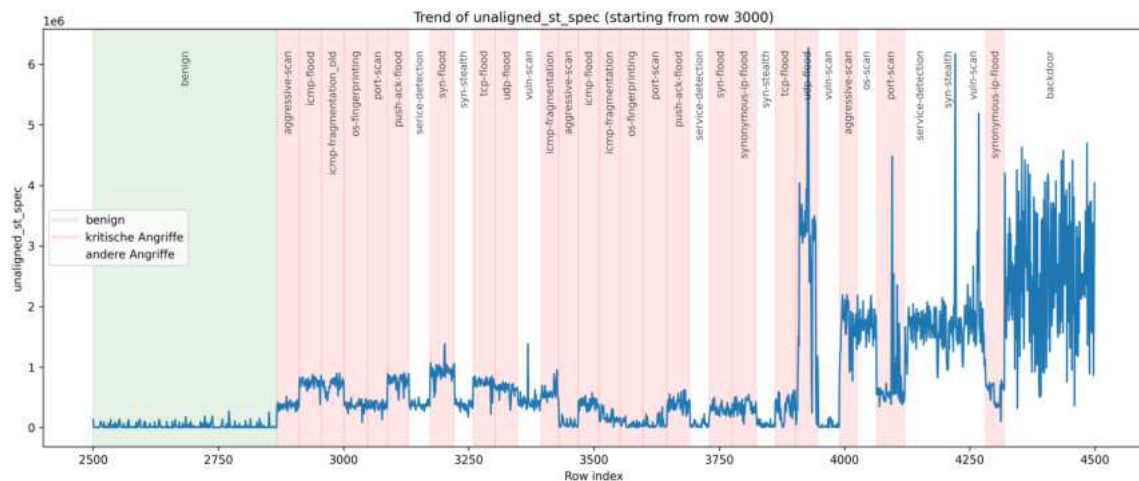


Figure 34: Timeline behavior of the `unaligned_st_spec` feature, from the CICEVSE2024 dataset.

7 LSTM Architecture: Structure, Training, and Optimization

7.1 LSTM Architecture and Preprocessing

A foundational understanding of the structure and principles of Long Short-Term Memory (LSTM) networks has already been provided. In this chapter, the specific architecture of the LSTM used in this work is described in more detail, highlighting the importance of time-step-based data analysis in this context.

Time-step analysis offers a major advantage of LSTMs over other deep neural network (DNN) models, particularly in the detection of cyber attacks or anomalies. Most such events do not occur abruptly. Instead, they often emerge gradually over time [27]. LSTM networks are specifically designed to recognize temporal patterns and sequential dependencies, making them especially well-suited for identifying these slow-developing behaviors.

In order to detect time-dependent attacks effectively, the training data must be well-organized so that the models can learn time-relevant patterns, both during training and later in the live system. Therefore, the preprocessing steps used to split the data into training and test sets must be carefully structured and should not be randomized.

At this stage, the data has generally already been cleaned, as described in chapter 6.

In this work, the dataset was divided into ten chronological blocks, each sorted by time steps. Seven of these blocks were combined to form the training dataset, while the remaining three were used for testing. The choice of blocks for training and testing ensured that both sets contained approximately the same distribution of the labels benign and attack, thereby avoiding class imbalance. Furthermore, the label distribution was verified for both sets to confirm that they were well-balanced and representative of the overall dataset.

The label distributions in both the training and test datasets were carefully controlled to ensure a balanced representation of benign and attack instances. The resulting proportions are as follows:

- Training dataset: benign = 86.82%, attack = 13.18%.
- Test dataset: benign = 84.93%, attack = 15.07%.

This balanced distribution allows the model to learn effectively while being evaluated on data that realistically reflects the variety of scenarios encountered in practice.

7.2 Parameter Configuration and Hyperparameter Optimization

In this chapter, a closer look is taken at the chosen structure of the LSTM architecture and it is explained.

First, the training and test datasets are loaded, and the necessary configurations for features and labels are prepared. This ensures that the datasets are consistently structured before being passed into the model, with a clear separation between feature values and corresponding labels.

In the next step, the data is preprocessed. The class labels are converted from strings (“benign”, attack) into binary values (0 and 1). In addition, all feature values are standardized using the `StandardScaler`. This scaling step is crucial for training an LSTM model effectively. Unscaled data can lead to numerical instability and poor convergence in neural networks. LSTMs are particularly sensitive to the scale of input features because they use sigmoid activation functions, which operate best when input values are within a small, centered range. If the features have very different magnitudes, such as bytes transmitted (e.g., in thousands) versus CPU usage (e.g., in fractions), the model may focus disproportionately on features with larger numeric ranges, leading to biased learning and suboptimal performance [46]. By standardizing the input features (i.e., transforming them to have zero mean and unit variance), it is ensured that the LSTM model can learn temporal dependencies more effectively and converge faster during training.

Since LSTMs are specifically designed to recognize temporal patterns, the dataset is further transformed into time sequences. A sliding window is applied over the data to generate fixed-length sequences. Importantly, only those sequences are retained in which all labels within the window are identical, meaning that the entire sequence is consistently classified as either benign or attack. This avoids ambiguous targets and ensures that the model learns clear and consistent classification patterns.

Table 6 summarizes the key components and layers used in the LSTM model configuration.

Component / Layer	Description / Configuration	Function / Role
First LSTM layer	LSTM with 50 units, returns full sequences	Captures temporal dependencies across input sequences and allows stacking of a second LSTM layer
Dropout after first LSTM	Regularization layer that sets 20% of inputs to zero during training	Reduces overfitting and improves model generalization
Second LSTM layer	LSTM with 15 units, returns only the final output	Processes temporal dependencies at a deeper level and condenses sequence information
Dropout after second LSTM	Regularization layer that sets 20% of inputs to zero during training	Further reduces overfitting and stabilizes learning
Dense output layer	Fully connected layer with 1 unit and sigmoid activation	Produces probability for binary classification (benign or attack)
Binary Cross-Entropy	Loss function for binary classification tasks	Measures the information loss between predicted probabilities and true binary labels
Adam	Optimization algorithm combining momentum and adaptive learning rates	Enables fast convergence and handles noisy and sparse gradients effectively

Table 6: Explanation of key components and layers used in the LSTM model configuration [47][48][49][50].

The model itself is designed as a stacked LSTM architecture. The first LSTM layer returns full sequences, allowing a second LSTM layer to process temporal dependencies at a deeper level. After each LSTM layer, dropout regularization is applied to improve the model’s ability to generalize. The final output layer uses a sigmoid activation function to generate the probability of a sample belonging to either the benign or attack class. This design provides a balanced trade-off between complexity and interpretability, making it well-suited for binary time-series classification tasks.

The training process is then carried out on the prepared sequences. The model is trained for 20 epochs, with a batch size of 32 samples. The batch size does not directly influence the final outcome of the model but impacts training efficiency and resource usage. Smaller batch sizes can produce noisier gradient updates, which may sometimes help generalization, while larger batches result in smoother updates at the cost of increased memory consumption. A batch size of 32 is a common compromise between these extremes [51].

In addition, it should be noted that an early stopping function could be used instead of training for a fixed number of 20 epochs. Early stopping monitors the validation performance and automatically stops training when no further improvements are observed, thereby saving computation time and preventing overfitting. However, to keep the implementation simple, this additional feature is not discussed in detail in this work.

After gaining an understanding of how the LSTM architecture was designed for this use case and the meaning of its various functions, the results will be reviewed in the next chapter.

8 Analysis and Evaluation

8.1 Experimental Setup and Evaluation Metrics

To evaluate the model objectively and verify whether it correctly detects anomalies, this work follows a systematic approach. Clearly separated test data from the dataset CICEVSE2024 as well as from the dataset collected in the course of this work were used. In addition, the model was tested in a live environment with real-time detection.

The dataset CICEVSE2024 and the dataset collected for this work were already split into training and test data, as described in chapter 7.1. The test data was used to evaluate the model's performance and consist of fixed time periods to ensure that the LSTM correctly utilizes its memory and detects attacks based on changes in the event data over time.

The trained LSTM model as well as the scaling parameters (`StandardScaler`) were exported and deployed for live classification on a resource-constrained Raspberry Pi system. Using Prometheus as a monitoring solution and a specially developed performance exporter as shown in Listing 1, measurement data could be provided at the desired interval.

To realistically test the model's behavior, various attack patterns (e.g. Flooding, Scans, Slowloris) were executed in an automated manner with precise time logging as shown in Listing 3. This enabled a targeted analysis of the classification output during typical threat scenarios.

Evaluation Metrics: The quality of anomaly detection was assessed using established metrics for binary classification:

- **Accuracy:** Proportion of correctly classified samples relative to the total number of samples.
- **Precision:** Proportion of actual attacks among all samples classified as attacks.
- **Recall:** Proportion of detected attacks among all actual attacks.
- **F1-Score:** Harmonic mean of Precision and Recall, particularly relevant for imbalanced classes.
- **Visualization:** Additionally, distribution and edge cases were illustrated using the Confusion Matrix, ROC Curve, and Precision-Recall Curve.

The focus, beyond the pure detection rate, was placed on the robustness of the model with respect to different types of attacks and possible data deviations during live operation.

These metrics serve as the foundation for the detailed result analysis in the following chapter and allow for a transparent evaluation of the practical applicability of the approach.

8.2 Results of Anomaly Detection Using the Machine Learning Model

Up to this point, a good understanding of the data and the features relevant for the detection of attacks has been established as well as the configuration of the LSTM model has been obtained. Now a closer look is taken at the results of the anomaly detection using the presented LSTM model on the CEVSE2024 dataset and the dataset recorded for this work.

The following table presents the key classification metrics for the anomaly detection task for the test dataset recorded for this work.

Table 7: Classification Metrics for Self-Collected Dataset

Class	Precision	Recall	F1-Score	Support
0 (Benign)	0.92	0.94	0.93	1,724
1 (Attack)	0.60	0.52	0.56	301
Accuracy	0.8775			
Macro avg	0.76	0.73	0.74	2,025
Weighted avg	0.87	0.88	0.87	2,025

The model shows high accuracy (0.88) on the self-collected dataset, with strong performance in identifying benign cases (Class 0). In contrast, precision and recall for attack detection (Class 1) are lower (0.60 and 0.52, respectively), which reduces the macro average and weighted average metrics.

The following table presents the key classification metrics for the anomaly detection task for the CEVSE2024 dataset.

Table 8: Classification Metrics for the CEVSE2024 Dataset

Class	Precision	Recall	F1-Score	Support
0 (Benign)	0.96	1.00	0.98	377
1 (Attack)	1.00	0.99	0.99	1,396
Accuracy	0.9915			
Macro avg	0.98	0.99	0.99	1,773
Weighted avg	0.99	0.99	0.99	1,773

In direct comparison, the results on the CEVSE2024 dataset are significantly stronger than those obtained on the self-collected dataset. The LSTM model achieves an overall accuracy of 0.99 with nearly perfect precision, recall, and F1-scores for both benign and attack classes.

In contrast, the results on the self-collected dataset show lower precision and recall values for attack detection, which reduces the macro average and weighted average metrics.

As already observed during the exploratory data analysis, a central reason for these differences lies in the characteristics of the attack traces themselves. In the self-collected dataset, the peaks of scanning attacks such as OS-Fingerprinting, Portscans, and Aggressive-Scans are barely distinguishable from benign traffic. In the CEVSE2024 dataset, however, these attacks produce clearly visible deviations in the selected features, which allows the LSTM model to separate them more reliably from benign cases.

Potential reasons for these discrepancies are not related to hardware, since the experimental setup was reproduced one-to-one. More plausible explanations include background processes that cannot be controlled to exactly the same degree, or differences in the tools and parameter settings used to perform the scans. These factors can lead to stronger variations in the selected features and thus improve the separability of attack traffic.

It is important to note that the LSTM model was not altered between the experiments. The same architecture and hyperparameters were applied to both datasets, and the same feature set was used, namely the features identified during feature selection with the Decision-Tree-Classifier on the self-collected dataset.

These results are consistent and remain unchanged when analyzing the confusion matrix and the direct detection of attacks. The following figures illustrate these results by comparing the self-collected dataset with the CEVSE2024 dataset. Nevertheless, it can be observed that flooding attacks have a high probability of being detected across different datasets, as long as the hardware is similar and simulates a charging station environment. This aspect will be discussed in more detail in a later chapter.

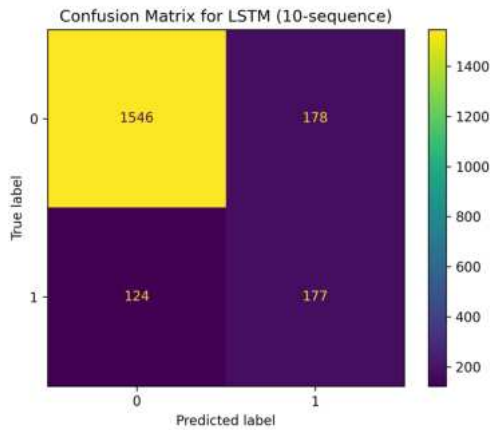


Figure 35: Confusion matrix for the self-collected dataset

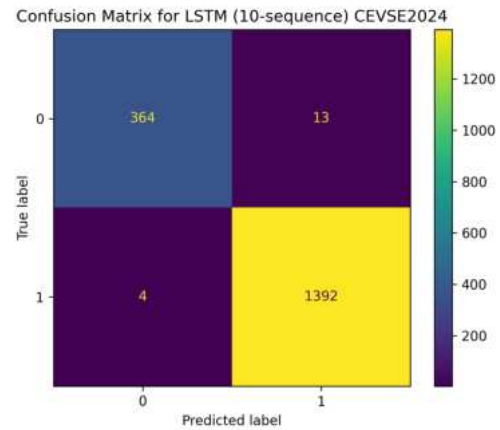


Figure 36: Confusion matrix for the CEVSE2024 dataset

It can be observed that in the self-collected dataset, the majority of samples are benign, while in the CEVSE2024 dataset the majority are attack samples. This results in a mirrored appearance of the confusion matrices.

Table 9: Misclassified attacks, Self-collected Dataset

Attack	Misclassified	Total
Aggressive-Scan	36	59
OS-Fingerprinting	34	59
Slowloris	25	45
Portscan	9	59
SYN-Flood	3	76
TCP-Flood	0	3

Table 10: Misclassified attacks, CEVSE2024 Dataset

Attack	Misclassified	Total
Aggressive-Scan	2	39
Backdoor	2	591
Cryptojacking	0	213
ICMP-Fragmentation	0	78
ICMP-Fragmentation Old	0	41
ICMP-Flood	0	41
OS-Fingerprinting	0	47
Portscan	0	40
Push-Ack Flood	0	43
Service Detection	0	42
SYN-Flood	0	49
SYN-Stealth	0	37
TCP-Flood	0	44
UDP-Flood	0	45
Vulnerability Scan	0	46

Scanning attacks are generally more difficult to detect, as reflected by the misclassification rates in both datasets. In particular, the Aggressive-Scan shows a misclassification even in the CEVSE2024 dataset.

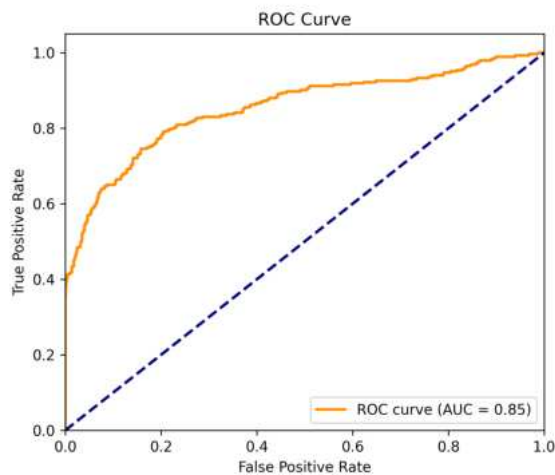


Figure 37: ROC curve for the self-collected dataset

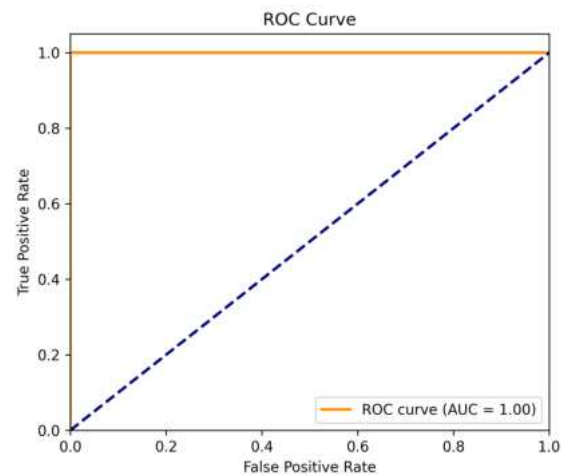


Figure 38: ROC curve for the CEVSE2024 dataset

The ROC curves illustrate the model's performance in distinguishing between benign and attack traffic. The self-collected dataset shows a lower area under the curve (AUC) compared to the CEVSE2024 dataset, indicating that the model is less effective at correctly distinguishing attacks from benign traffic on the self-collected dataset. This lower AUC reflects a higher rate of misclassifications and a reduced overall discriminative capability, which can be attributed to the varying strong peaks observed in the self-collected dataset, as already illustrated in the EDA. In contrast, the high AUC on the CEVSE2024 dataset indicates near-perfect separation between benign and attack samples.

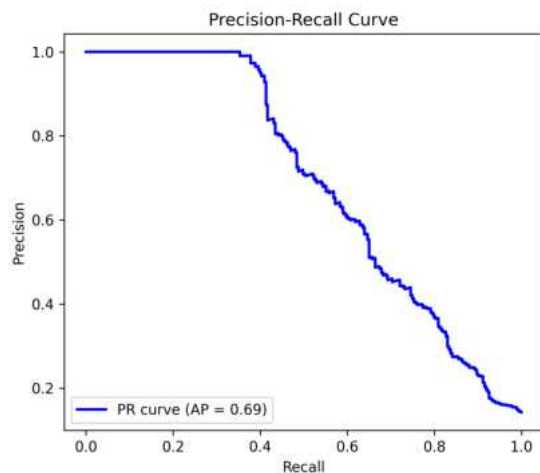


Figure 39: Precision-Recall curve for the self-collected dataset

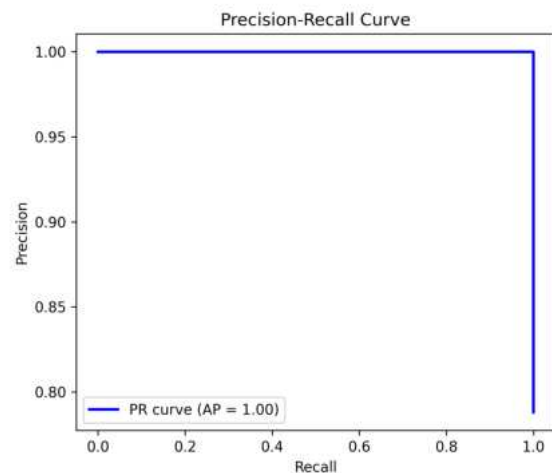


Figure 40: Precision-Recall curve for the CEVSE2024 dataset

The Precision-Recall curves highlight a clear difference between the two datasets. For the self-collected dataset, the model achieves an average precision of 0.69, resulting in a curve that indicates a noticeable trade-off between precision and recall. This suggests that while the model can detect many attacks, it also produces a non-negligible number of false positives or misses some attacks. In contrast, the CEVSE2024 dataset yields a perfect score of 1.0, where the curve remains at the upper boundary, indicating flawless separation between benign and attack samples. This demonstrates that the model performs ideally on the CEVSE2024 dataset, while performance on the self-collected dataset remains more challenging.

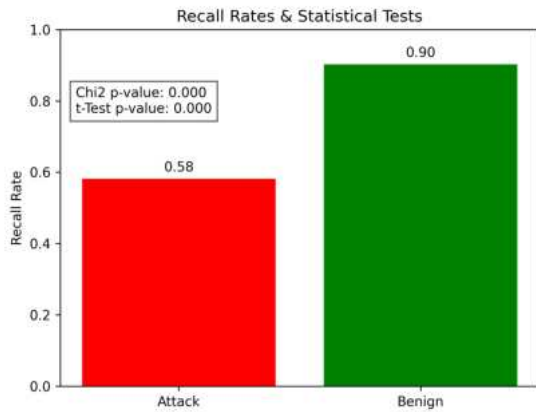


Figure 41: Recall statistics for the self-collected dataset

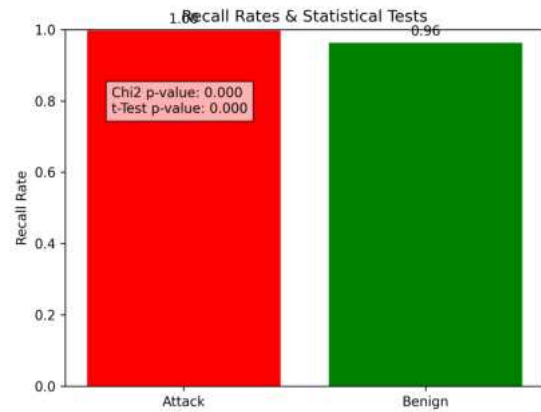


Figure 42: Recall statistics for the CEVSE2024 dataset

It can be observed, similar to the previous results, that some attacks (e.g., OS-Fingerprinting, Aggressive-Scan) are more difficult to detect in the self-collected dataset. More importantly, however, this figure shows that the p-value in both datasets is lower than 0.05. This indicates that the LSTM model's detection is not random [52], but rather that the benign and attack data differ significantly, confirming the reliability of the results.

After testing and analyzing with the monitored system, this work also includes testing the LSTM model in a live system. This is important to ensure that the model can detect attacks even if the data do not have the same timestamps, and to observe what happens if the environment is overloaded so that new data do not arrive at the same intervals as the training data when the processor is under heavy load.

At first, it was necessary to store the LSTM model as a .keras file and also save the scaler as a .pkl file. This makes it possible to scale all live data in the same way as the training data.

The system continuously collects live performance metrics from a monitored environment, which is only possible through collection with the **perf** tool and processing in the Prometheus database. Some metrics require the calculation of differences between successive measurements to reflect true activity changes and avoid permanent accumulation, as this can cause problems with the LSTM model if a counter is reset unexpectedly.

The collected data are normalized using a pre-trained scaler and then fed into a pre-trained LSTM model to predict whether the system is under attack or operating normally. The results, including probabilities for both benign and attack states, are logged into a CSV file for further analysis, while real-time feedback is printed to the console. This approach allows monitoring and detecting anomalies in a live environment, closely simulating conditions that differ from

the controlled training setup.

After live testing the system for about six hours and 25 attacks, the following results are shown in figure 43:

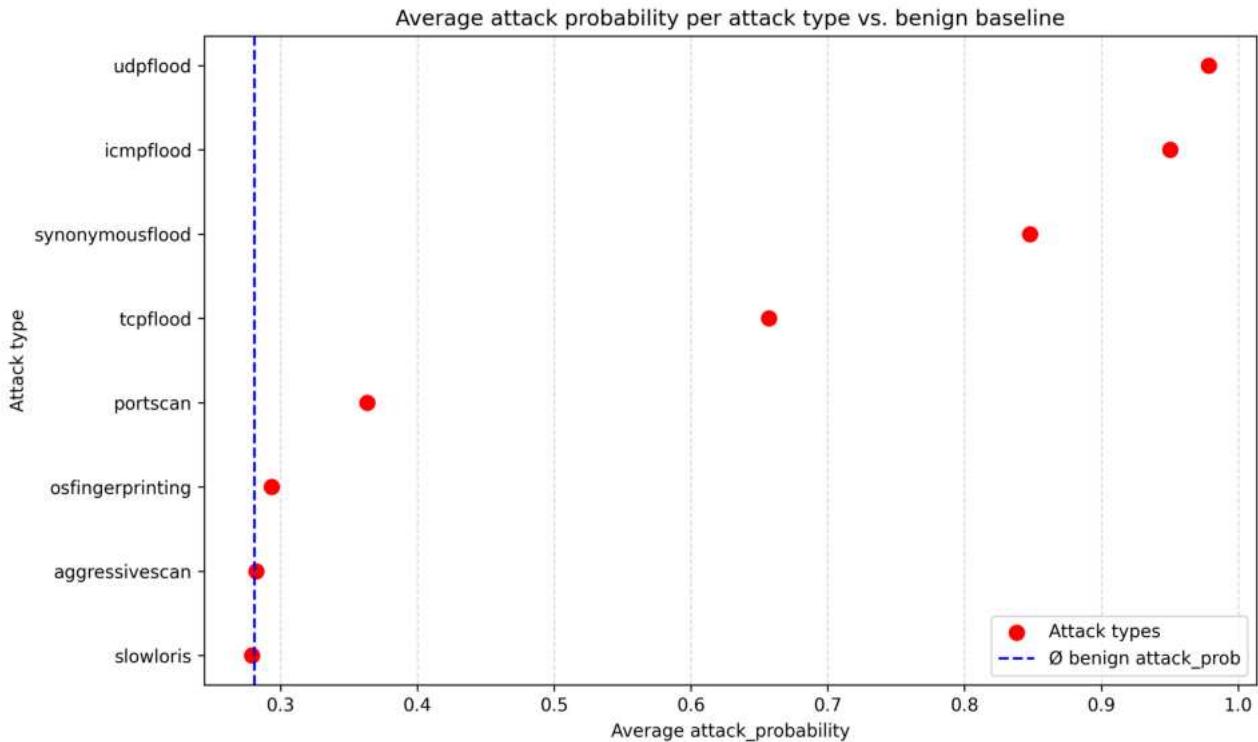


Figure 43: Live data classification results

It can be observed that the equal attack probability was about 20% and reached the highest peaks during the flooding attacks, in the best case approaching almost 100% attack probability. Additionally, during the time sequence when the Portscan attacks occurred, the attack probability increased significantly, leading to a clear rise. As expected, before the scanning attacks, OS-Fingerprinting and Aggressive-Scans showed only small peaks, as did the Slowloris attack.

9 Discussion

The evaluation of the model demonstrates that the proposed approach performs reliably across both the CICEVSE2024 dataset and the self-collected dataset. In particular, flooding attacks were consistently detected with very high accuracy, confirming that the selected features provide clear and distinguishable patterns for this type of malicious activity. The strong performance against flooding attacks indicates that the LSTM model in combination with decision tree based feature selection is capable of capturing temporal dependencies and amplifying signal

peaks that are characteristic of such attacks.

However, the detection of scanning activities proved to be more challenging. The reason for this lies in the more subtle nature of scan-related anomalies, which often lack distinctive peaks in the observed features.

When comparing the results of this work with related studies, particularly the study presented in the paper "Embedded feature selection in lstm networks with multi-objective evolutionary ensemble learning for time series forecasting" [45], a strong alignment can be observed. Both approaches demonstrate that combining decision tree based feature selection with an LSTM classifier leads to consistently high classification performance, with F1-scores exceeding 0.85. Furthermore, when the dataset exhibits strong peaks in the selected features, as is the case with the CICEVSE2024 dataset, F1-scores can surpass 0.95. This confirms that Decision-Tree-Classifiers provide a powerful and efficient method for selecting features when deploying LSTM-based models.

This work also extends the discussion to embedded-hardware environments. A charging station setup similar to the one described in the paper "Enhancing EV Charging Station Security Using a Multi-dimensional Dataset" [26] and implemented here could execute the detection algorithm efficiently without exceeding hardware constraints. The evaluation indicates that the optimal number of selected features lies between 11 and 20, balancing accuracy with computational efficiency [26]. The detection algorithm, as well as the supporting monitoring tools such as `perf`, are not resource-intensive [53], which underlines the suitability of this approach for real-world embedded systems.

As highlighted in both this work and related literature, flooding attacks can be detected with high reliability under comparable hardware configurations. However, the robustness of the detection results strongly depends on the underlying system and the specific hardware events that can be recorded, as well as on the similarity of their characteristics. If the hardware architecture differs significantly or if kernel-level event traces deviate, the model's performance may degrade. This emphasizes the need for adaptive feature engineering and retraining when applying the model to new or heterogeneous environments.

Consequently, the model must be able to determine which hardware events are available and reliable on the target system, and to identify which features are most relevant for attack detection in that specific context. Since hardware performance counters vary across processor generations and architectures, it cannot be assumed that the same set of events will always be accessible or equally meaningful. This variability underlines the importance of a flexible and adaptive approach to feature selection when deploying the model on different hardware platforms [54].

Another important limitation is the "black-box" nature of the LSTM model. As described

in chapter 2.4, the complex design composed of multiple gates and memory cells creates significant challenges in interpreting how these models arrive at specific decisions. This lack of interpretability makes it difficult to explain why particular patterns of hardware events are classified as attacks, especially under varying system conditions. As a result, the robustness and trustworthiness of the model may be reduced when deployed in real-world, diverse, or evolving scenarios.

This “black-box” problem manifests in several ways:

- LSTMs have an intricate internal structure that involves several nonlinear operations, which complicates the understanding of how input data is processed and decisions are made.
- The model’s sensitivity to hyperparameters, potential overfitting, and training instability further increase the complexity of the interpretability challenge.
- The standard outputs of LSTM models do not provide any explanation for the decisions made, which is a significant drawback in security-critical applications such as intrusion detection.

The future work part is a discussion about potential approaches to address these limitations will be discussed, such as the application of explainable AI (XAI) techniques (e.g., saliency maps, LIME, or SHAP) or the use of simpler surrogate models that provide more transparency in their decision-making processes [55].

Despite these limitations, the work presented here provides a strong foundation for the application of intrusion detection systems in electric vehicle charging stations. The results demonstrate that the proposed combination of LSTM and decision tree based feature selection can detect attacks with high accuracy, especially flooding attacks, while maintaining efficiency suitable for embedded hardware.

10 Future Work

One limitation observed during the evaluation is that, under strong flooding attacks, the embedded device may become so constrained that it cannot provide timely feedback on event data. In particular, Prometheus queries may fail to deliver results because the detection process is not prioritized sufficiently within the system. To address this, an investigation on methods to ensure that the detection algorithm always operates within a reserved portion of system resources that cannot be interrupted or consumed by other processes. This would guarantee reliable monitoring and responsiveness, even under conditions of severe resource exhaustion.

Furthermore, it will be necessary to test whether the detection system can also recognize attack types that were not explicitly part of the training dataset. Evaluating the model's ability to generalize to unseen attacks will provide valuable insights into its robustness and practical applicability in dynamic real-world environments.

Another important step is the extension of the detection system beyond a single EVSE towards multiple EVSEs operating simultaneously. By integrating several charging stations into a distributed detection environment, robustness can be improved and anomalous behavior can be identified more effectively. Such a setup would not only increase resilience against individual failures but also strengthen the overall security of the charging infrastructure.

In addition, the LSTM model should be trained with a broader set of attack scenarios and extended feature sets. As already discussed in the *Discussion*, a method should be explored to make the decision-making process of the LSTM more transparent. In particular, techniques such as saliency maps, Local Interpretable Model-Agnostic Explanations (LIME), and Shapley Additive Explanations (SHAP) [55] could be applied to visualize and quantify the contribution of individual input features to the model's predictions. These methods would support a better understanding of why certain anomalies are detected, thereby increasing the transparency and trustworthiness of the system.

Features capturing network traffic should also be incorporated alongside hardware performance events. This would reduce the dependency on hardware-related signals alone and enable the system to detect anomalies reflected in communication patterns, thereby increasing detection accuracy and coverage.

In addition, an approach based on transfer learning for LSTM models should be considered to improve applicability across different hardware architectures. Such a strategy would allow the model to adapt to new environments without requiring extensive retraining, thereby increasing

scalability and practicality in real-world deployments. Approaches of this kind have already been discussed in the literature, for instance in “Transfer Learning Across Heterogeneous Features for Efficient Tensor Program Generation”, and should be specifically evaluated in the context of intrusion detection in EVSEs [56].

Bibliography

- [1] S. B. Mitikiri, Y. Tiwari, V. L. Srinivas, and M. Pal, “Regression based anomaly detection in electric vehicle state of charge fluctuations through analysis of evci data,” *arXiv preprint arXiv:2401.01580v1*, 2024, Accessed: 2025-07-09. [Online]. Available: <https://arxiv.org/abs/2401.01580v1>.
- [2] European Commission. “2050 long-term strategy: Climate-neutral europe by 2050.” Accessed: 2025-07-09. (2020), [Online]. Available: https://climate.ec.europa.eu/eu-action/climate-strategies-targets/2050-long-term-strategy_en.
- [3] I. E. Agency. “Global ev outlook 2022.” Accessed: 2025-07-09. (2022), [Online]. Available: <https://www.iea.org/reports/global-ev-outlook-2022>.
- [4] European Automobile Manufacturers Association (ACEA). “Interactive map: Correlation between electric car sales and charging point availability - 2023 data.” Accessed: 2025-07-09. (2023), [Online]. Available: <https://www.acea.auto/figure/interactive-map-correlation-between-electric-car-sales-and-charging-point-availability-2023-data/>.
- [5] D. Ramos. “Cybercriminals target electric vehicle chargers.” (), [Online]. Available: <https://www.silicon.eu/cybercriminals-target-electric-vehicle-chargers-17047.html> (visited on 07/29/2025).
- [6] M. J. H. V. Thiruppathy Kesavan, “Anomaly detection with grid sentinel framework for electric vehicle charging stations in a smart grid environment,” *Scientific Reports*, vol. 15, no. 1, p. 400, 2025, Accessed: 2025-07-09. DOI: 10.1038/s41598-025-00400-z. [Online]. Available: <https://www.nature.com/articles/s41598-025-00400-z>.
- [7] P. Bertoldi, F. Cappa, G. D. Chiara, F. M. D’Angeli, and P. Zancanella, “Cybersecurity for electric vehicle charging infrastructure: Challenges and best practices,” *World Electric Vehicle Journal*, vol. 15, no. 12, p. 571, 2025, Accessed: 2025-07-09. DOI: 10.3390/wevj15120571. [Online]. Available: <https://www.mdpi.com/2032-6653/15/12/571>.
- [8] ReSiLENT Project. “Resilient project.” Accessed: 2025-07-10. (2025), [Online]. Available: <https://resilient-project.de/>.
- [9] Stöhr Mobility GmbH. “Stöhr mobility gmbh.” Accessed: 2025-07-10. (2025), [Online]. Available: <https://www.stoehr-mobility.de/>.
- [10] ChargeIQ GmbH. “Chargeiq gmbh.” Accessed: 2025-07-10. (2025), [Online]. Available: <https://chargeiq.de/>.
- [11] ASVIN. “Asvin.” Accessed: 2025-07-10. (2025), [Online]. Available: <https://asvin.io/>.
- [12] Y. Hanel. “Interior view of a wallbox.” Own image. (2025).
- [13] European Union. “General data protection regulation (gdpr) – online 5: Principles relating to processing of personal data.” Accessed: 2025-07-10. (2018), [Online]. Available: <https://gdpr-info.eu/art-5-gdpr/>.

- [14] European Union. “General data protection regulation (gdpr) – online 6: Lawfulness of processing.” Accessed: 2025-07-10. (2018), [Online]. Available: <https://gdpr-info.eu/art-6-gdpr/>.
- [15] I. H. Sarker, “Ai-based modeling: Techniques, applications and research issues towards automation, intelligent and smart systems,” *SN Computer Science*, vol. 3, no. 2, p. 158, 2022, Accessed: 2025-07-10. DOI: 10.1007/s42979-022-01043-x. [Online]. Available: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC8830986/>.
- [16] A. BANERJEE, “Anomaly detection: A survey,” *University of Minnesota*, 2009, Accessed: 2025-07-15. [Online]. Available: <http://cucis.ece.northwestern.edu/projects/DMS/publications/AnomalyDetection.pdf>.
- [17] V. J. Hodge and J. Austin, “A survey of outlier detection methodologies,” *Artificial Intelligence Review*, 2004, Accessed: 2025-07-15. [Online]. Available: https://www-users.york.ac.uk/~vjh5/myPapers/Hodge+Austin_OutlierDetection_AIRE381.pdf.
- [18] PeruriVenkataAnusha, “Anomaly detection techniques,” *International Journal of Innovative Technology and Exploring Engineering*, 2019, Accessed: 2025-07-15. [Online]. Available: <https://www.ijitee.org/wp-content/uploads/papers/v9i1/A3910119119.pdf>.
- [19] K. Maharana, “A review: Data pre-processing and data augmentation techniques,” *ScienceDirect - Knowledge and Information Systems*, 2022, Accessed: 2025-07-15. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2666285X22000565>.
- [20] The pandas development team. “Pandas documentation, version 2.3.1.” Accessed: 2025-07-23. (2025), [Online]. Available: <https://pandas.pydata.org/docs/>.
- [21] M. Waskom and the Seaborn development team. “Seaborn: Statistical data visualization.” Accessed: 2025-07-23. (2025), [Online]. Available: <https://seaborn.pydata.org/>.
- [22] B. T. Jijo and A. M. Abdulazeez, “Decision tree explanation,” *Classification Based on Decision Tree Algorithm for Machine Learning*, 2021, Accessed: 2025-07-09. [Online]. Available: <https://jastt.org/index.php/jasttpath/article/view/65/24>.
- [23] L. Breiman, “Random forests,” 2001, Accessed: 2025-07-15. [Online]. Available: <https://www.stat.berkeley.edu/~breiman/randomforest2001.pdf>.
- [24] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation forest,” 2008, Accessed: 2025-07-15. [Online]. Available: <http://www.lamda.nju.edu.cn/publication/icdm08b.pdf>.
- [25] R. C. Staudemeyer and E. R. Morris, “Understanding lstm – a tutorial into long short-term memory recurrent neural networks,” 2019, Accessed: 2025-07-09. [Online]. Available: <https://arxiv.org/pdf/1909.09586>.
- [26] S. D. Emmanuel Dana Buedi Ali A. Ghorbani and R. L. Ferreira, “Enhancing ev charging station security using a multi-dimensional dataset: Cicevse2024,” 2024, Accessed: 2025-07-16. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-031-65172-4_11.

- [27] M. R. Rahman, B. Wroblewski, Q. Matthews, B. Morgan, T. Menzies, and L. Williams, "Mining temporal attack patterns from cyberthreat intelligence reports," 2025, Accessed: 2025-07-27. [Online]. Available: <https://ar5iv.labs.arxiv.org/html/2401.01883>.
- [28] M. B. M. H. Ali, "Deep learning-based intrusion detection system for electric vehicle charging station," 2020, Accessed: 2025-07-16. [Online]. Available: <https://ieeexplore.ieee.org/document/9243152>.
- [29] S. Kottilingal, "Comparative analysis of feature selection techniques for lstm based network intrusion detection models," 2024, Accessed: 2025-07-26. [Online]. Available: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4979065.
- [30] hpc-wiki contributors. "Perf – linux performance monitoring tool." Accessed: 2025-07-18. (2025), [Online]. Available: <https://hpc-wiki.info/hpc/Perf>.
- [31] D. Hodges. "Perf_exporter: A perf exporter for prometheus." Version v0.0.1, Zugriff am 18.07.2025. (2019), [Online]. Available: https://github.com/hodgesds/perf_exporter.
- [32] V. M. Weaver, "Linux perf event features and overhead," pp. 1–8, 2015, Accessed: 2025-07-18.
- [33] R. V. Lim, W. Y. Alkowiileet, D. Carrillo-Cisneros, and I. D. Scherson, "Computationally efficient multiplexing of events on hardware counters," 2014, Accessed: 2025-07-18. [Online]. Available: <https://www.kernel.org/doc/ols/2014/ols2014-lim.pdf>.
- [34] P. A. 2014-2025. "Prometheus - open source metrics and monitoring for your systems and services." Zugriff am 22.07.2025. (2025), [Online]. Available: <https://prometheus.io/>.
- [35] P. B.C., H. Maddirala, and S. M., "Implementing an effective infrastructure monitoring solution with prometheus and grafana," 2024, Accessed: 2025-07-22. [Online]. Available: <https://www.ijcaonline.org/archives/volume186/number38/pragathi-2024-ijca-923873.pdf>.
- [36] I. Stetsenko and A. Myroniuk, "Software for collecting and analyzing metrics in highly loaded applications based on the prometheus monitoring system," 2024, Accessed: 2025-07-22. [Online]. Available: <https://ela.kpi.ua/server/api/core/bitstreams/8138e408-804b-45f6-9842-9e7b41e50dc0/content>.
- [37] T. kernel development community. "Event tracing." (), [Online]. Available: <https://www.kernel.org/doc/html/v5.0/trace/events.html> (visited on 08/15/2025).
- [38] G. community. "Types of cache misses." (), [Online]. Available: <https://www.geeksforgeeks.org/computer-organization-architecture/types-of-cache-misses/> (visited on 08/15/2025).
- [39] arm limited. "Pmu events." (), [Online]. Available: <https://developer.arm.com/documentation/100442/0100/debug-descriptions/pmu/pmu-events?lang=en> (visited on 08/15/2025).
- [40] B. Gregg. "Perf examples." (), [Online]. Available: <https://www.brendangregg.com/perf.html> (visited on 08/15/2025).

- [41] P. J. Drongowski. “Perf tutorial: Counting hardware performance events.” (), [Online]. Available: <https://sandsoftwaresound.net/perf/perf-tut-count-hw-events/> (visited on 08/15/2025).
- [42] T. kernel development community. “What is numa?” (), [Online]. Available: <https://www.kernel.org/doc/html/v5.6/vm/numa.html> (visited on 08/17/2025).
- [43] T. kernel development community. “Bus-independent device accesses.” (), [Online]. Available: <https://docs.kernel.org/next/driver-api/device-io.html> (visited on 08/17/2025).
- [44] “The socket buffer.” (), [Online]. Available: <http://oldvger.kernel.org/~davem/skb.html> (visited on 08/17/2025).
- [45] R. Espinosa, F. Jim´enez, and J. Palm, “Embedded feature selection in lstm networks with multi-objective evolutionary ensemble learning for time series forecasting,” 2023, Accessed: 2025-08-17. [Online]. Available: <https://arxiv.org/pdf/2312.17517>.
- [46] T. Cooijmans, N. Ballas, C. Laurent, Çağlar Gülgehr, and A. Courville, “Recurrent batch normalization,” 2017, Accessed: 2025-07-27. [Online]. Available: <https://arxiv.org/pdf/1603.09025>.
- [47] R. Arora, P. Bartlett, P. Mianjy, and N. Srebro, “Dropout: Explicit forms and capacity control,” 2021, Accessed: 2025-07-27. [Online]. Available: <https://proceedings.mlr.press/v139/arora21a/arora21a.pdf>.
- [48] M. Saeed. “A gentle introduction to sigmoid function.” (), [Online]. Available: <https://machinelearningmastery.com/a-gentle-introduction-to-sigmoid-function/> (visited on 07/27/2025).
- [49] D. Ramos, J. Franco-Pedroso, A. Lozano-Diez, and J. Gonzales-Rodriguez, “Deconstructing cross-entropy for probabilistic binary classifiers,” 2018, Accessed: 2025-07-27. [Online]. Available: <https://www.mdpi.com/1099-4300/20/3/208>.
- [50] D. P. Kingma and J. L. Ba, “Adam: A method for stochastic optimization,” 2015, Accessed: 2025-07-27. [Online]. Available: <https://arxiv.org/pdf/1412.6980>.
- [51] “How does the batch size parameter affect the training process in a neural network?” (), [Online]. Available: <https://eitca.org/artificial-intelligence/eitc-ai-dlrf-deep-learning-with-tensorflow/tensorflow/using-more-data/examination-review-using-more-data/how-does-the-batch-size-parameter-affect-the-training-process-in-a-neural-network/> (visited on 08/18/2025).
- [52] “P-wert.” (), [Online]. Available: <https://trainingsmanufaktur.de/lexikon/p-wert/> (visited on 08/21/2025).
- [53] “Understanding perf: A powerful performance analysis tool.” (), [Online]. Available: <https://cloudtuned.hashnode.dev/understanding-perf-a-powerful-performance-analysis-tool> (visited on 08/21/2025).
- [54] G. Zellweger, D. Lin, and T. Roscoe, “So many performance events, so little time,” 2016, Accessed: 2025-08-19. [Online]. Available: https://people.inf.ethz.ch/troscoe/pubs/zellweger_apsys_2016.pdf.

- [55] Prof.G.ShankarLingam and K.ThirupathiReddy, "Drawbacks of lstm algorithm: A case study," 2025, Accessed: 2025-08-19. [Online]. Available: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=5080605.
- [56] G. Verma, Z. Xie, M. Emani, S. Raskar, A. M. Malik, and B. Chapman, "Transfer learning across heterogeneous features for efficient tensor program generation," 2023, Accessed: 2025-08-19. [Online]. Available: <https://arxiv.org/html/2304.05430v2>.

Appendix

```
1  package main
2  import (
3      "fmt"
4      "io/ioutil"
5      "net/http"
6      "os/exec"
7      "strconv"
8      "strings"
9      "time"
10
11     "gopkg.in/yaml.v2"
12 )
13
14 // Config represents the structure of the YAML configuration file.
15 // It includes high-level performance counters (HCP), software events (SW
16 //   ), and hardware events (HW).
17 type Config struct {
18     HCP []string `yaml:"hcp"`
19     SW  []string `yaml:"sw"`
20     HW  []string `yaml:"hw"`
21 }
22
23 // Global variable to store the most recent performance data.
24 var perfData string
25
26 // getPerfData runs 'perf stat' with the specified events for a short
27 //   duration (0.1s)
28 // and returns the raw output as a string.
29 func getPerfData(events []string) (string, error) {
30     eventList := strings.Join(events, ",")
31     cmd := exec.Command("perf", "stat", "-e", eventList, "--", "sleep", "
32         0.1")
33     output, err := cmd.CombinedOutput()
34     if err != nil {
35         return "", err
36     }
37     return string(output), nil
38 }
39
40 // parsePerfData filters and extracts only the relevant lines from perf
41 //   output,
42 // based on the provided list of events.
43 func parsePerfData(data string, events []string) string {
44     lines := strings.Split(data, "\n")
45     var result []string
46     for _, line := range lines {
```

```
43     for _, event := range events {
44         if strings.Contains(line, event) {
45             result = append(result, line)
46             break
47         }
48     }
49 }
50 return strings.Join(result, "\n")
51 }
52
53 // formatForPrometheus converts the filtered performance data into
Prometheus-compatible metrics format.
54 func formatForPrometheus(data string) string {
55     lines := strings.Split(data, "\n")
56     var result []string
57     for _, line := range lines {
58         parts := strings.Fields(line)
59         if len(parts) >= 2 {
60             // Clean up the metric name
61             metricName := strings.ReplaceAll(parts[1], ".", "_")
62             // metricName = strings.ReplaceAll(metricName, "-", "_")
63             // Remove decimal points from the number and parse it
64             metricValue := strings.ReplaceAll(parts[0], ".", "")
65             if value, err := strconv.ParseFloat(metricValue, 64); err == nil
66             {
67                 // Format value appropriately for Prometheus (integer or
float)
68                 if value == float64(int(value)) {
69                     result = append(result, fmt.Sprintf("%s %d", metricName,
70                         int(value)))
71                 } else {
72                     result = append(result, fmt.Sprintf("%s %f", metricName,
73                         value))
74                 }
75             }
76         }
77     }
78     return strings.Join(result, "\n")
79 }
80
81 // metricsHandler handles HTTP requests and returns the formatted
performance metrics.
82 func metricsHandler(w http.ResponseWriter, r *http.Request) {
83     fmt.Fprintf(w, formatForPrometheus(perfData))
84 }
85
86 // chunkEvents splits the list of events into smaller chunks of a given
size.
```

```
84 // This helps avoid issues with perf's limit on the number of
    // simultaneous counters.
85 func chunkEvents(events []string, chunkSize int) [][]string {
86     var chunks [][]string
87     for i := 0; i < len(events); i += chunkSize {
88         end := i + chunkSize
89         if end > len(events) {
90             end = len(events)
91         }
92         chunks = append(chunks, events[i:end])
93     }
94     return chunks
95 }
96
97 // validateEvent checks if a given performance event is available on the
    // system by using 'perf list'.
98 func validateEvent(event string) bool {
99     cmd := exec.Command("perf", "list")
100    output, err := cmd.CombinedOutput()
101    if err != nil {
102        fmt.Println("Error listing perf events:", err)
103        return false
104    }
105    return strings.Contains(string(output), event)
106 }
107
108 func main() {
109     // Load the YAML configuration file
110     config := Config{}
111     data, err := ioutil.ReadFile("config.yml")
112     if err != nil {
113         fmt.Println("Error reading config file:", err)
114         return
115     }
116     err = yaml.Unmarshal(data, &config)
117     if err != nil {
118         fmt.Println("Error parsing config file:", err)
119         return
120     }
121
122     // Validate and collect all available events from the config
123     var validEvents []string
124     for _, event := range append(config.HCP, config.SW...) {
125         if validateEvent(event) {
126             validEvents = append(validEvents, event)
127         }
128     }
129     for _, event := range config.HW {
```

```
130     if validateEvent(event) {
131         validEvents = append(validEvents, event)
132     }
133 }
134
135 // Split the events into manageable chunks (TODO: dynamically adjust
136 // based on CPU performance)
137 eventChunks := chunkEvents(validEvents, 6)
138
139 // Start a background goroutine to periodically collect performance
140 // data
141 go func() {
142     for {
143         var allPerfData []string
144         for _, chunk := range eventChunks {
145             data, err := getPerfData(chunk)
146             if err != nil {
147                 fmt.Println("Error:", err)
148                 allPerfData = append(allPerfData, "Error fetching data")
149             } else {
150                 allPerfData = append(allPerfData, parsePerfData(data,
151                     chunk))
152             }
153         }
154         // Combine data from all chunks into one string for Prometheus
155         // export
156         perfData = strings.Join(allPerfData, "\n")
157         time.Sleep(100 * time.Millisecond)
158     }
159 }()
160
161 // Start the HTTP server that exposes the '/metrics' endpoint for
162 // Prometheus scraping
163 http.HandleFunc("/metrics", metricsHandler)
164 fmt.Println("Starting server on :9011")
165 if err := http.ListenAndServe(":9011", nil); err != nil {
166     fmt.Println("Error starting server:", err)
167 }
168 }
```

Listing 1: Go script for performance monitoring via perf for Prometheus

```
1 import requests
2 import csv
3 from datetime import datetime, timedelta
4 from collections import defaultdict
5 import numpy as np
6
7 from config_allevents import FEATURES
```

```
8
9 PROMETHEUS_URL = "http://192.168.1.25:9090"
10 STEP = "5" # seconds
11 CHUNK_HOURS = 2
12
13 start_time = datetime.fromisoformat("2025-07-22T18:00:00.000000")
14 end_time = datetime.fromisoformat("2025-07-23T14:40:00.000000")
15
16 def build_chunks(start, end, chunk_hours):
17     chunks = []
18     chunk_delta = timedelta(hours=chunk_hours)
19     current_start = start
20     while current_start < end:
21         current_end = min(current_start + chunk_delta, end)
22         chunks.append((
23             current_start.isoformat("T") + "Z",
24             current_end.isoformat("T") + "Z"
25         ))
26         current_start = current_end
27     return chunks
28
29 def query_metric_chunked(metric, chunks):
30     all_results = []
31     for chunk_start, chunk_end in chunks:
32         url = f"{PROMETHEUS_URL}/api/v1/query_range"
33         params = {
34             "query": metric,
35             "start": chunk_start,
36             "end": chunk_end,
37             "step": STEP,
38         }
39         response = requests.get(url, params=params)
40         response.raise_for_status()
41         all_results.extend(response.json()["data"]["result"])
42     return all_results
43
44 def main():
45     chunks = build_chunks(start_time, end_time, CHUNK_HOURS)
46     data = defaultdict(dict)
47     timestamps = set()
48     values_dict = {}
49     metric_query_map = {}
50
51     for feat in FEATURES:
52         prom_name = feat.replace("-", "_")
53         url = f"{PROMETHEUS_URL}/api/v1/query"
54         hit = False
55
```

```
56     # 1. sum(prom_name) abfragen
57     query = f"sum({prom_name})"
58     params = {'query': query}
59     try:
60         resp = requests.get(url, params=params)
61         resp.raise_for_status()
62         results = resp.json()['data']['result']
63         if results:
64             value = float(results[0]['value'][1])
65             values_dict[feat] = value
66             metric_query_map[feat] = query
67             hit = True
68     except Exception:
69         pass
70
71     if hit:
72         continue
73
74     # 2. sum(perf_prom_name) abfragen
75     prom_name_perf = 'perf_' + prom_name
76     query = f"sum({prom_name_perf})"
77     params = {'query': query}
78     try:
79         resp = requests.get(url, params=params)
80         resp.raise_for_status()
81         results = resp.json()['data']['result']
82         if results:
83             value = float(results[0]['value'][1])
84             values_dict[feat] = value
85             metric_query_map[feat] = query
86             hit = True
87     except Exception:
88         pass
89
90     values_array = []
91     feature_names_for_array = []
92
93     for feat in FEATURES:
94         feature_names_for_array.append(feat)
95         values_array.append(values_dict.get(feat, np.nan))
96
97     for feat, querystr in metric_query_map.items():
98         results = query_metric_chunked(querystr, chunks)
99         for series in results:
100             for timestamp, value in series["values"]:
101                 ts = datetime.utcfromtimestamp(float(timestamp)).
102                     isoformat()
103                 data[ts][feat] = value
```

```
103         timestamps.add(ts)
104
105     sorted_timestamps = sorted(timestamps)
106
107     all_columns = list(FEATURES)
108
109     with open("evse_metrics_wide.csv", "w", newline="") as f:
110         writer = csv.writer(f)
111         header = ["timestamp"] + all_columns
112         writer.writerow(header)
113         for ts in sorted_timestamps:
114             row = [ts] + [data[ts].get(col, "") for col in all_columns]
115             writer.writerow(row)
116
117 if __name__ == "__main__":
118     main()
```

Listing 2: Python script for scraping performance monitoring from Prometheus into an CSV file

```
1  import subprocess
2  import time
3  from datetime import datetime
4
5  commands = [
6      ("Slowloris", ["python3", "slowloris.py", "192.168.1.25", "-s", "500"
7          ]),
8      ("UDP Flood", ["sudo", "hping3", "--flood", "--udp", "-p", "80", "
9          192.168.1.25"]),
10     ("ICMP Flood", ["sudo", "hping3", "--flood", "-i", "1", "192.168.1.25"]),
11     ("TCP Flood", ["sudo", "hping3", "--flood", "-p", "80", "192.168.1.25
12         "]),
13     ("Synonymous Flood", ["sudo", "hping3", "--flood", "--rand-source", "
14         -p", "80", "192.168.1.25"]),
15     ("Portscan", ["bash", "-c", "for i in {1..1000}; do sudo nmap -sT
16         192.168.1.25; sleep 1; done"]),
17     ("OS Fingerprinting", ["bash", "-c", "for i in {1..1000}; do sudo
18         nmap -O 192.168.1.25; sleep 1; done"]),
19     ("Aggressive Scan", ["bash", "-c", "for i in {1..1000}; do sudo nmap
20         -A 192.168.1.25; sleep 1; done"]),
21 ]
22
23 LOG_FILE = "timestamp_attacks.txt"
24 RUN_TIME = 15 * 60      # 15 Minuten
25 BREAK_TIME = 20 * 60   # 20 Minuten
26
27 def log_msg(msg):
28     with open(LOG_FILE, "a") as f:
29         f.write(msg + "\n")
```

```

24 while True:
25     for name, cmd in commands:
26         start_time = datetime.now()
27         print(f"Starte {name}: {' '.join(cmd)}")
28         log_msg(f"Start: {start_time.strftime('%Y-%m-%d %H:%M:%S')} -
                Angriff: {name} - Befehl: {' '.join(cmd)}")
29         try:
30             proc = subprocess.Popen(cmd)
31             proc.wait(timeout=RUN_TIME)
32             note = "Erfolgreich beendet"
33         except subprocess.TimeoutExpired:
34             proc.terminate()
35             try:
36                 proc.wait(10)
37             except Exception:
38                 proc.kill()
39             note = "Nach 15 Minuten abgebrochen"
40         end_time = datetime.now()
41         log_msg(f"Ende: {end_time.strftime('%Y-%m-%d %H:%M:%S')} ({note})")
42         log_msg("Pause startet jetzt.\n")
43         time.sleep(BREAK_TIME)

```

Listing 3: Python script for automated attacks and storage of the timestamps

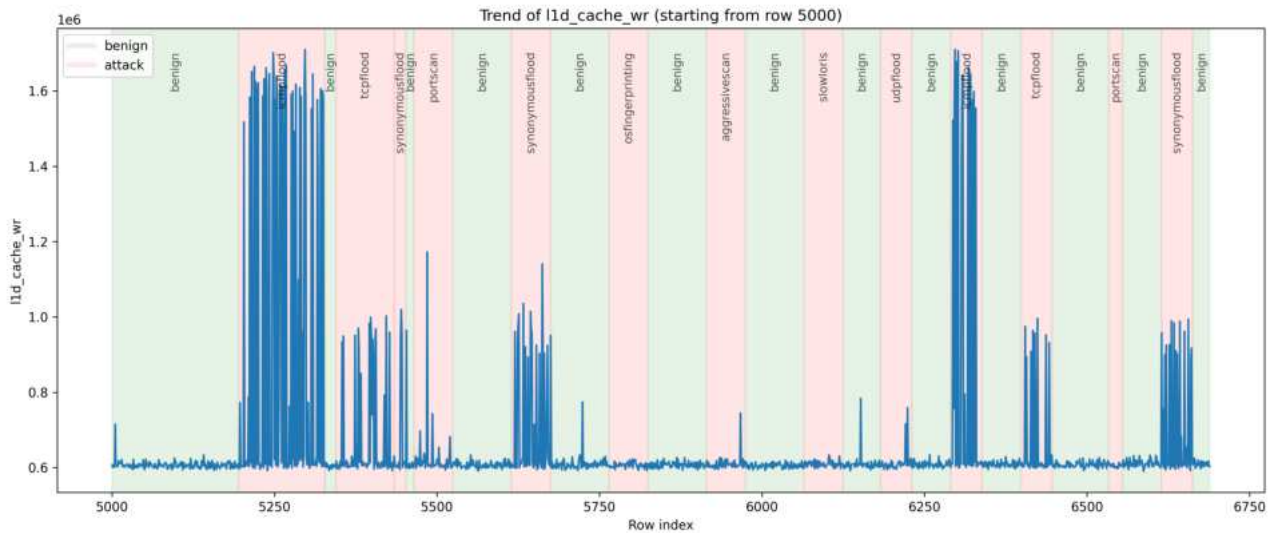


Figure 44: l1d_cache_wr timeline

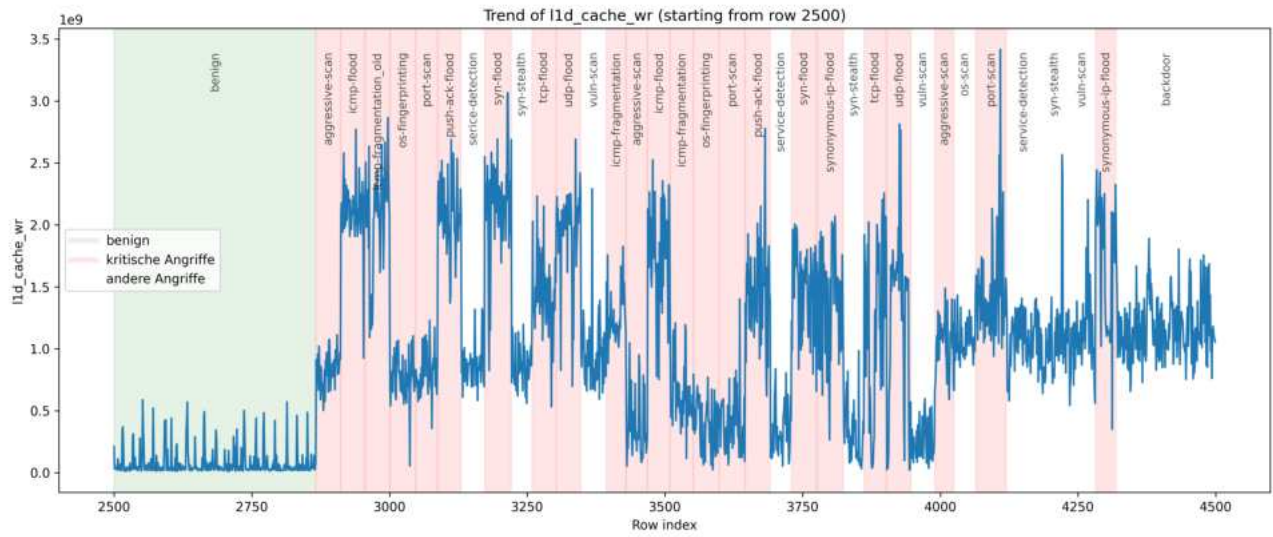


Figure 45: l1d_cache_wr timeline (CICEVSE2024)

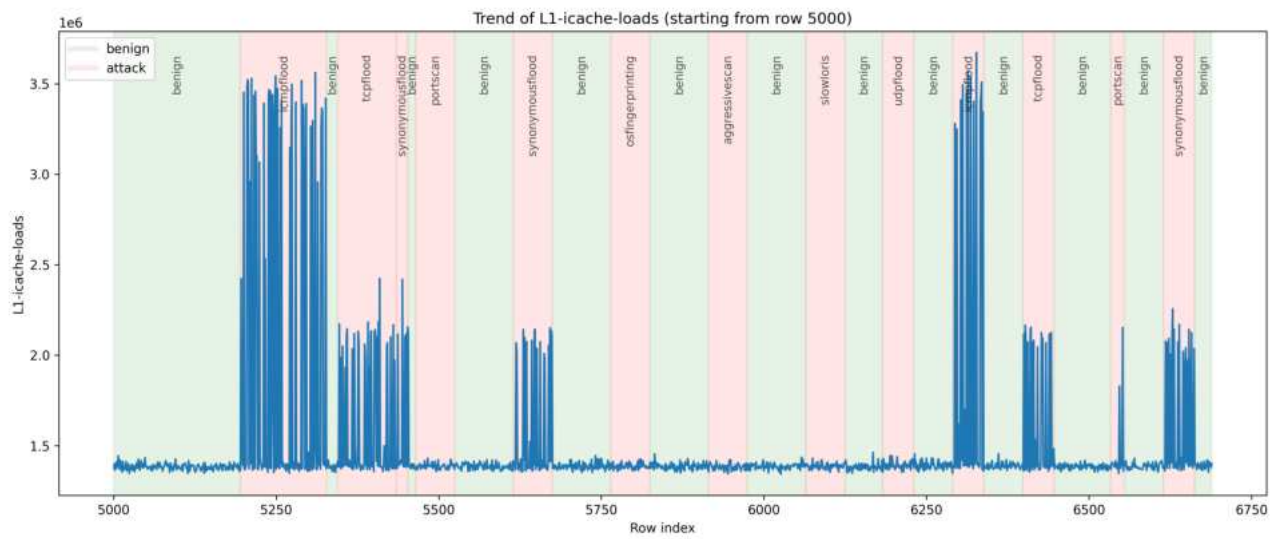


Figure 46: instruction timeline

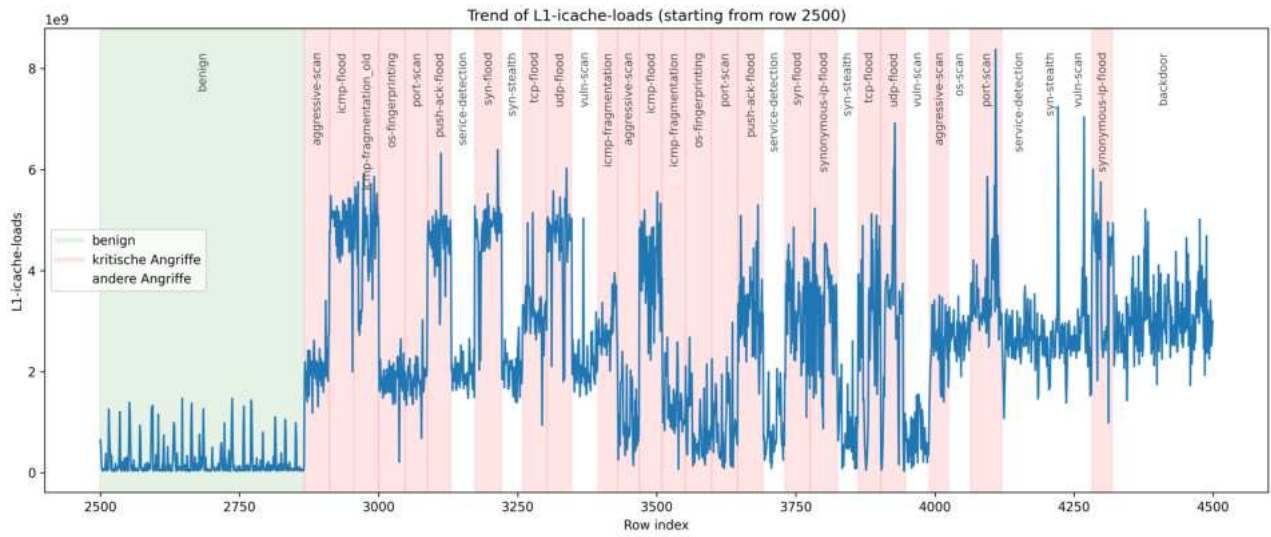


Figure 47: instruction timeline (CICEVSE2024)

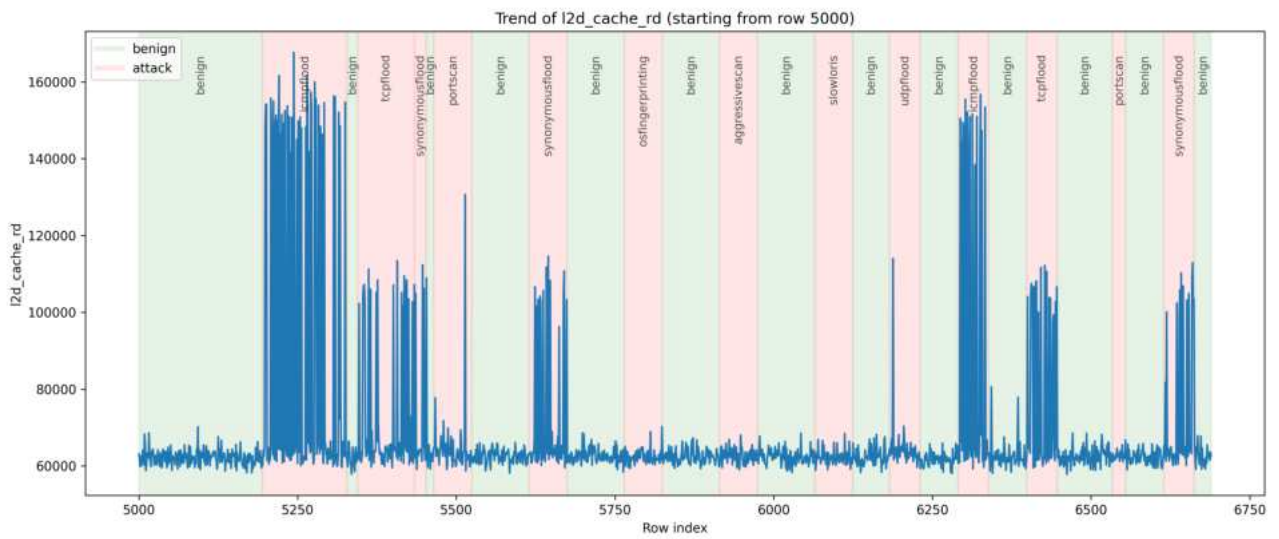


Figure 48: l2d_cache_rd timeline

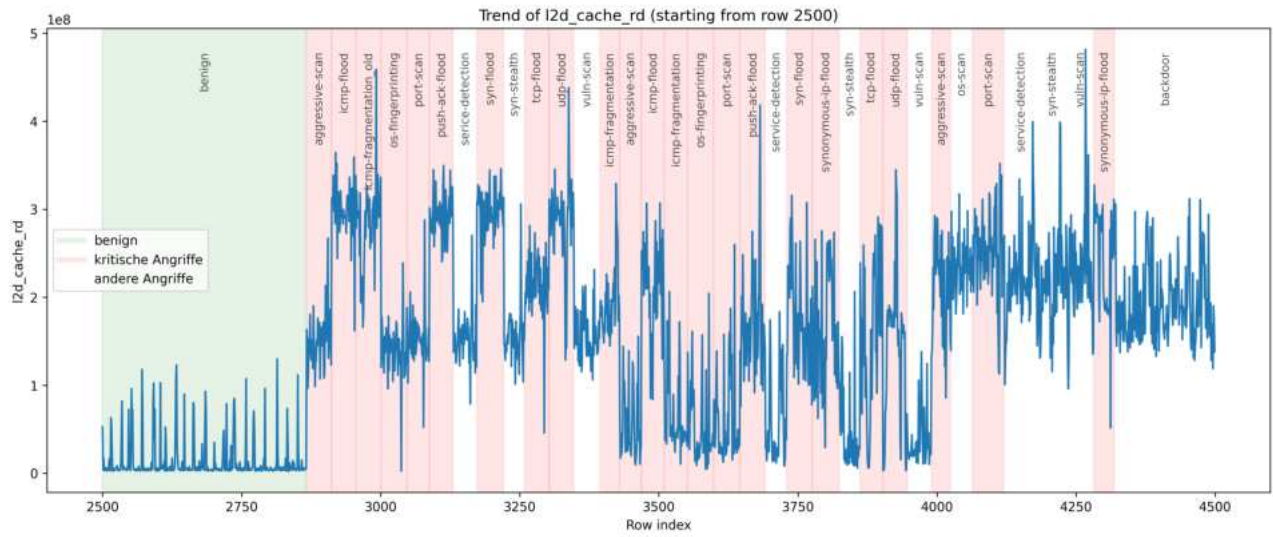


Figure 49: l2d_cache_rd timeline (CICEVSE2024)

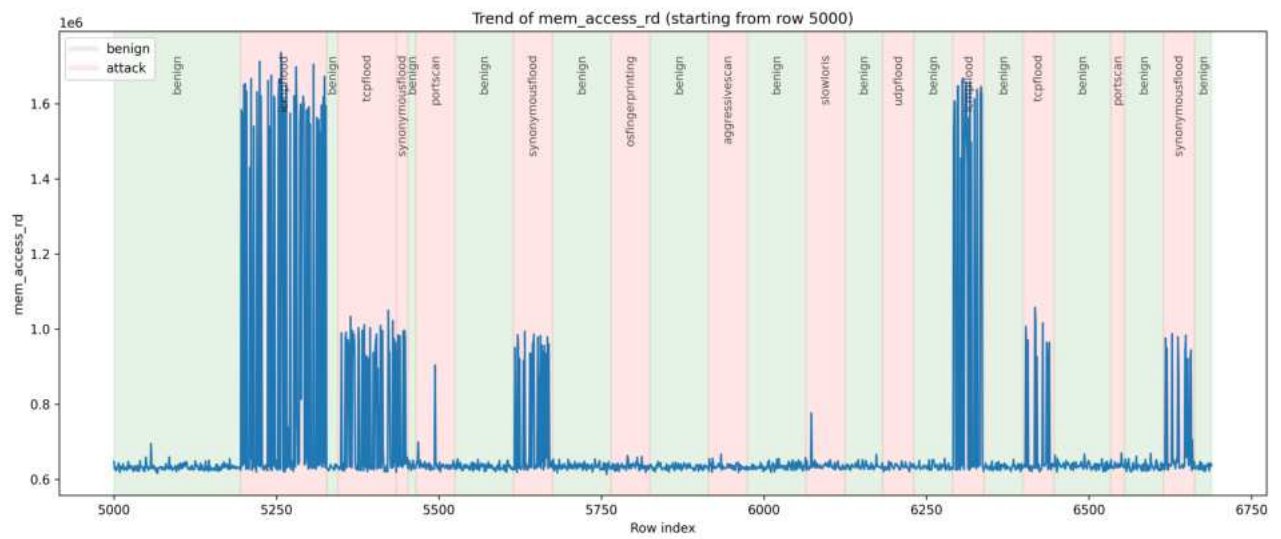


Figure 50: mem_access_rd timeline

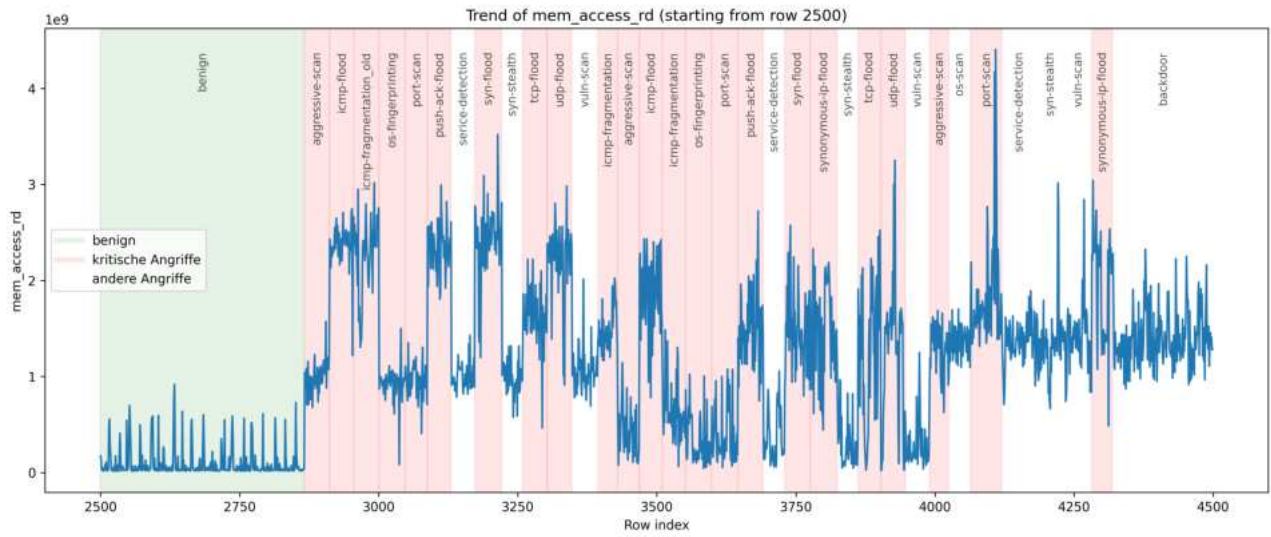


Figure 51: mem_access_rd timeline (CICEVSE2024)

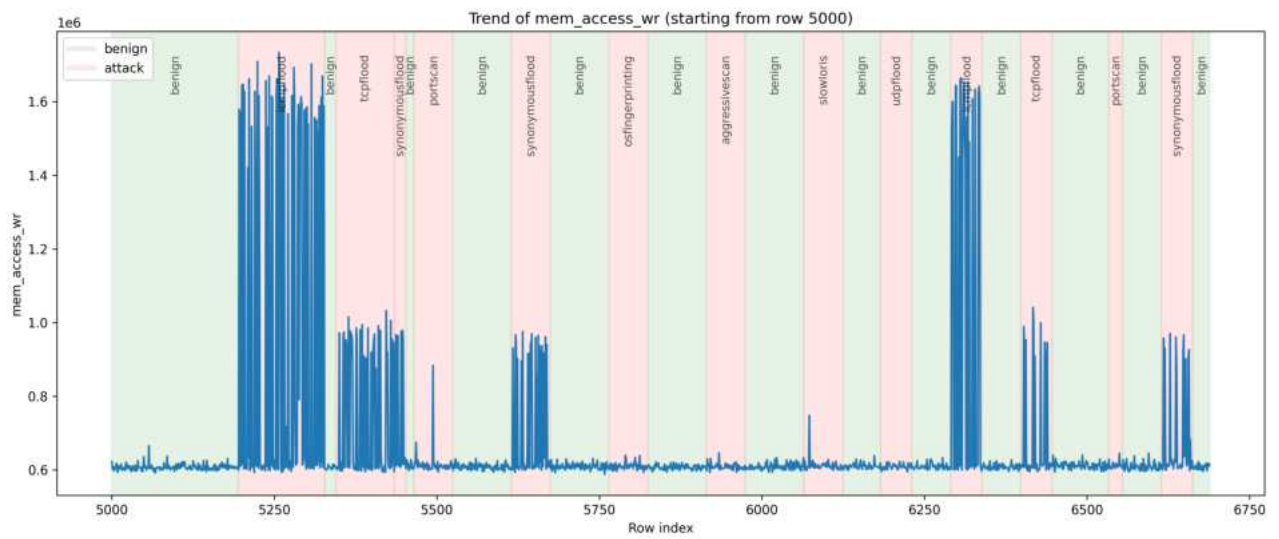


Figure 52: mem_access_wr timeline

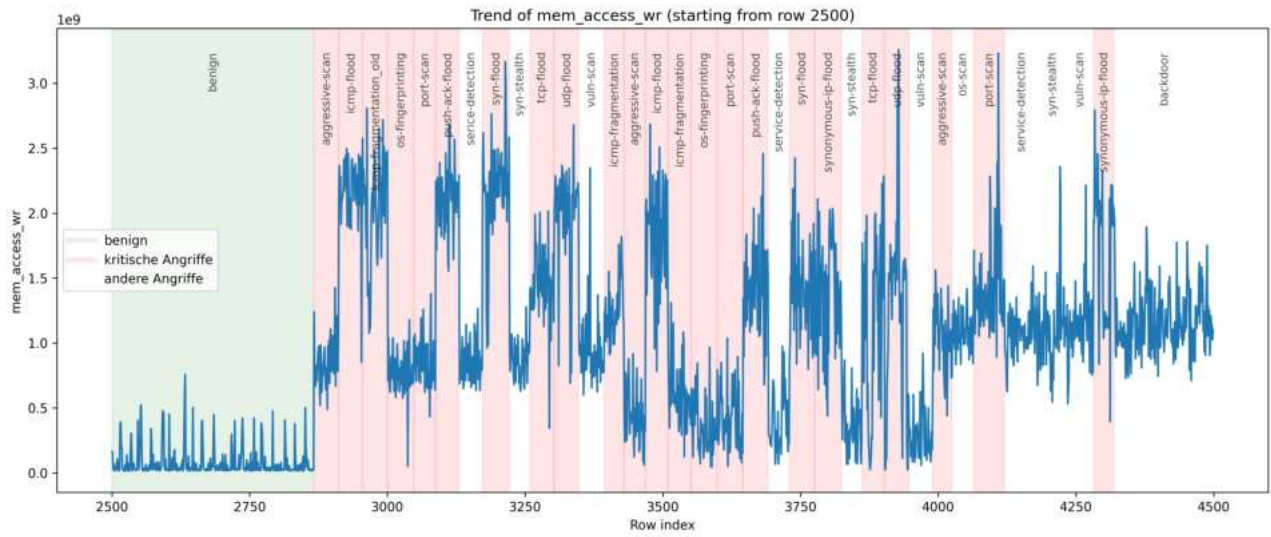


Figure 53: mem_access_wr timeline (CICEVSE2024)

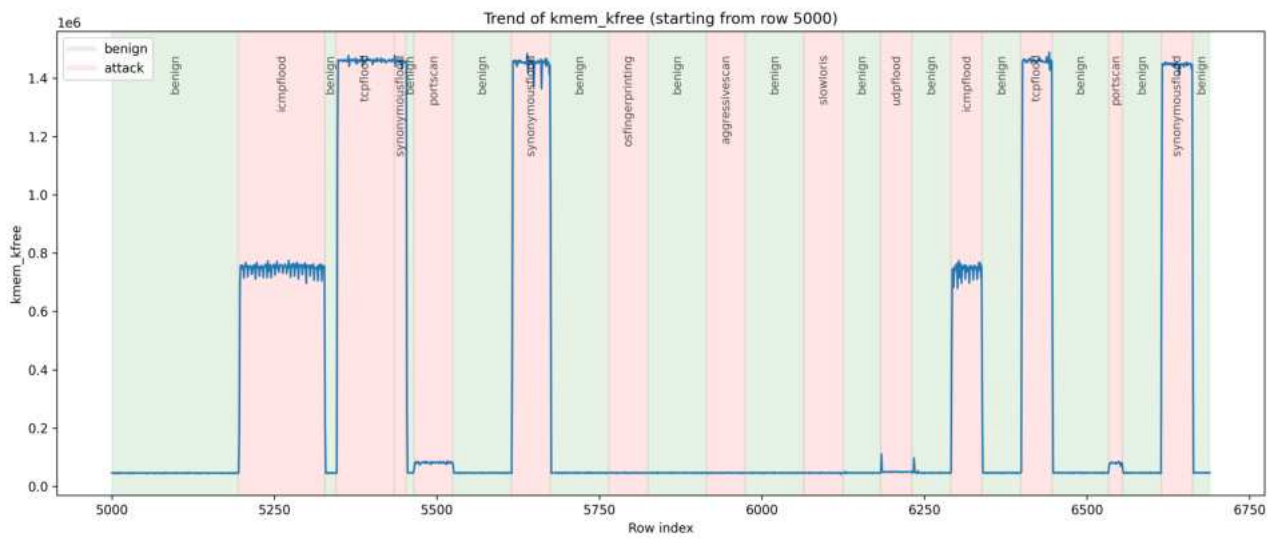


Figure 54: kmem_kfree timeline

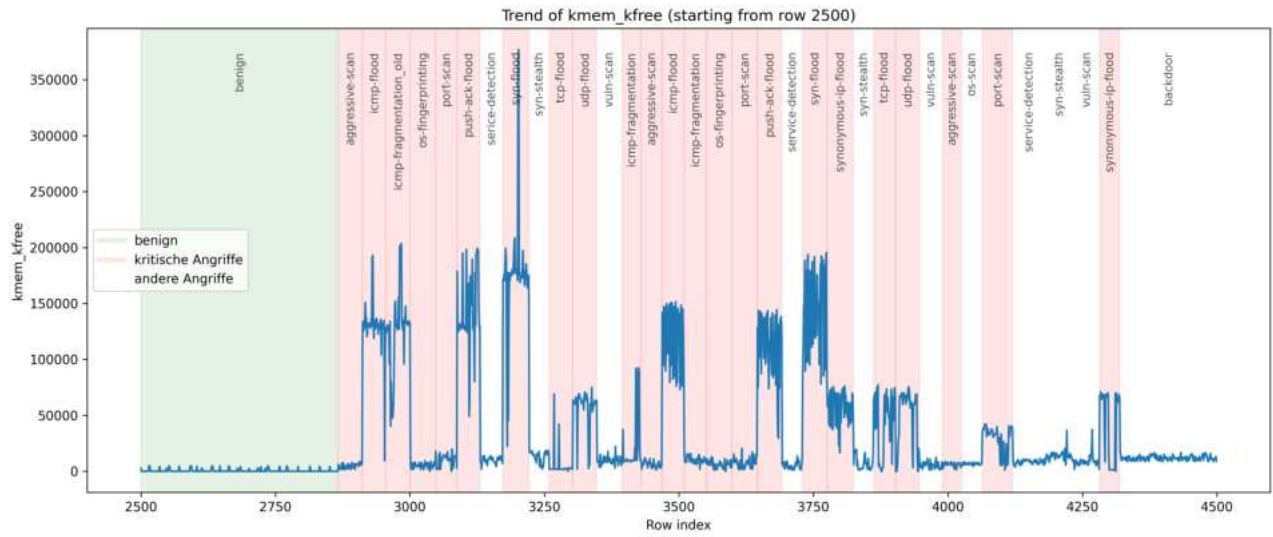


Figure 55: kmem_kfree timeline (CICEVSE2024)

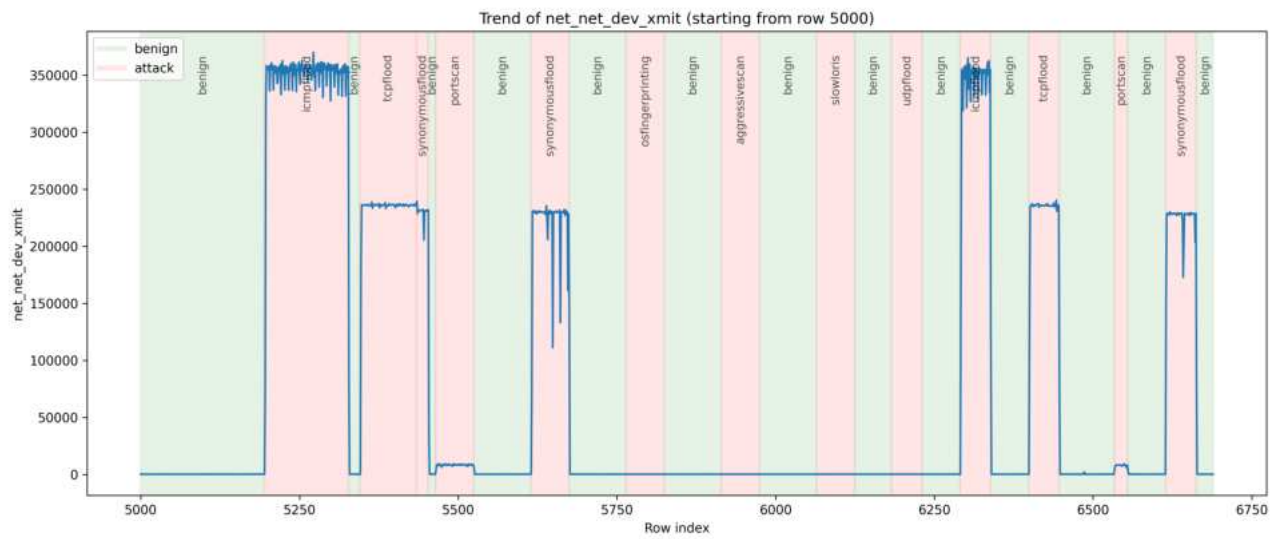


Figure 56: net_dev_xmit timeline

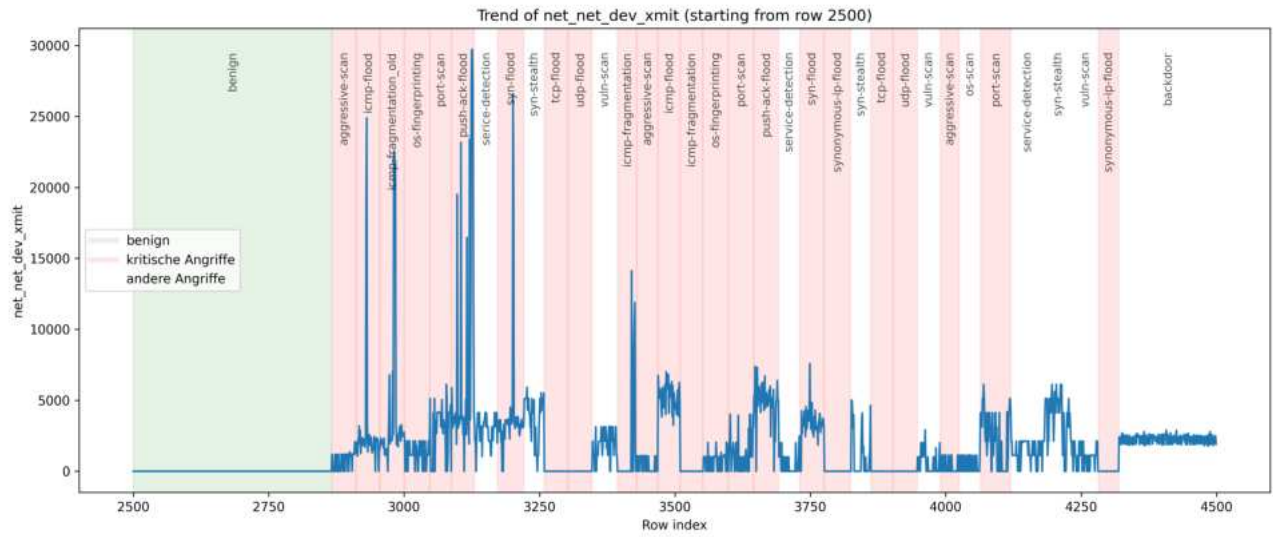


Figure 57: net_dev_xmit timeline (CICEVSE2024)

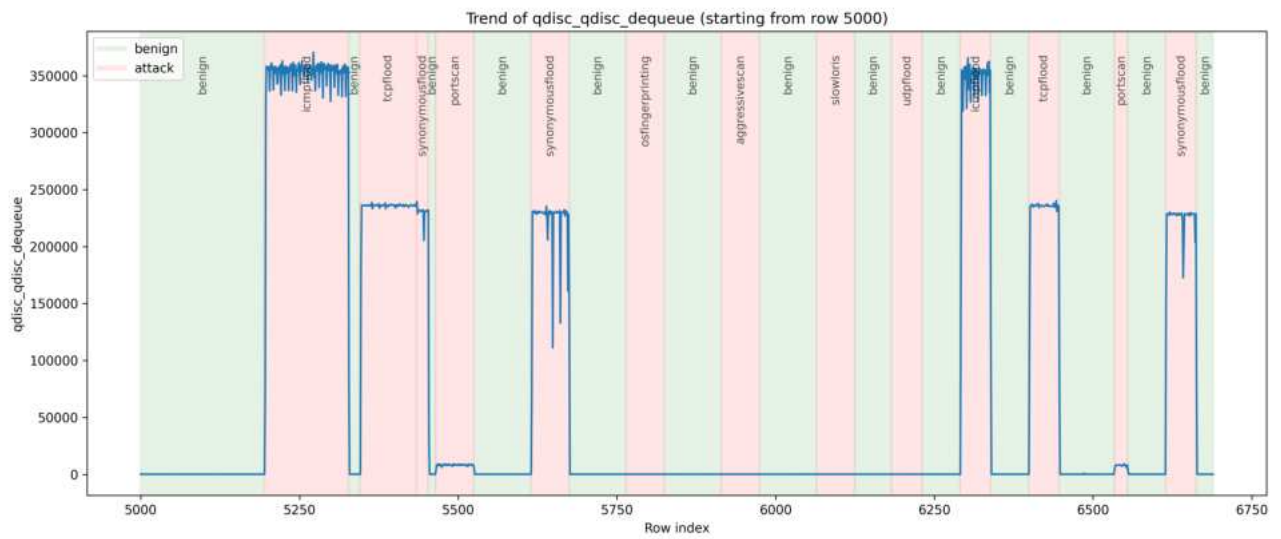


Figure 58: qdisc_dequeue timeline

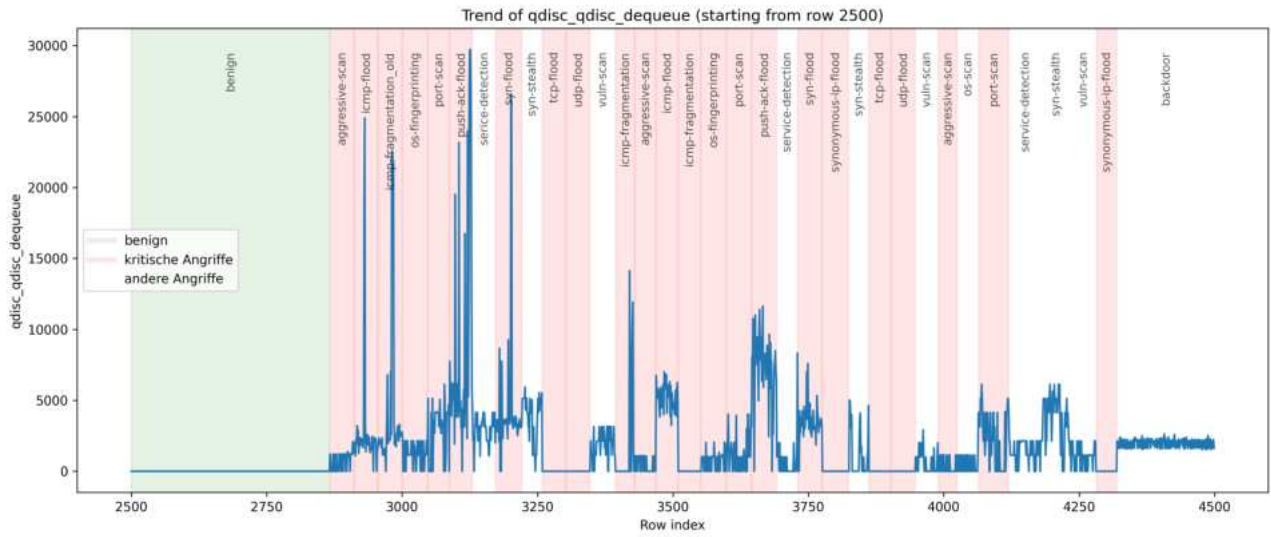


Figure 59: qdisc_dequeue timeline (CICEVSE2024)

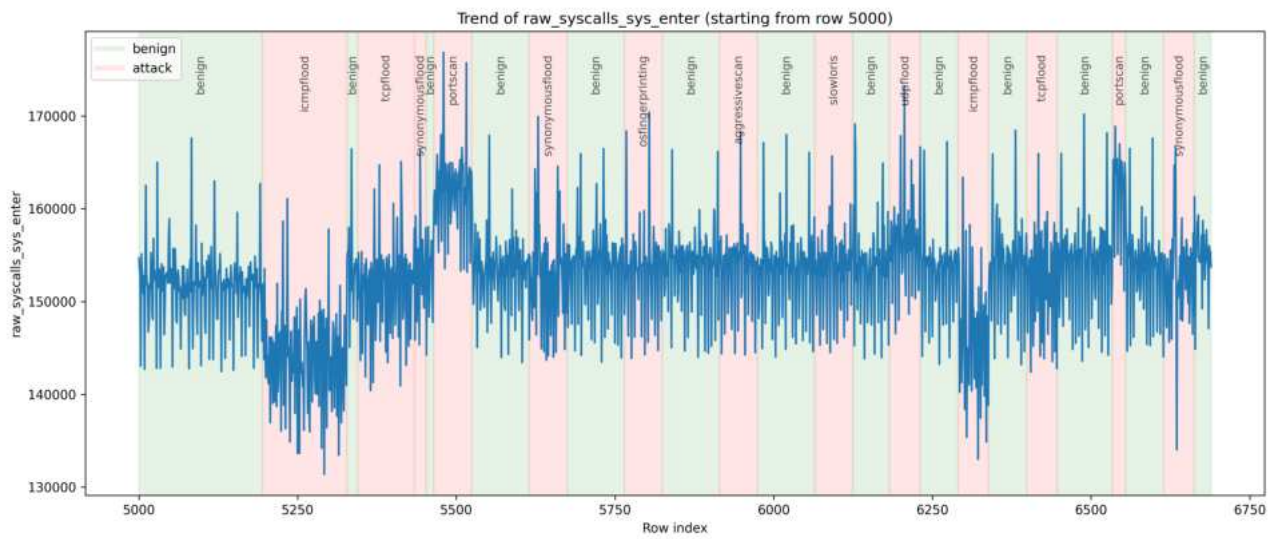


Figure 60: raw_syscalls_sys_enter timeline

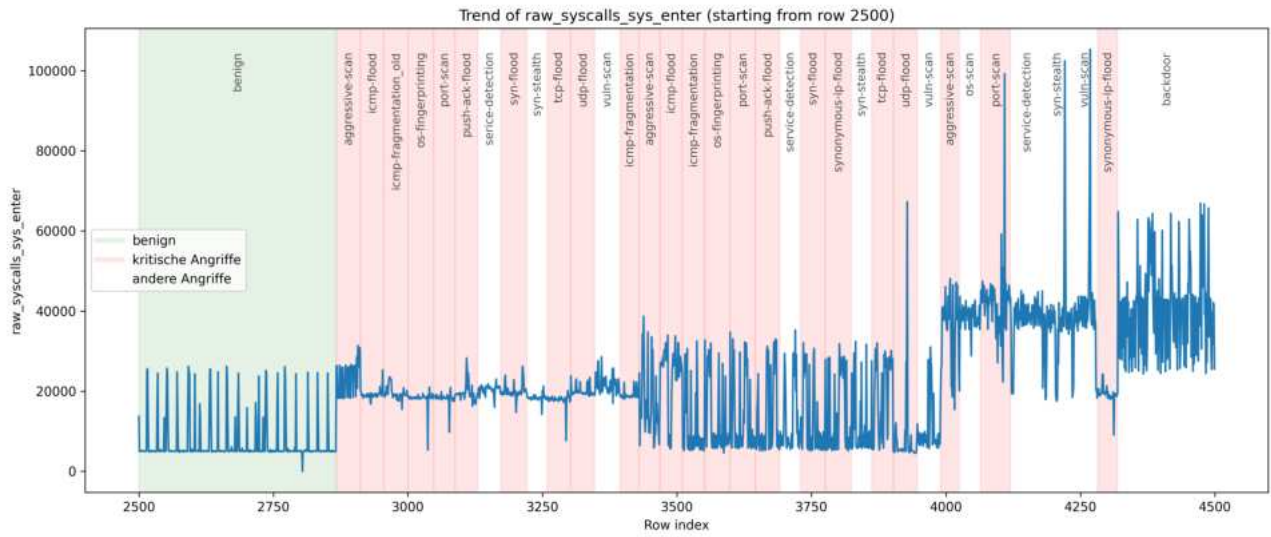


Figure 61: raw_syscalls_sys_enter timeline (CICEVSE2024)

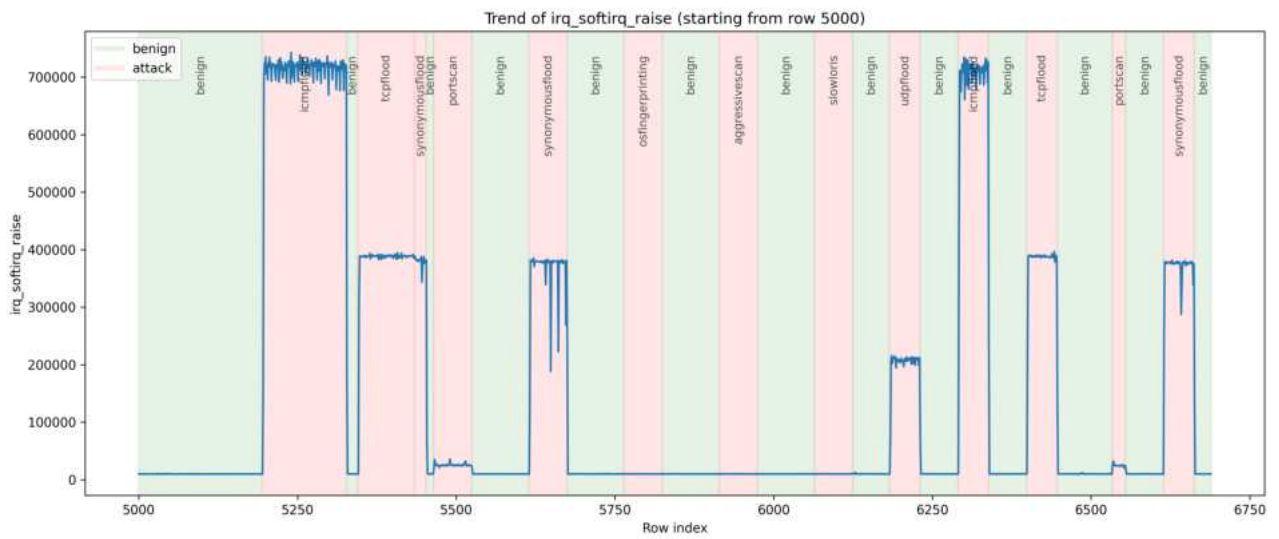


Figure 62: irq_softirq_raise timeline

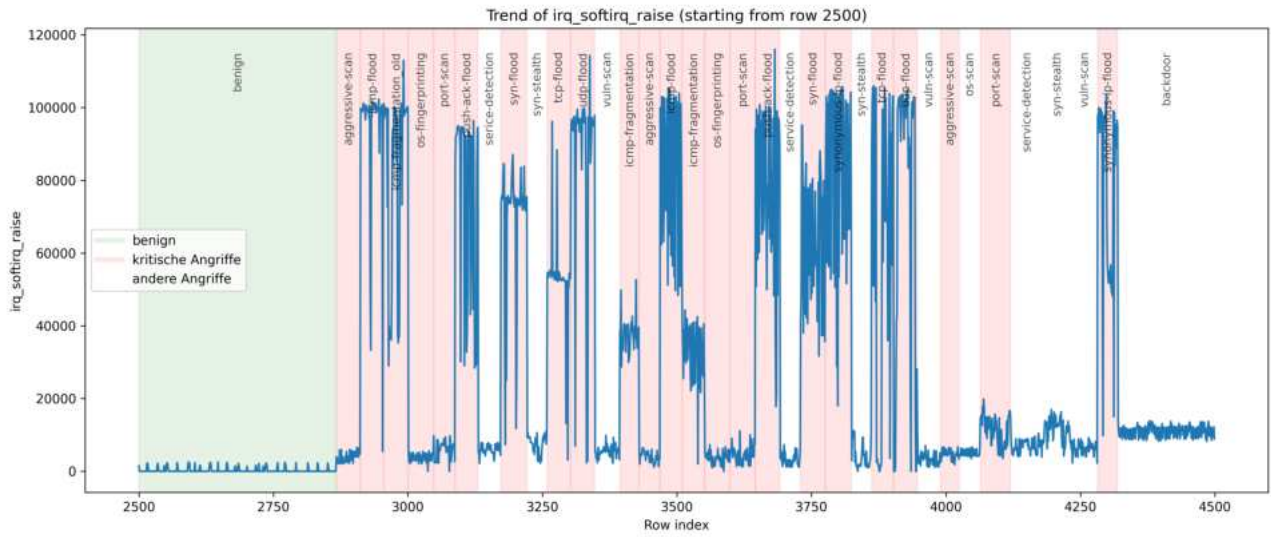


Figure 63: irq_softirq_raise timeline (CICEVSE2024)

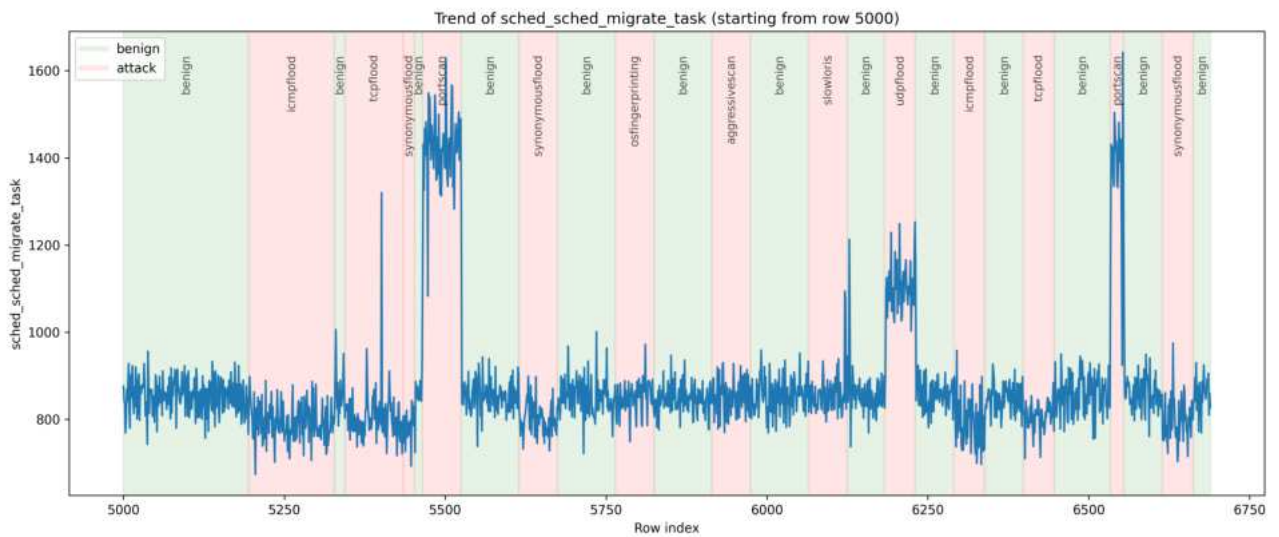


Figure 64: sched_migrate.task timeline

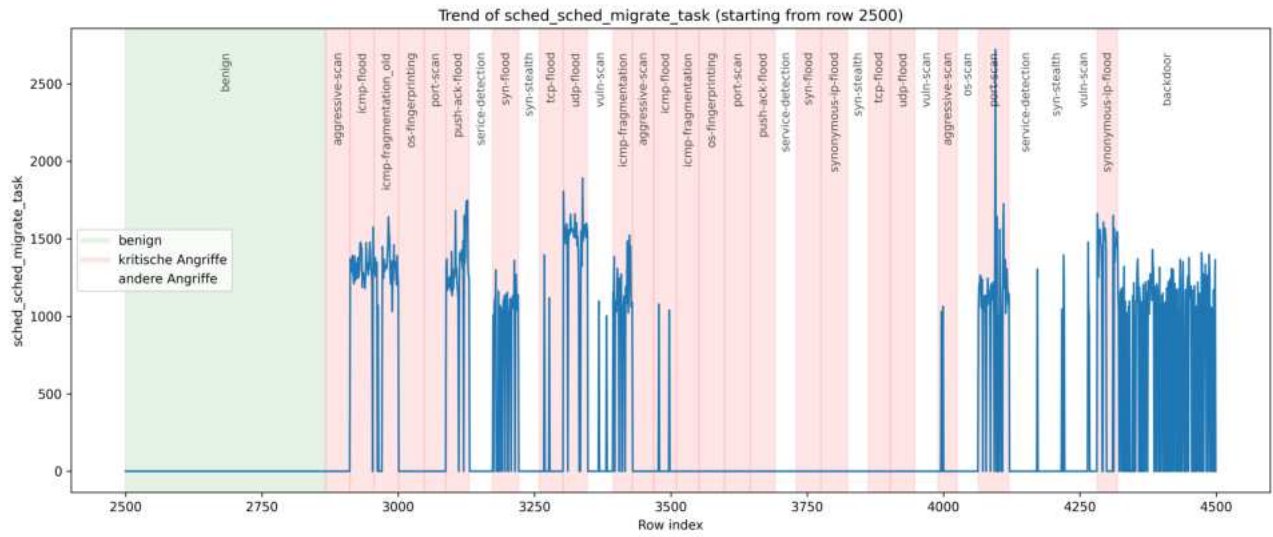


Figure 65: sched_migrate_task timeline (CICEVSE2024)

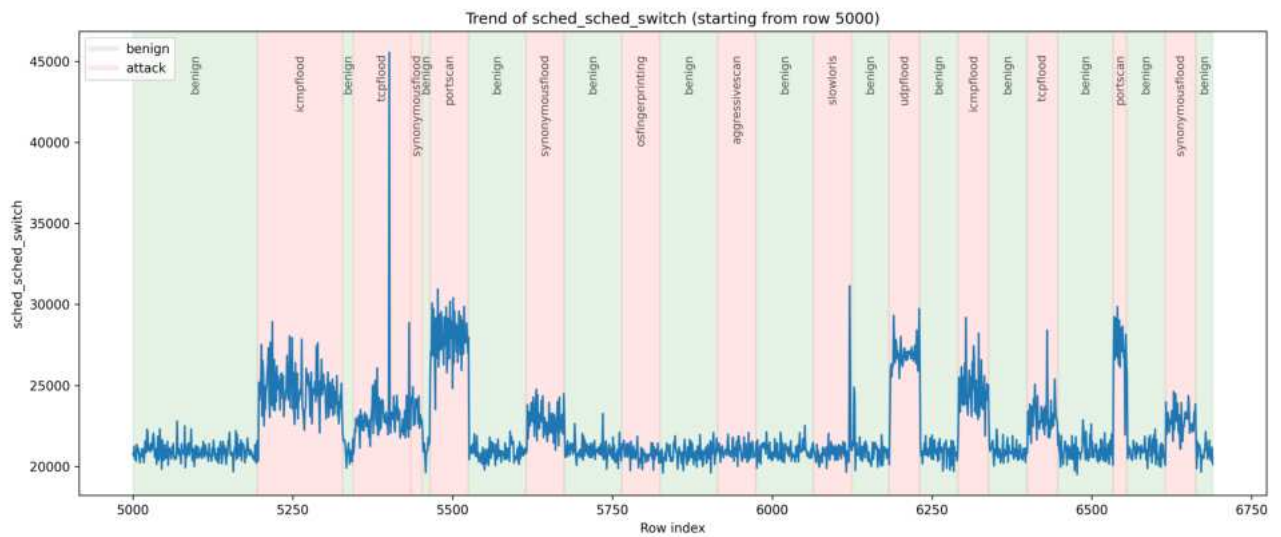


Figure 66: sched_switch timeline

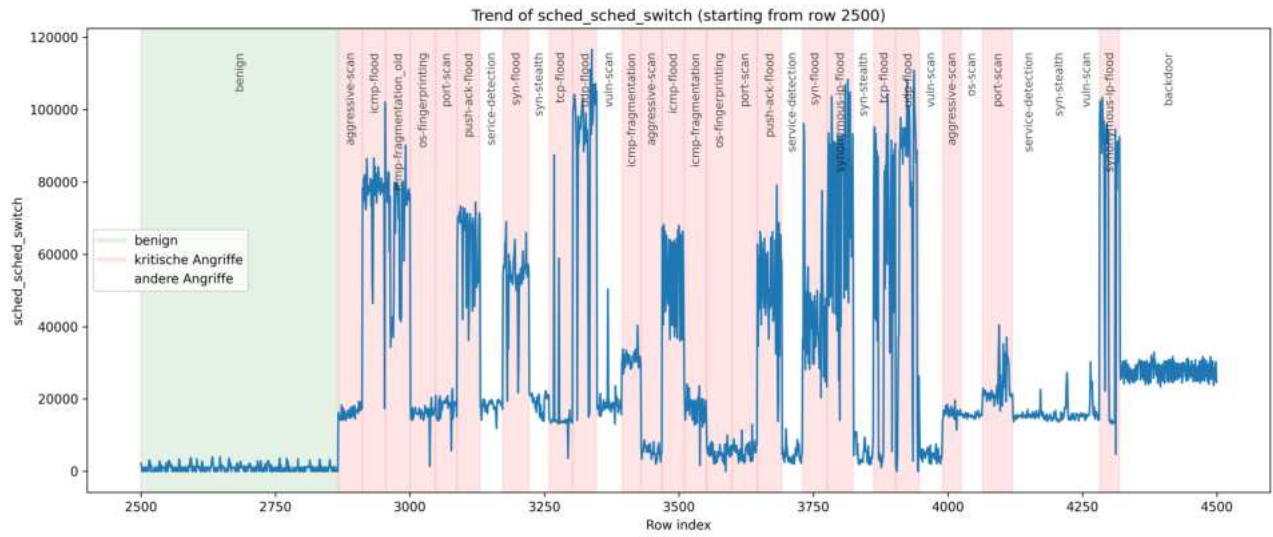


Figure 67: sched_switch timeline (CICEVSE2024)

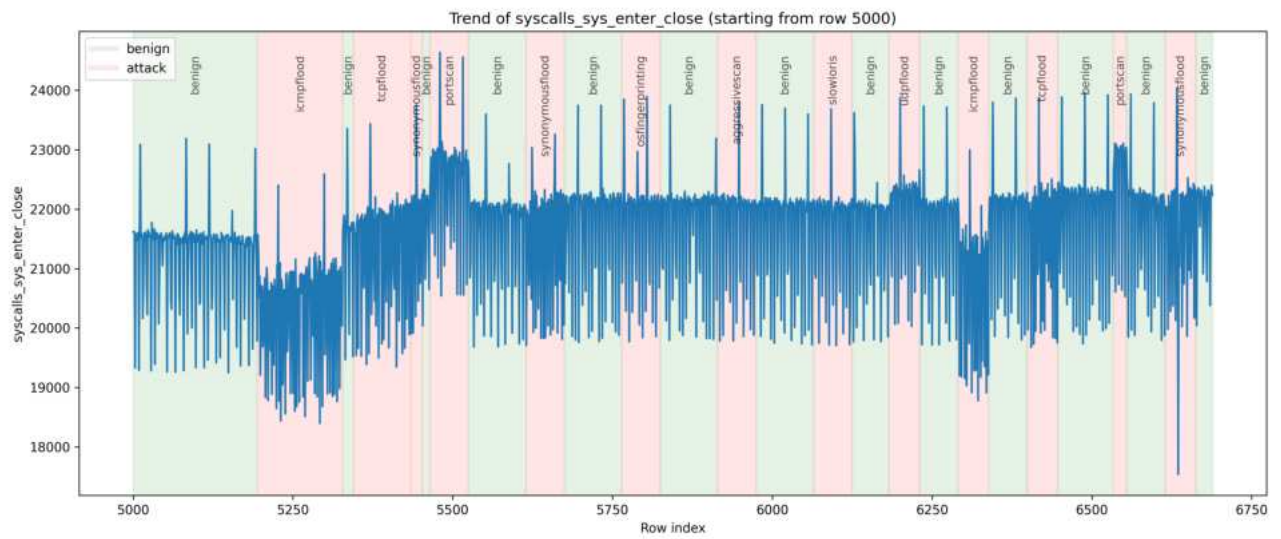


Figure 68: syscalls_sys_enter_close timeline

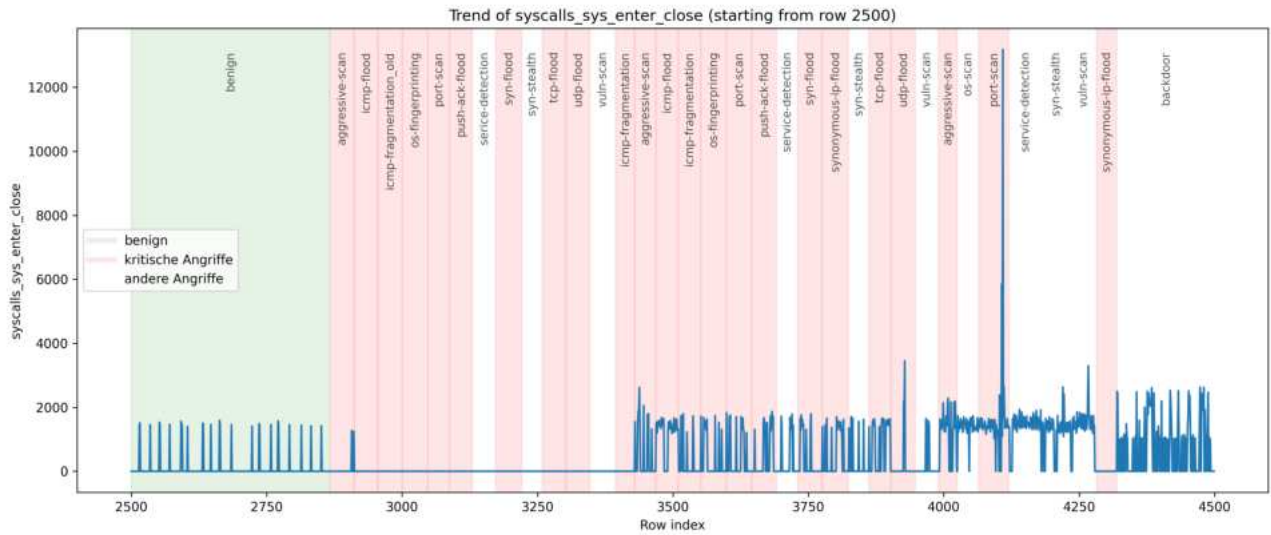


Figure 69: syscalls_sys_enter_close timeline (CICEVSE2024)

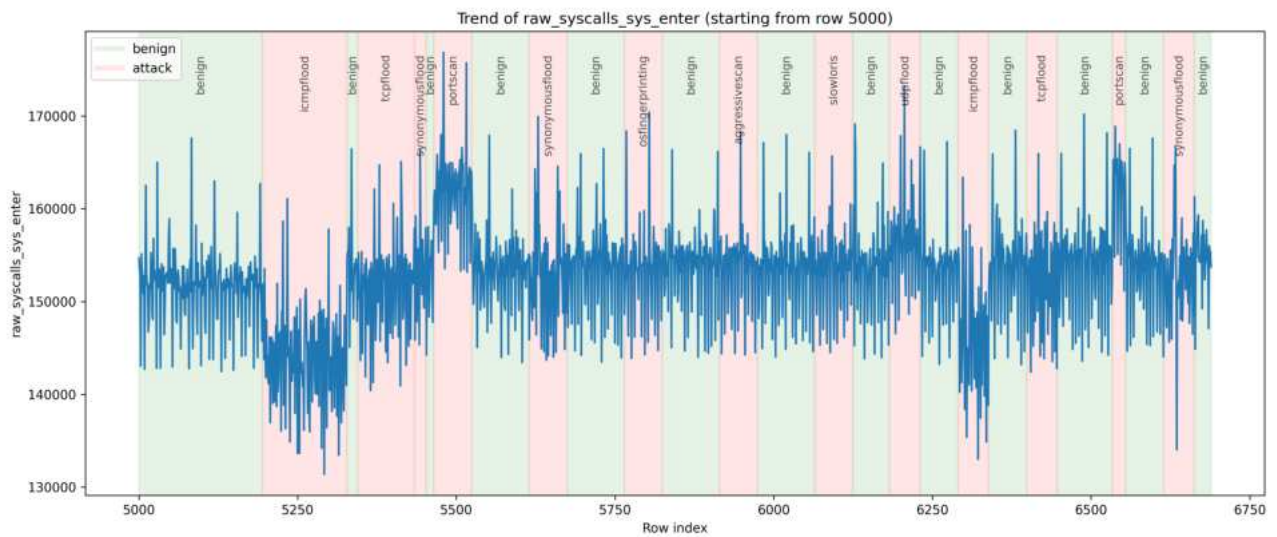


Figure 70: syscalls_sys_enter_read timeline

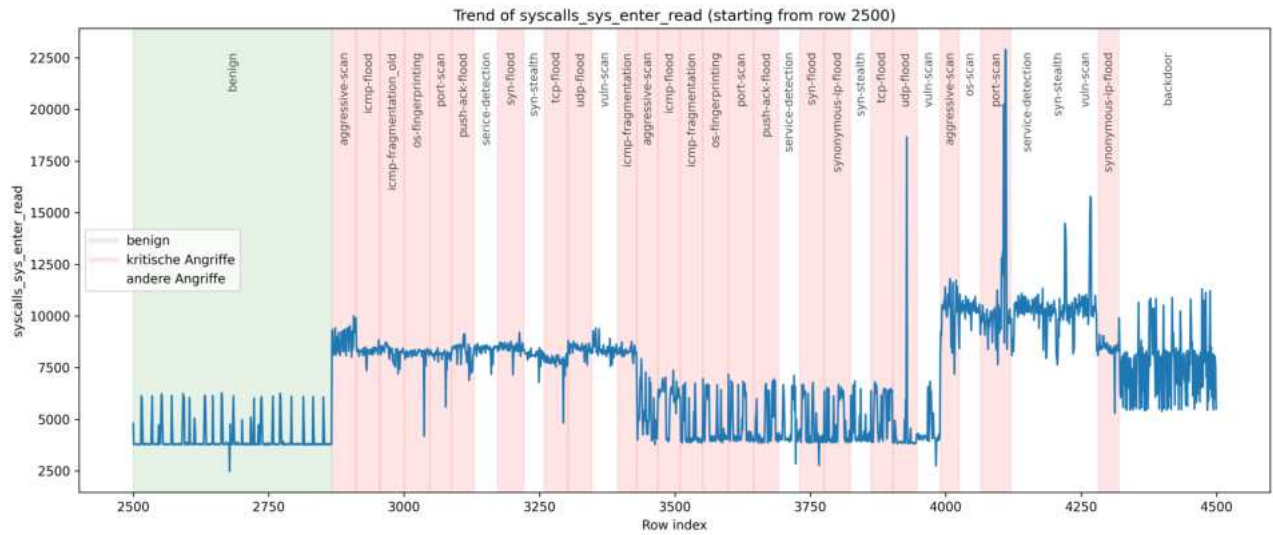


Figure 71: syscalls_sys_enter_read timeline (CICEVSE2024)

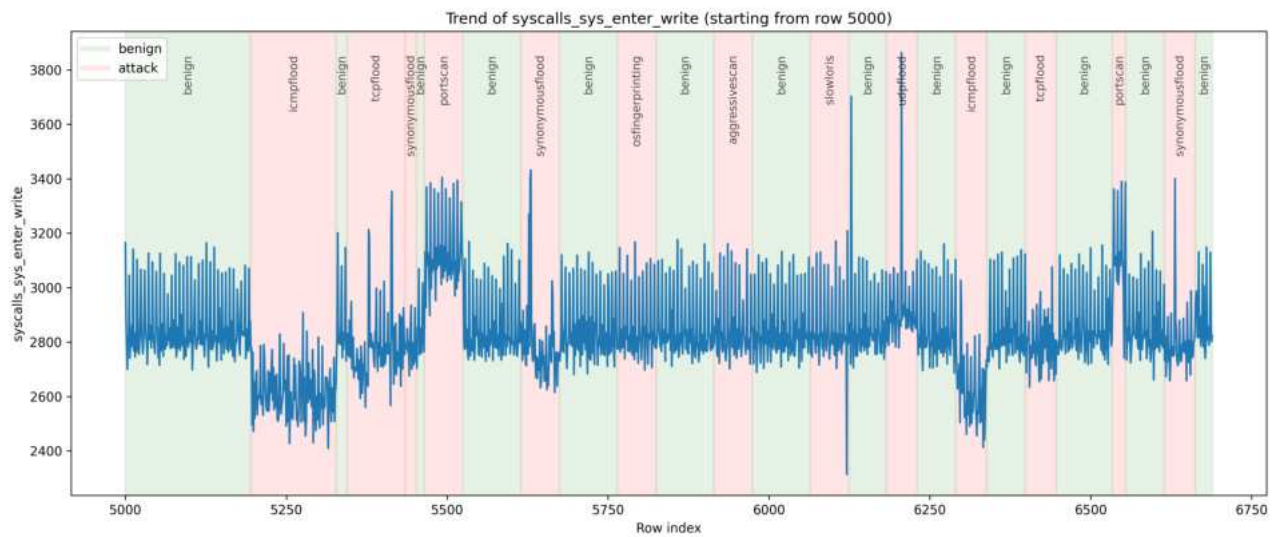


Figure 72: syscalls_sys_enter_write timeline

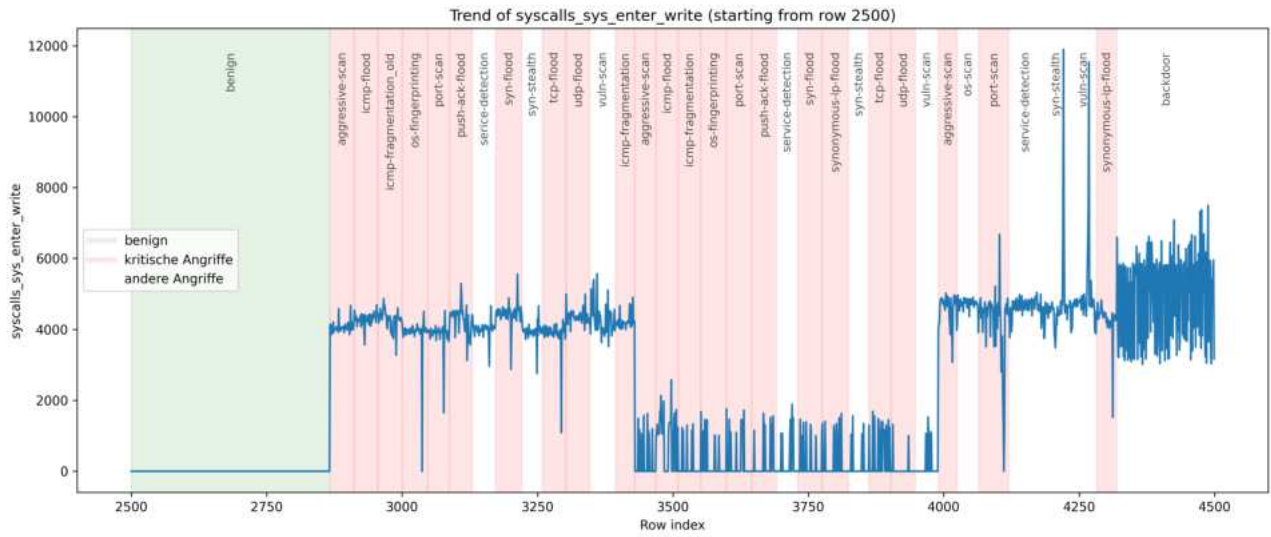


Figure 73: syscalls_sys_enter_write timeline (CICEVSE2024)

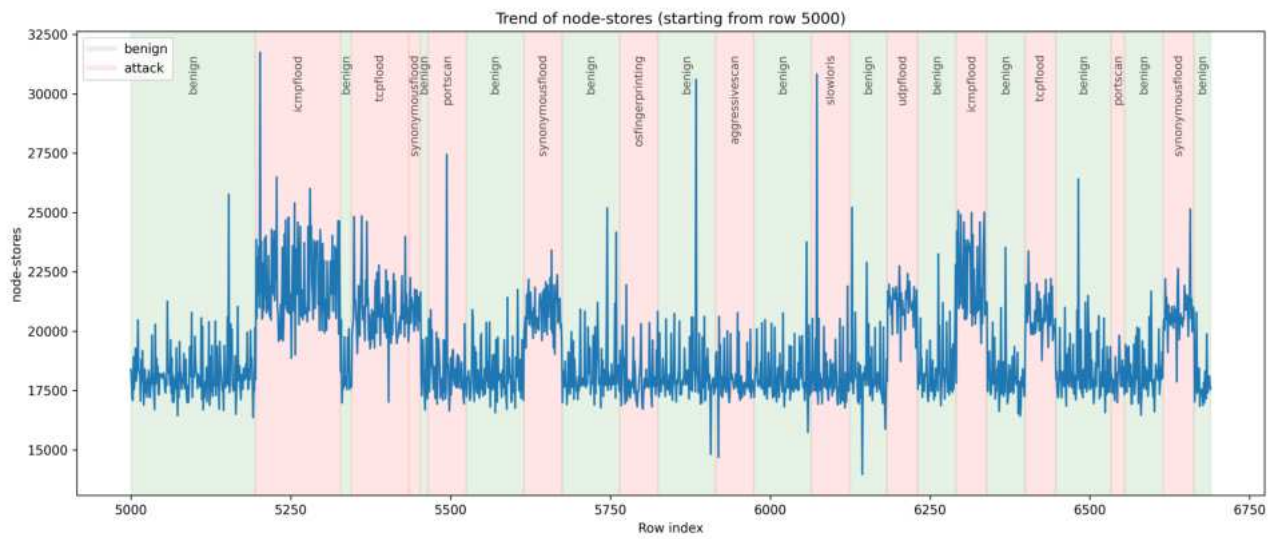


Figure 74: node-stores timeline

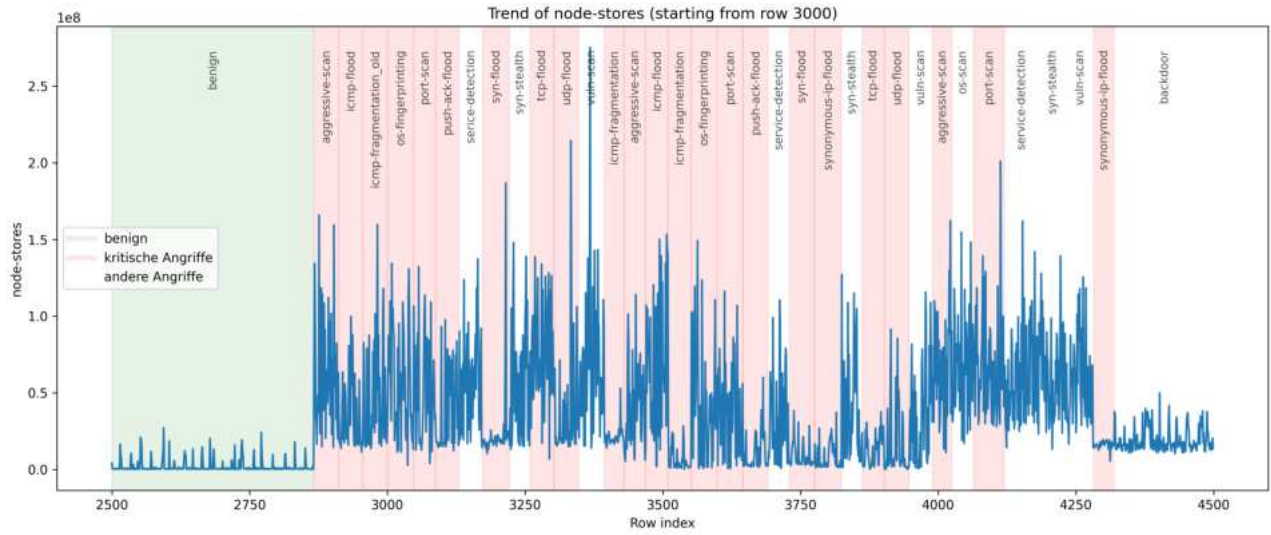


Figure 75: node-stores timeline (CICEVSE2024)

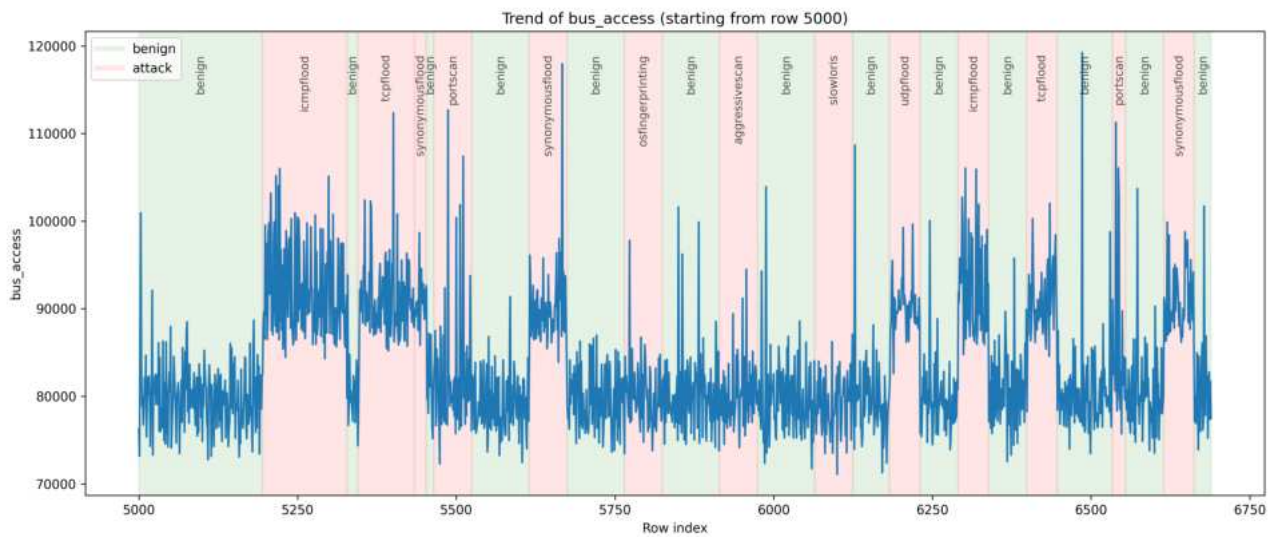


Figure 76: bus.access timeline

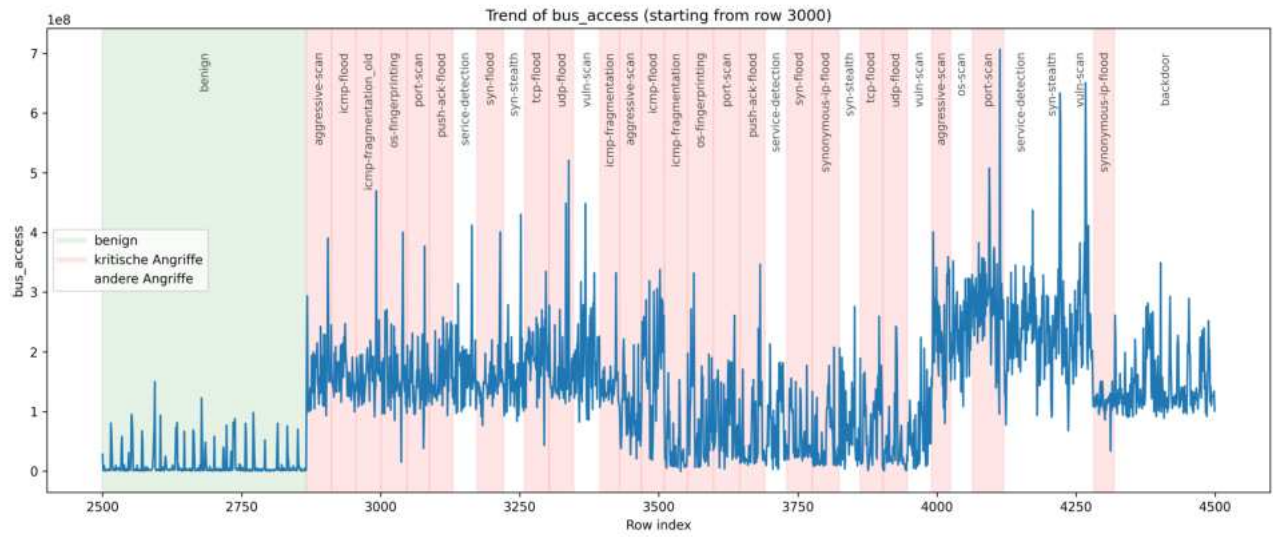


Figure 77: bus_access timeline (CICEVSE2024)

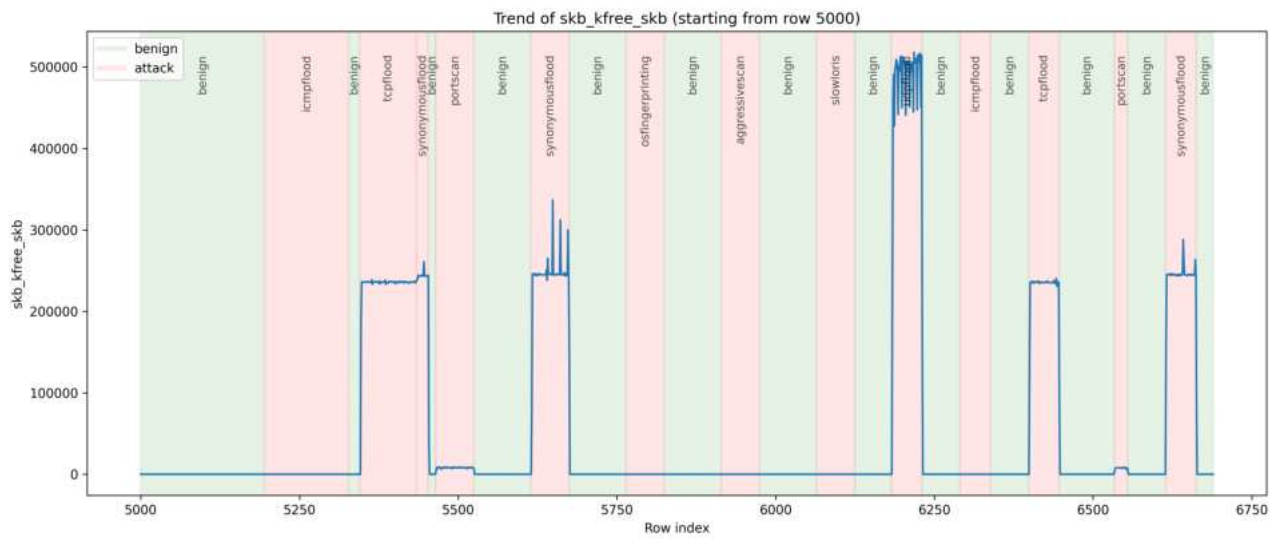


Figure 78: skb_kfree_skb timeline

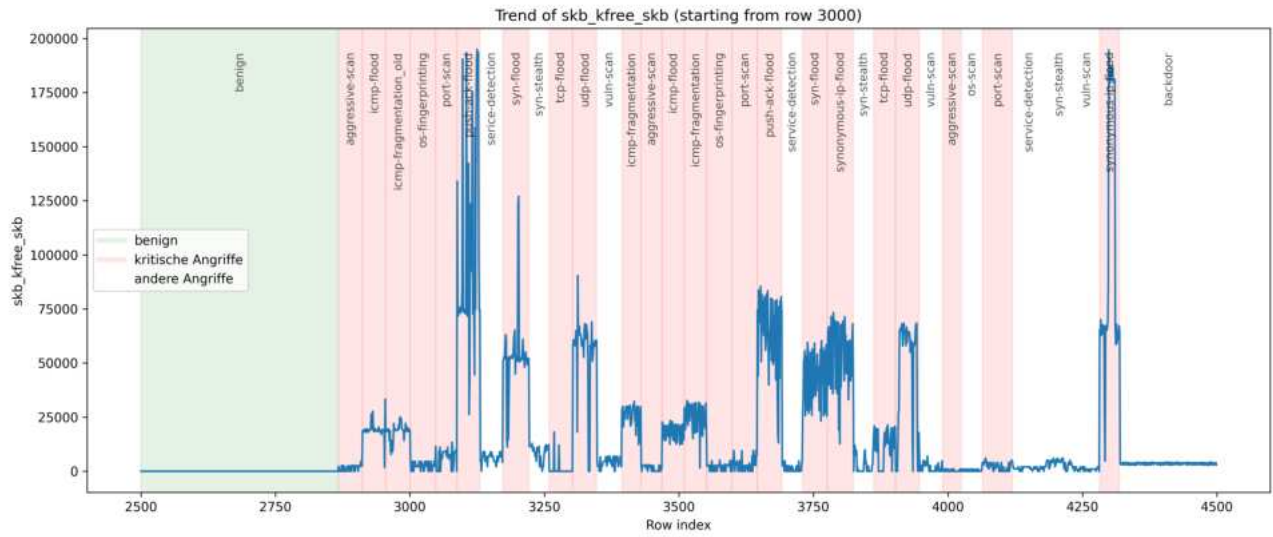


Figure 79: skb_kfree_skb timeline (CICEVSE2024)

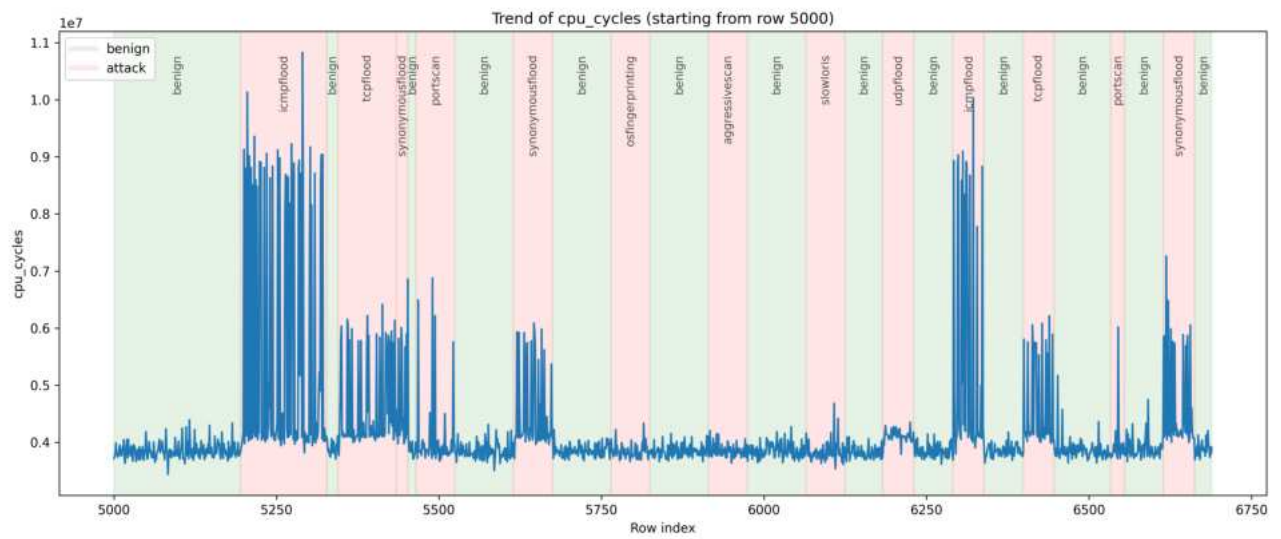


Figure 80: cpu_cycles timeline

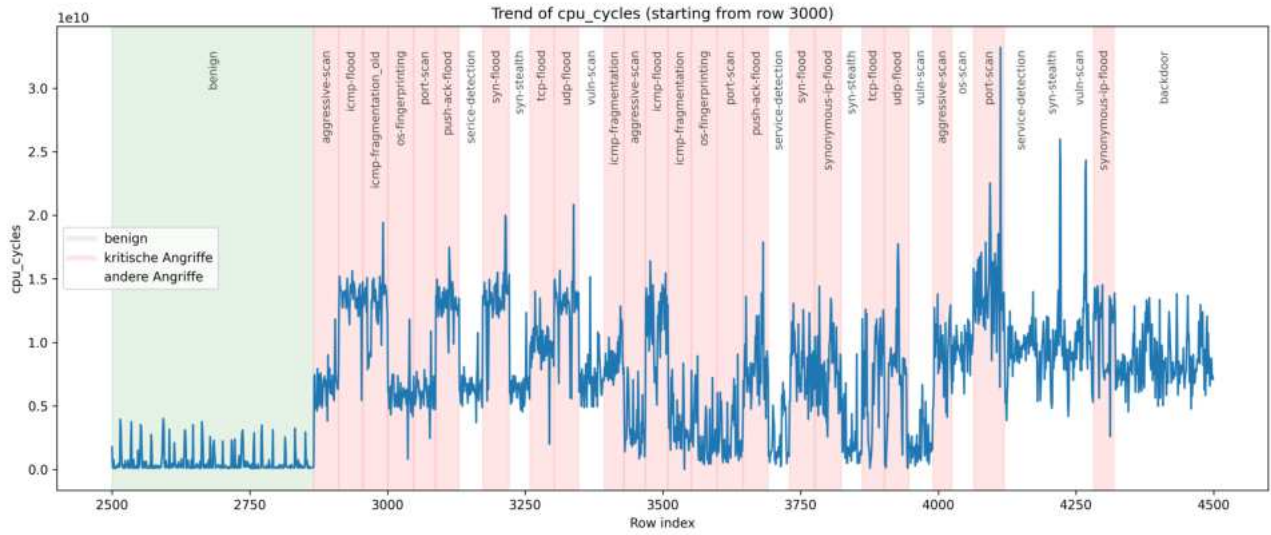


Figure 81: cpu_cycles timeline (CICEVSE2024)

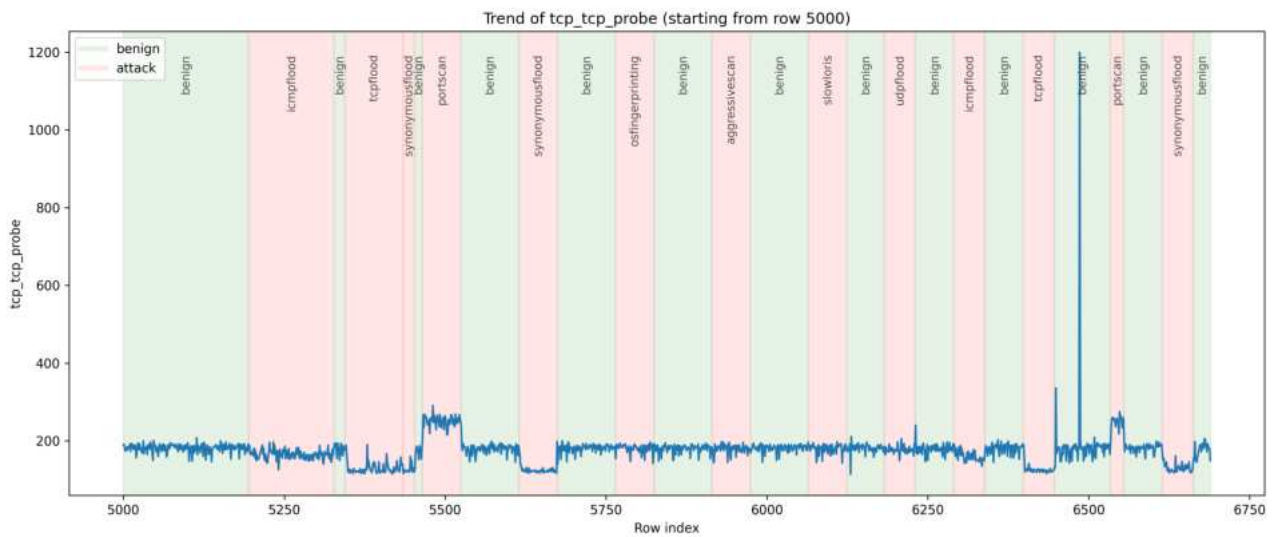


Figure 82: tcp_tcp_probe timeline

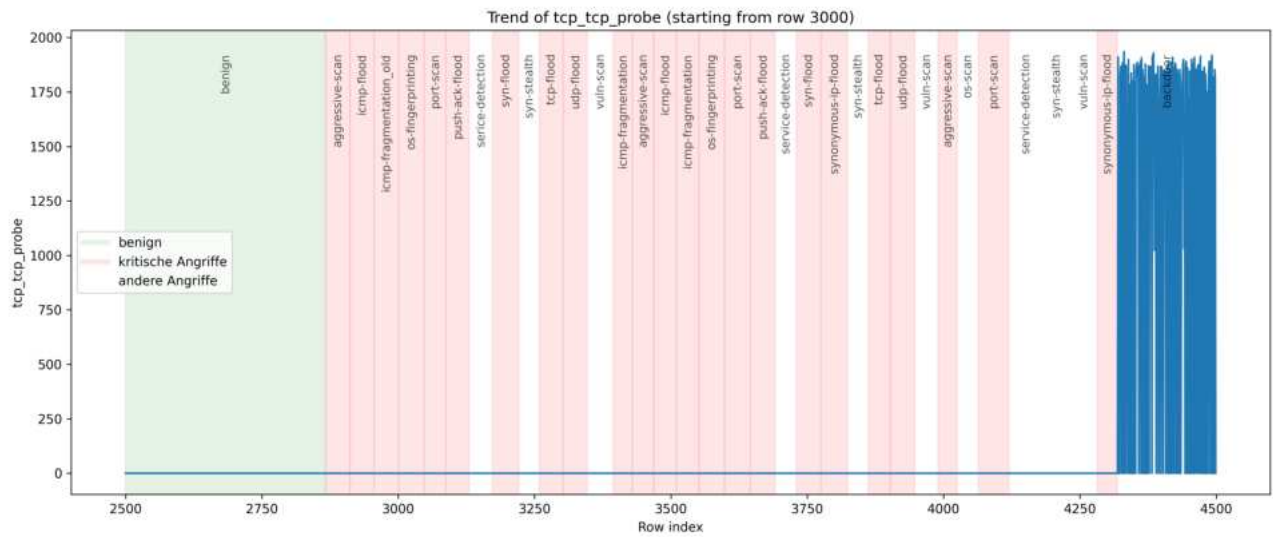


Figure 83: tcp_tcp_probe timeline (CICEVSE2024)

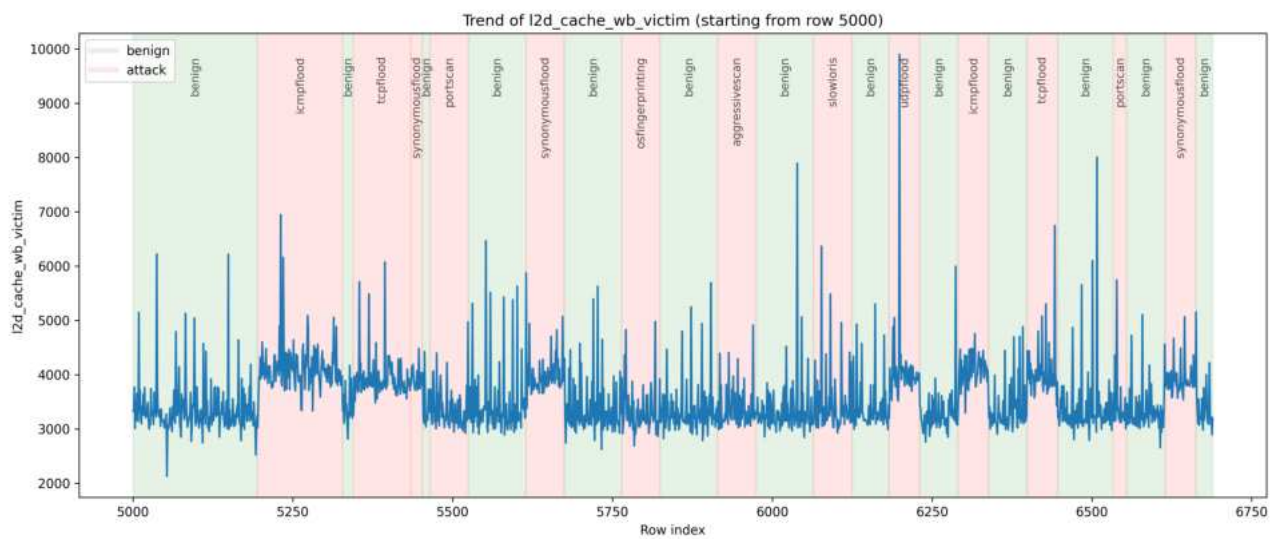


Figure 84: l2d_cache_wb_victim timeline

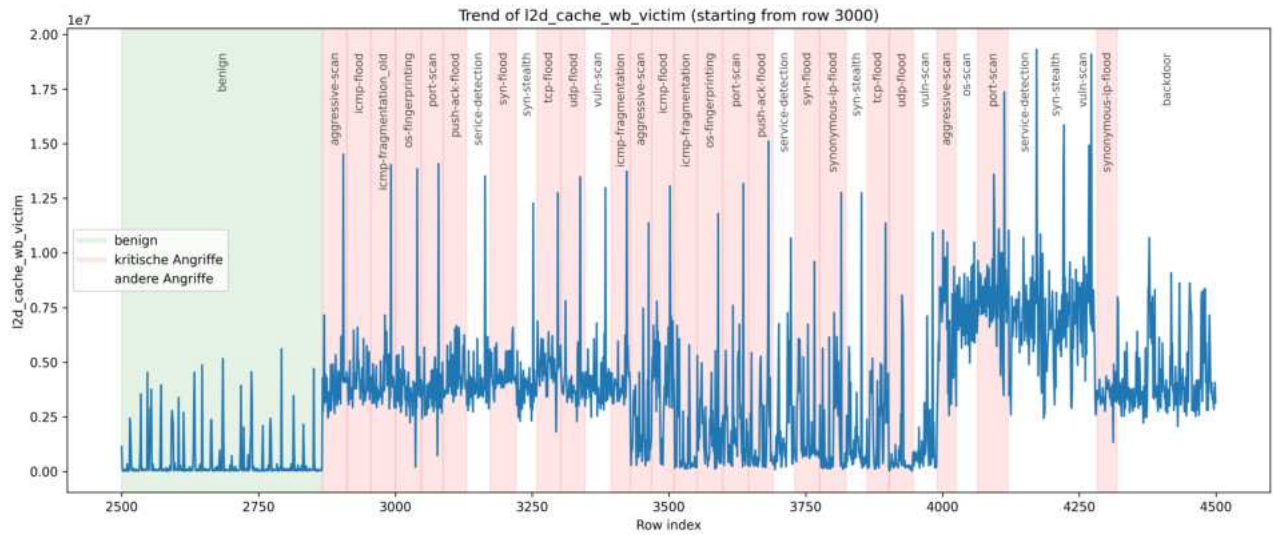


Figure 85: l2d_cache_wb_victim timeline (CICEVSE2024)

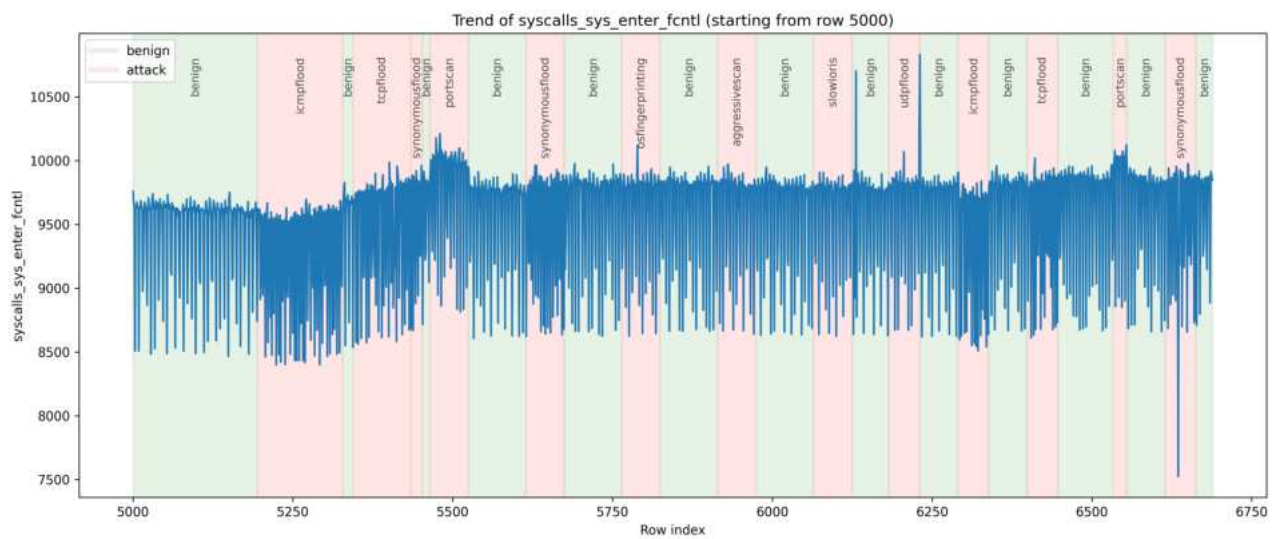


Figure 86: syscalls.sys_enter_fcntl timeline

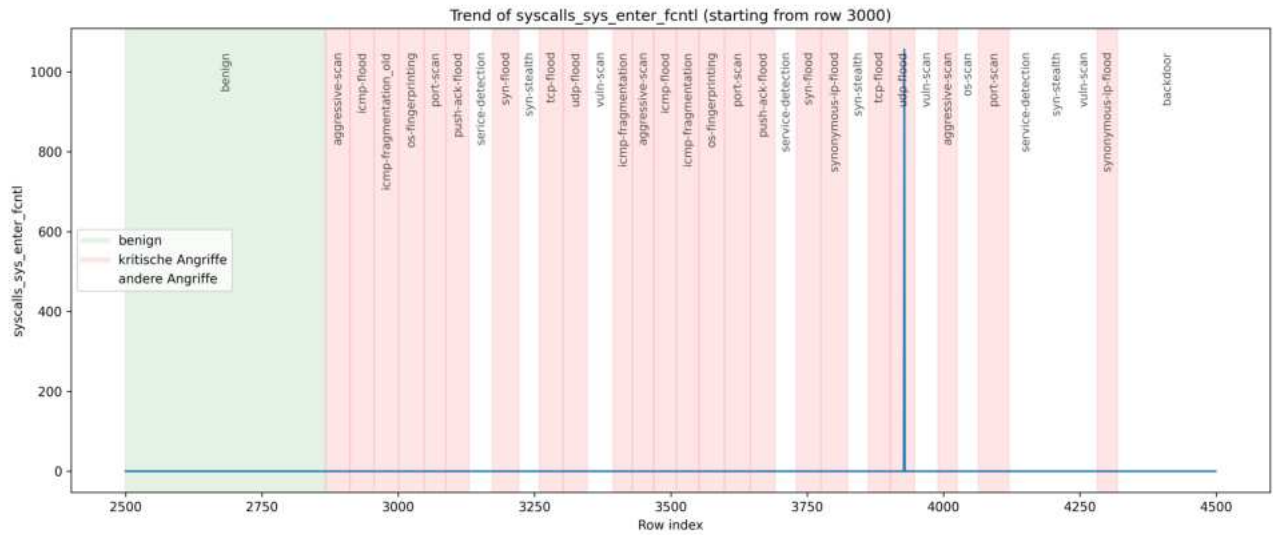


Figure 87: syscalls_sys_enter_fcntl timeline (CICEVSE2024)

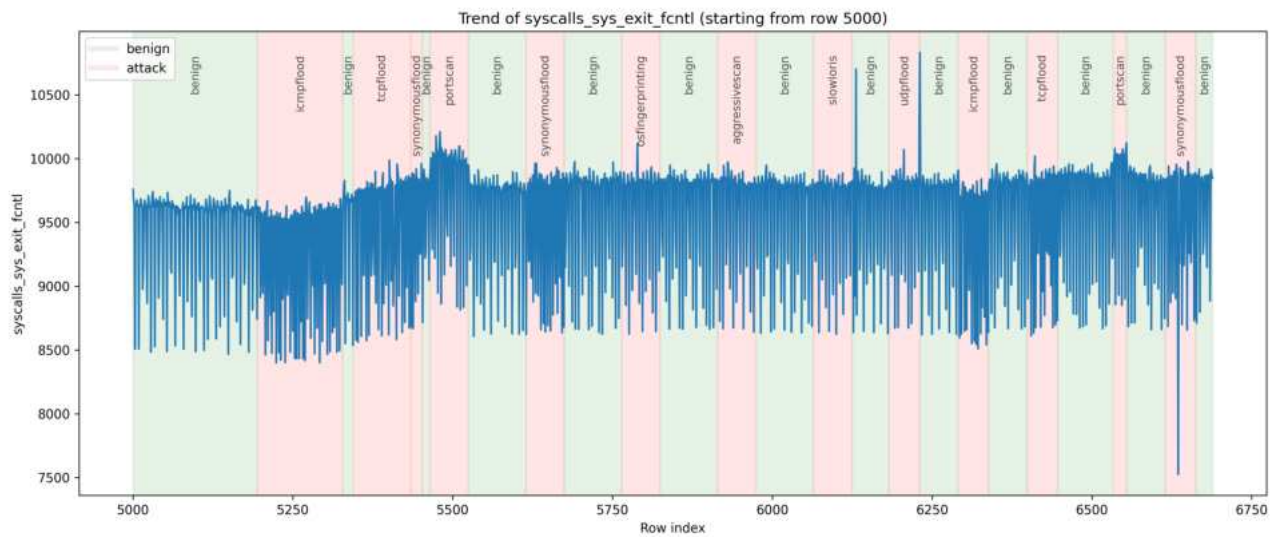


Figure 88: syscalls_sys_exit_fcntl timeline

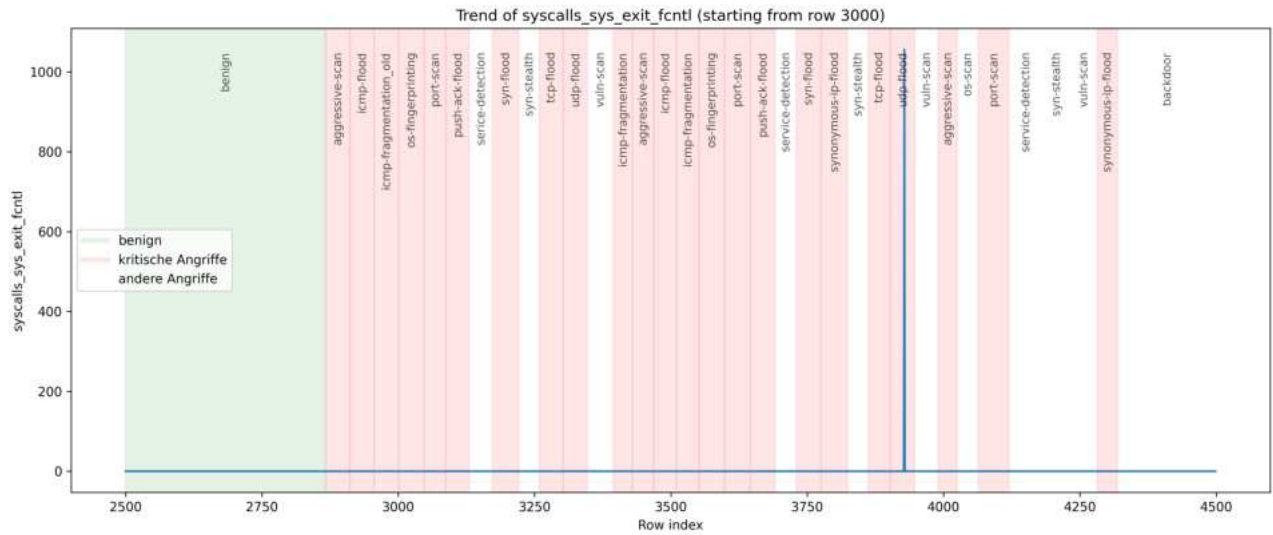


Figure 89: syscalls_sys_exit_fcntl timeline (CICEVSE2024)

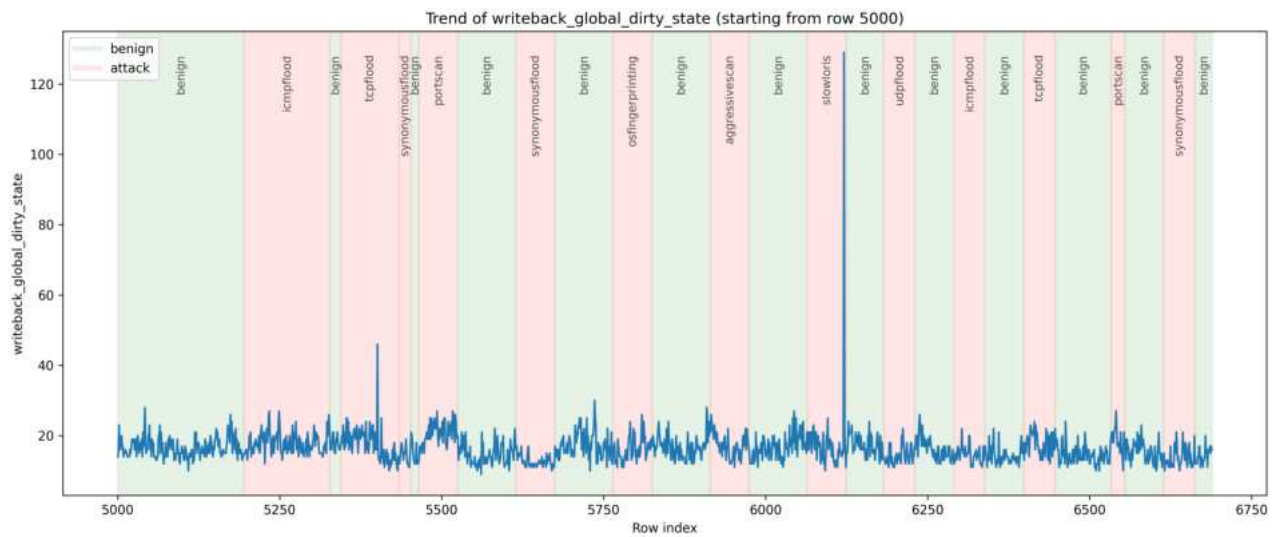


Figure 90: writeback_global_dirty_state timeline

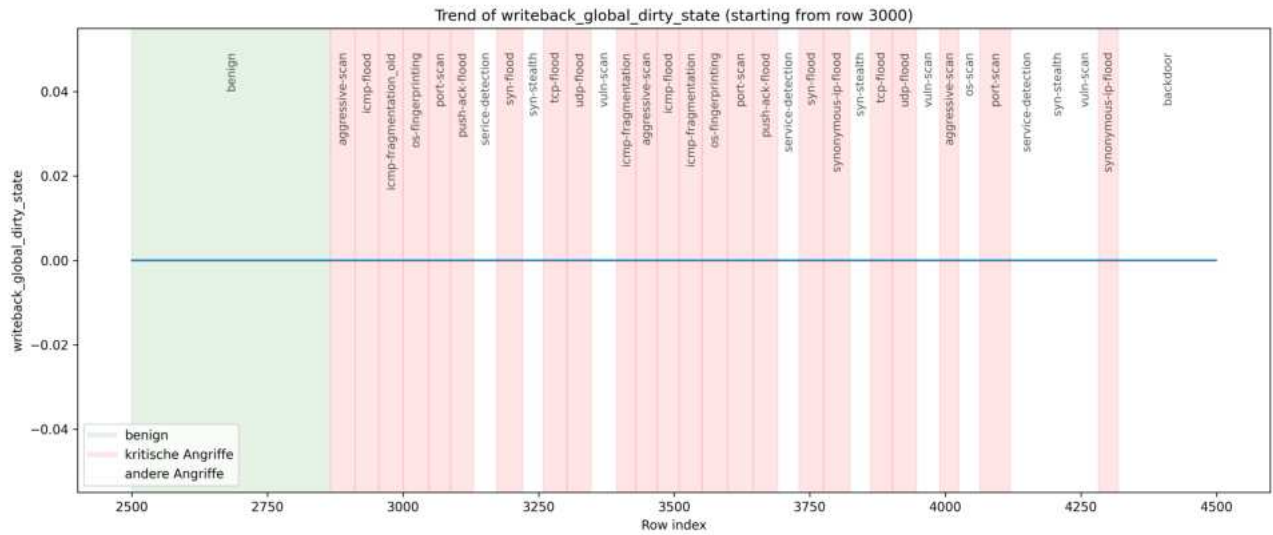


Figure 91: writeback_global_dirty_state timeline (CICEVSE2024)

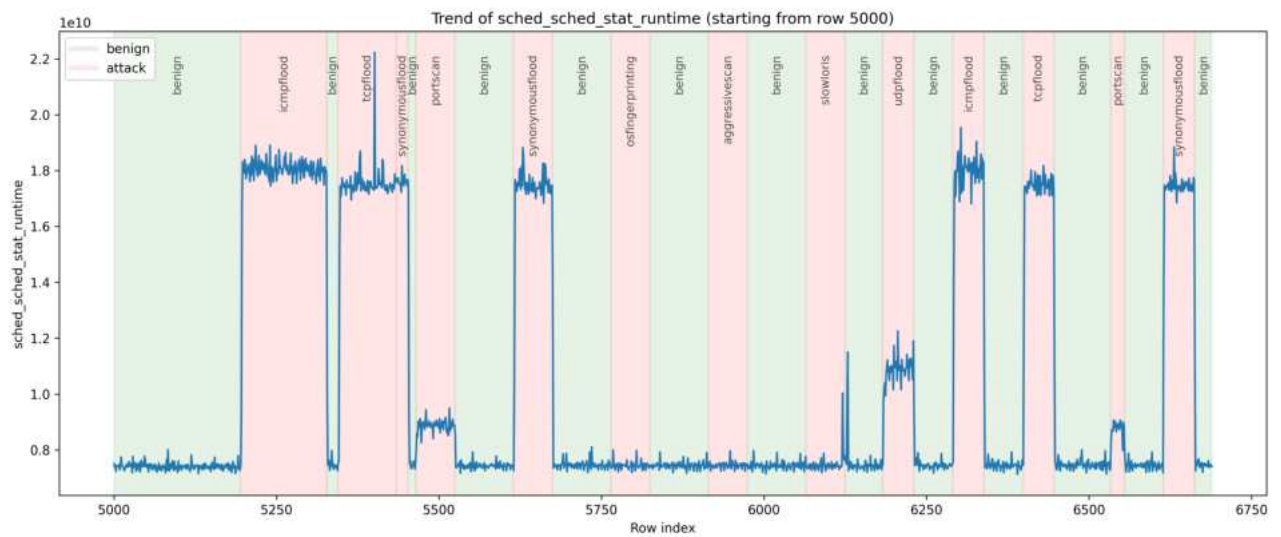


Figure 92: sched_sched_stat runtime timeline

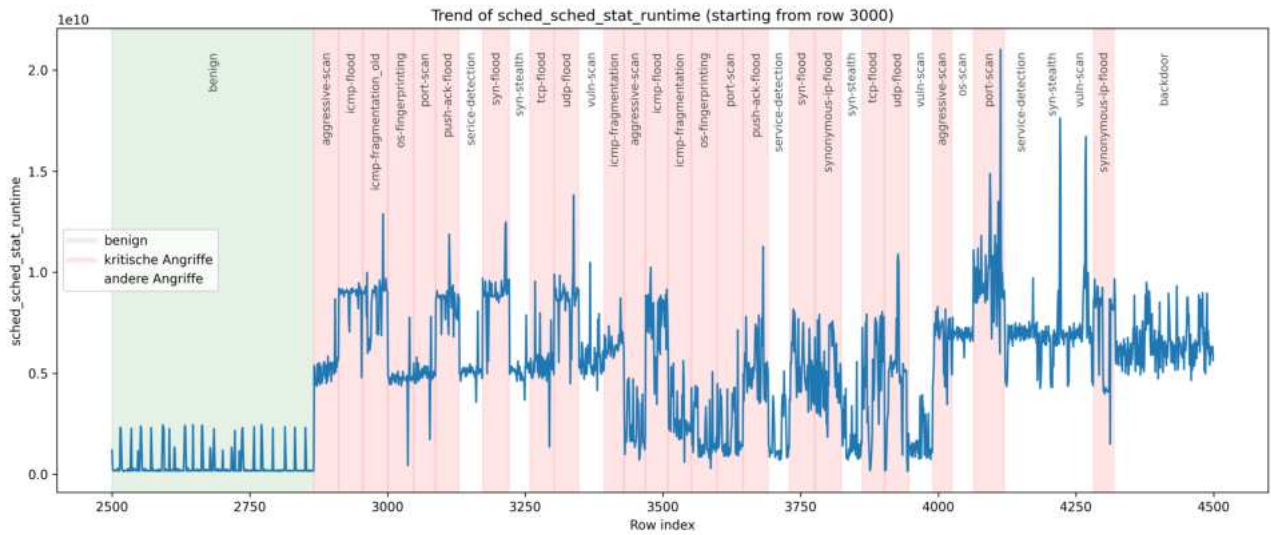


Figure 93: sched_sched_stat_runtime timeline (CICEVSE2024)

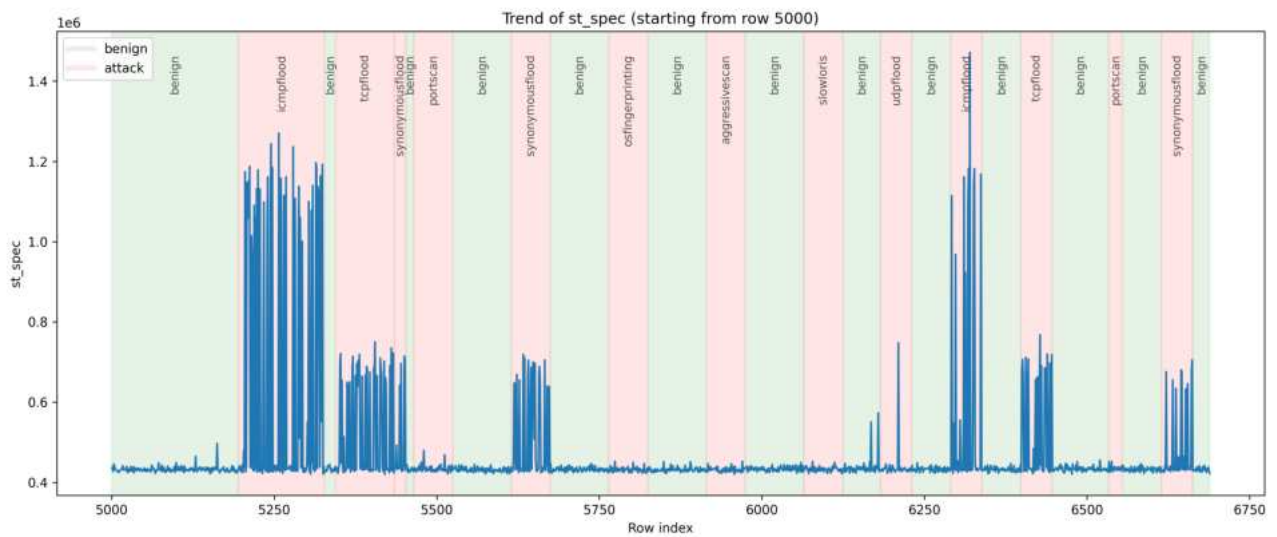


Figure 94: st_spec timeline

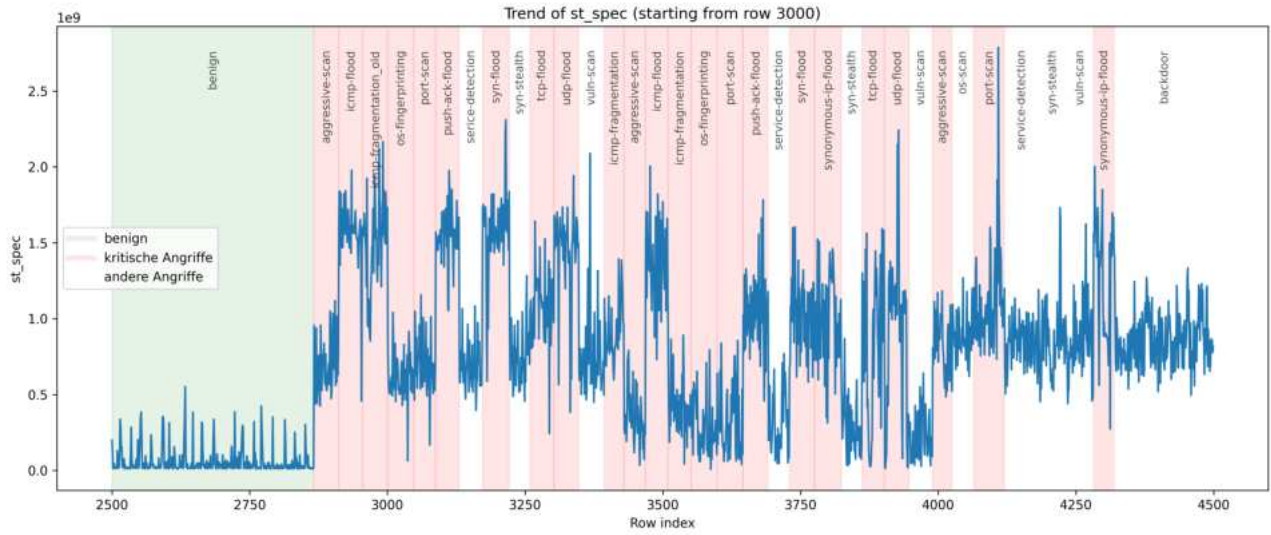


Figure 95: st_spec timeline (CICEVSE2024)

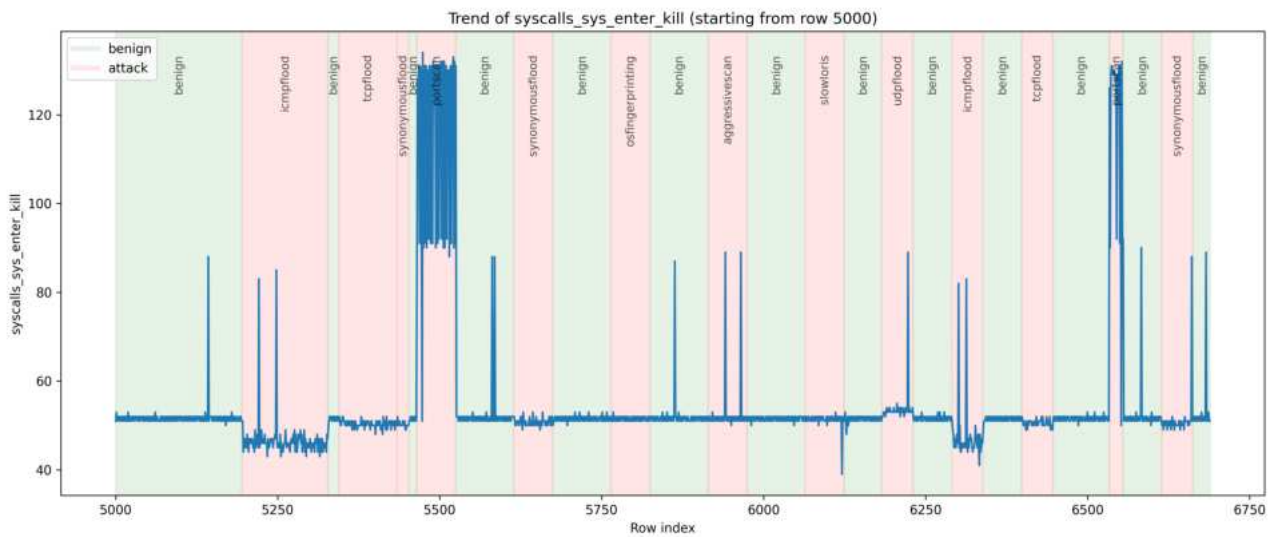


Figure 96: syscalls_sys_enter_kill timeline

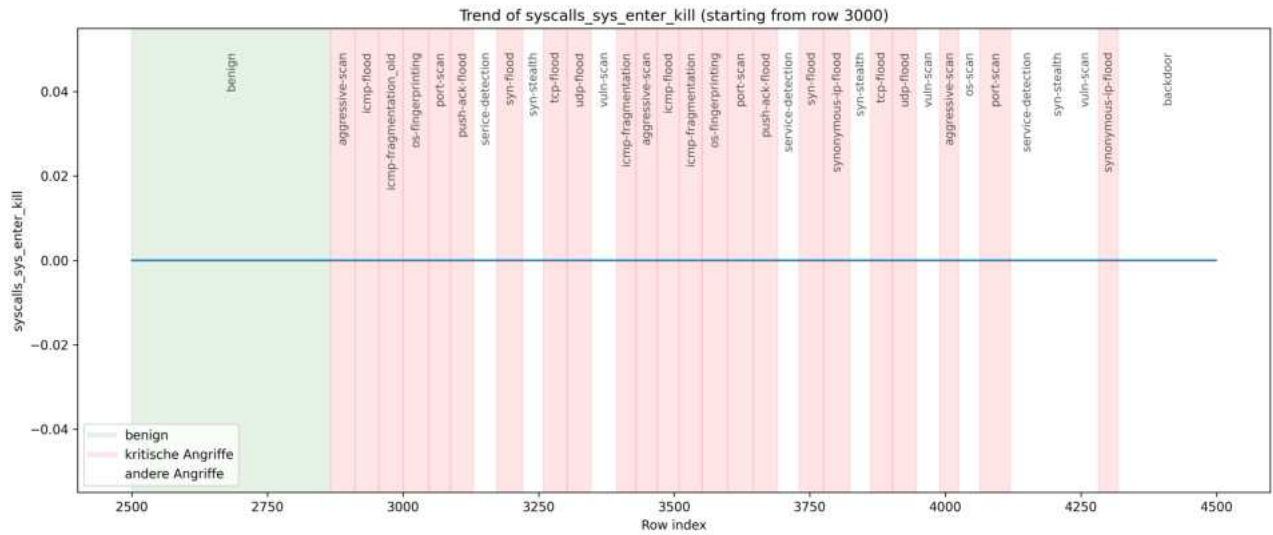


Figure 97: syscalls_sys_enter_kill timeline (CICEVSE2024)

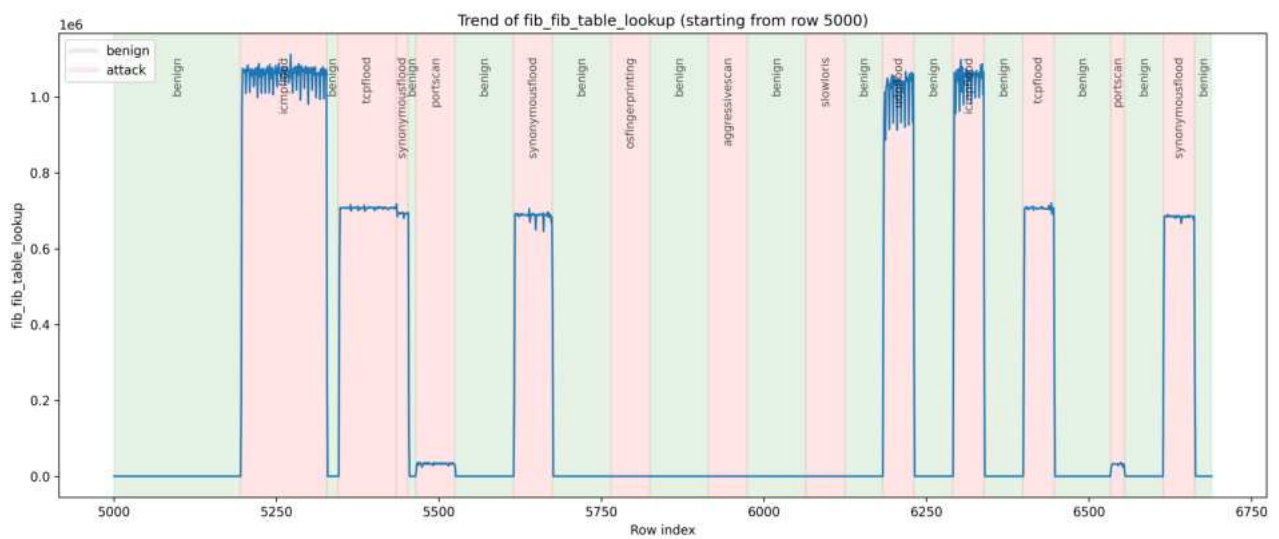


Figure 98: fib_fib_table.lookup timeline

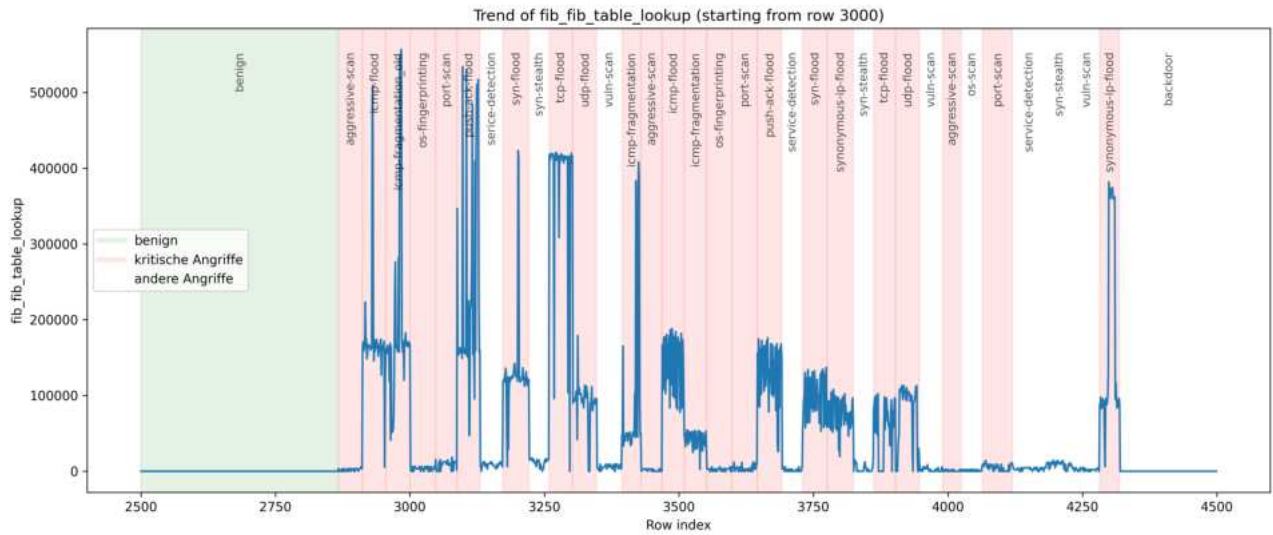


Figure 99: fib_fib_table_lookup timeline (CICEVSE2024)

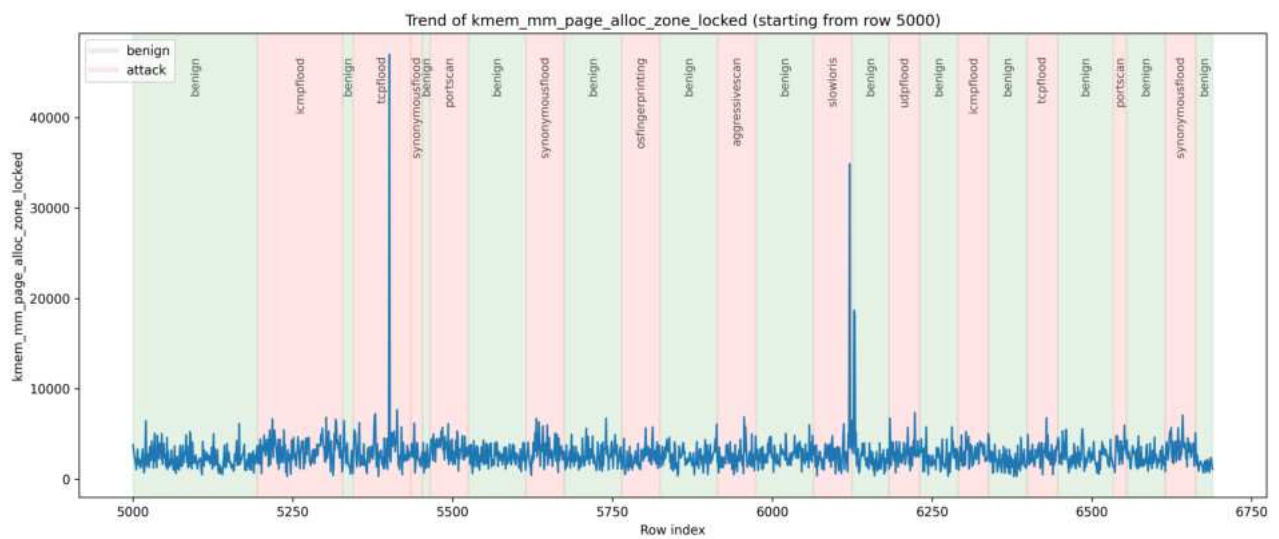


Figure 100: kmem_mm_page_alloc.zone_locked timeline

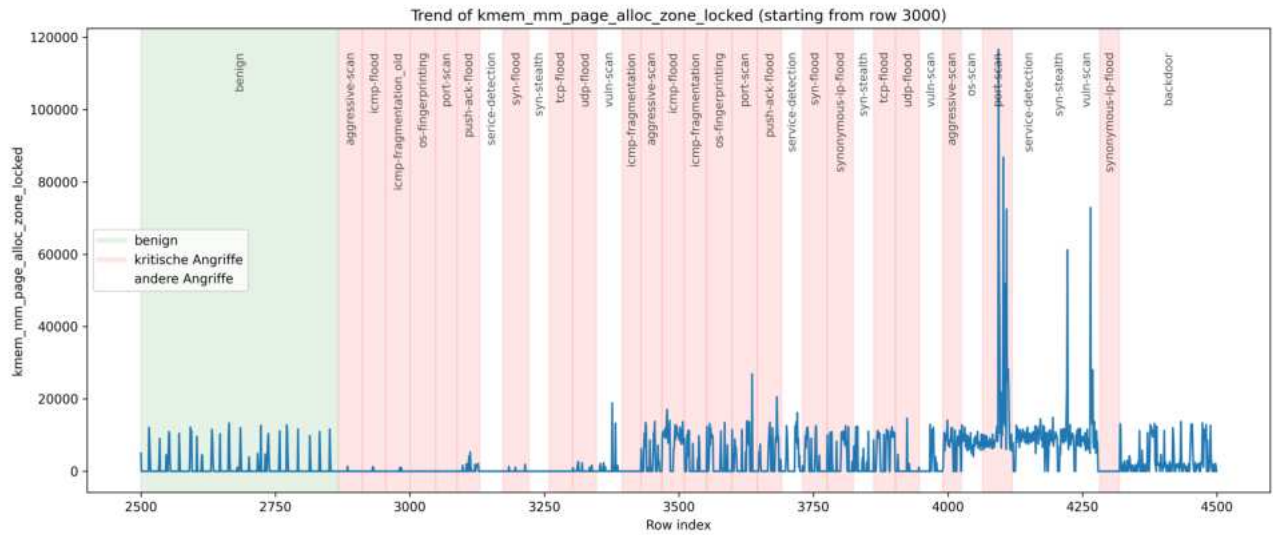


Figure 101: kmem_mm_page_alloc_zone_locked timeline (CICEVSE2024)

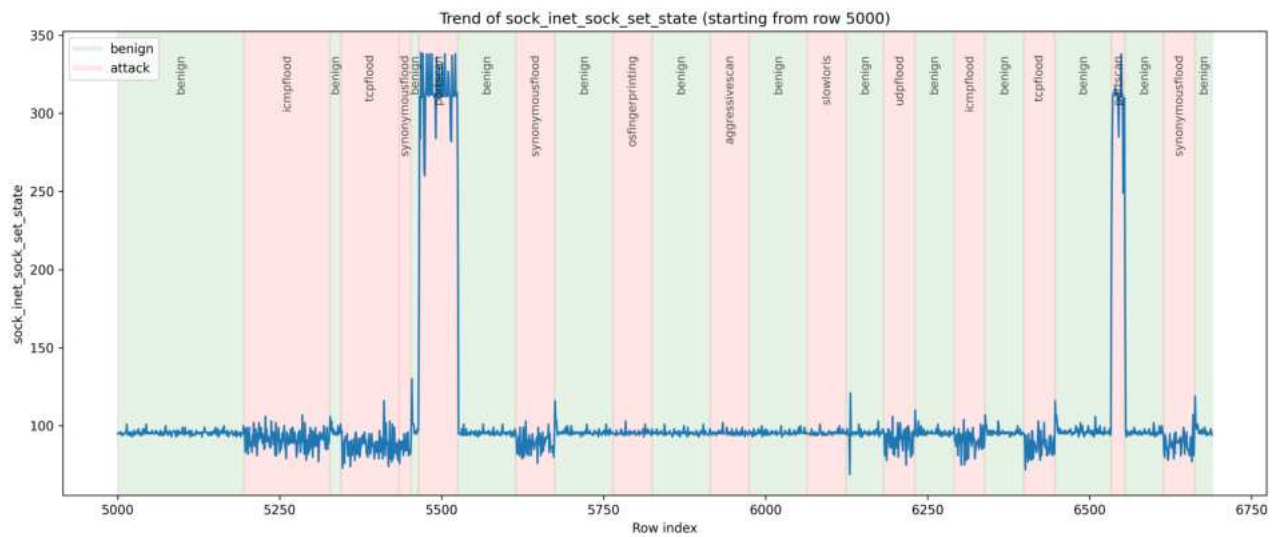


Figure 102: sock_inet_sock_set_state timeline

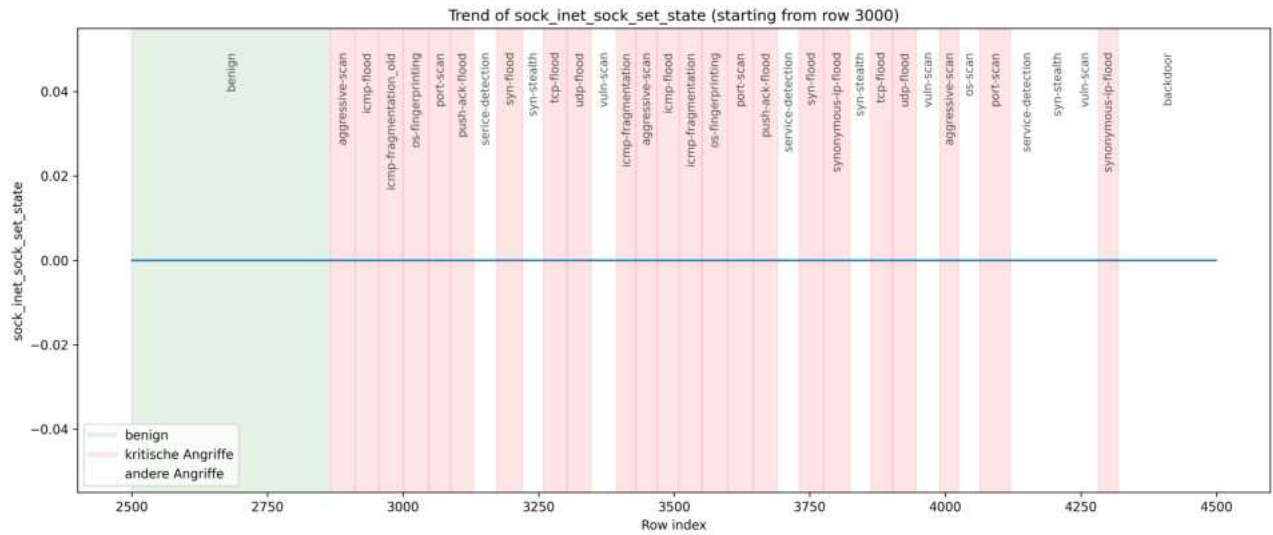


Figure 103: sock_inet_sock_set_state timeline (CICEVSE2024)

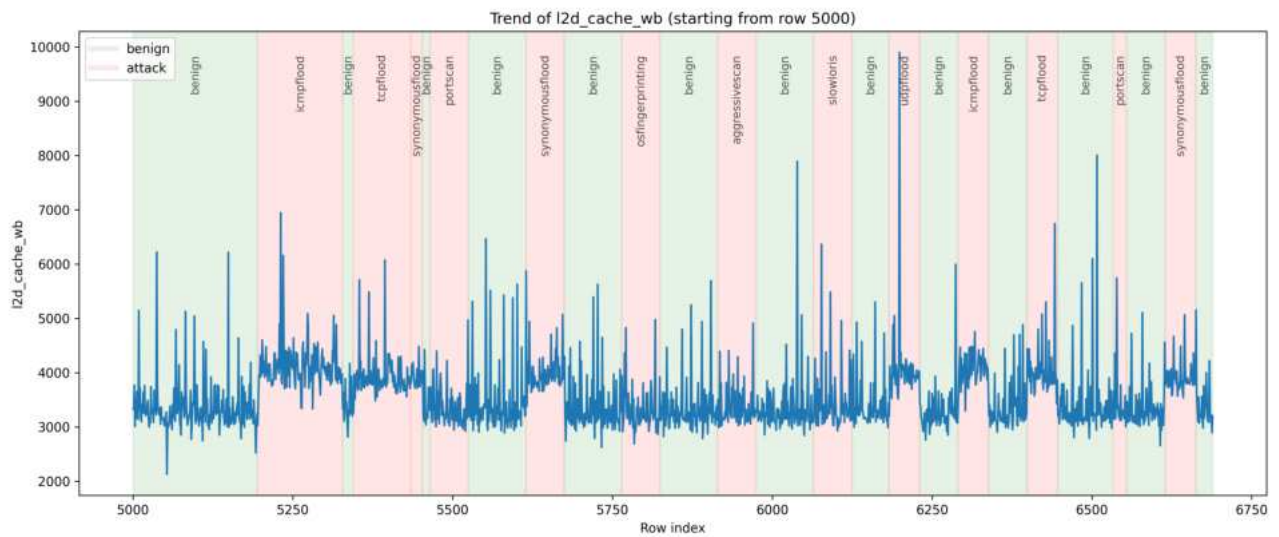


Figure 104: l2d_cache_wb timeline

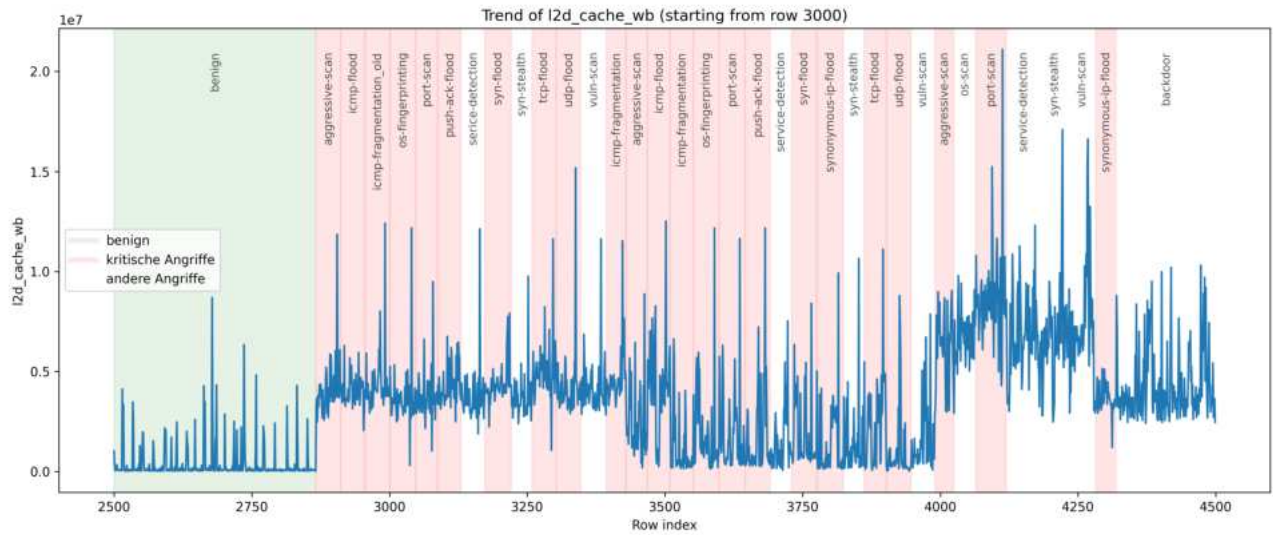


Figure 105: l2d_cache_wb timeline (CICEVSE2024)

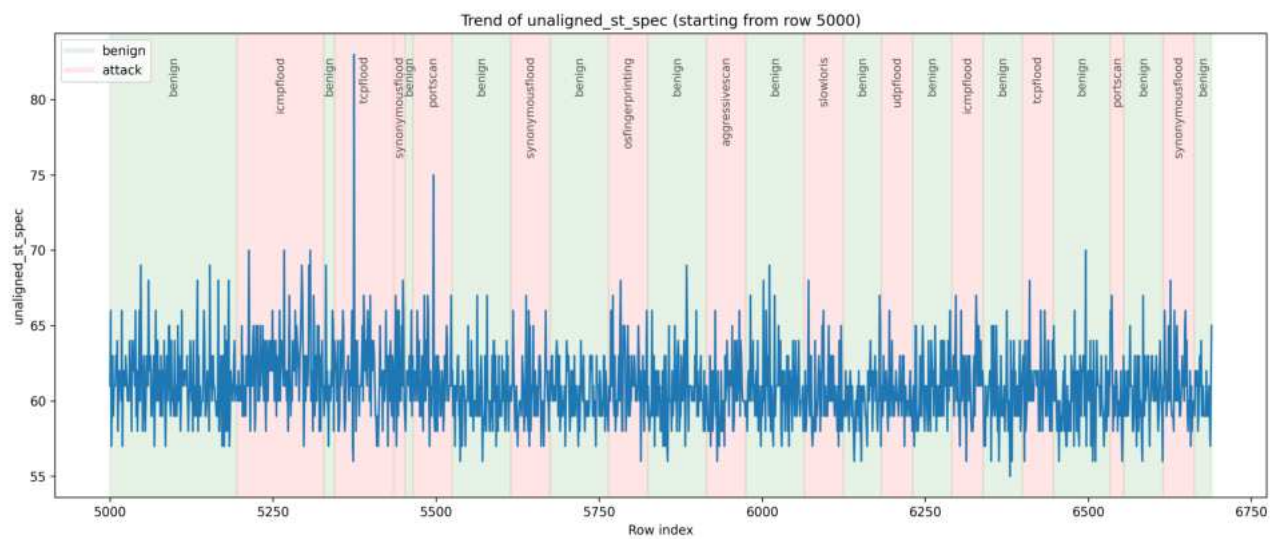


Figure 106: unaligned_st_spec timeline

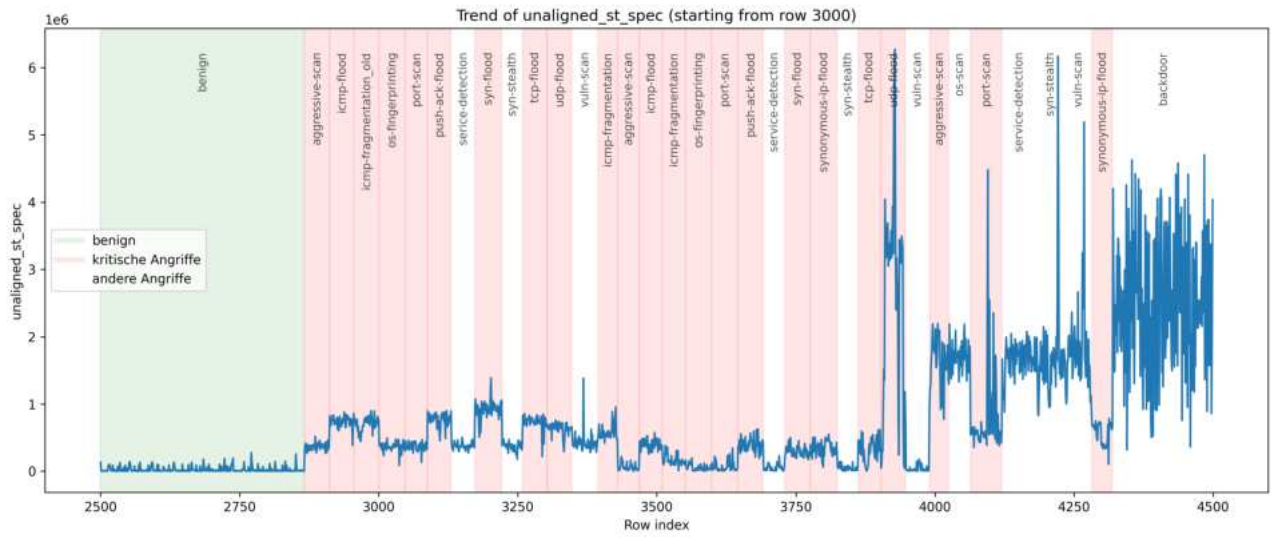


Figure 107: unaligned_st_spec timeline (CICEVSE2024)

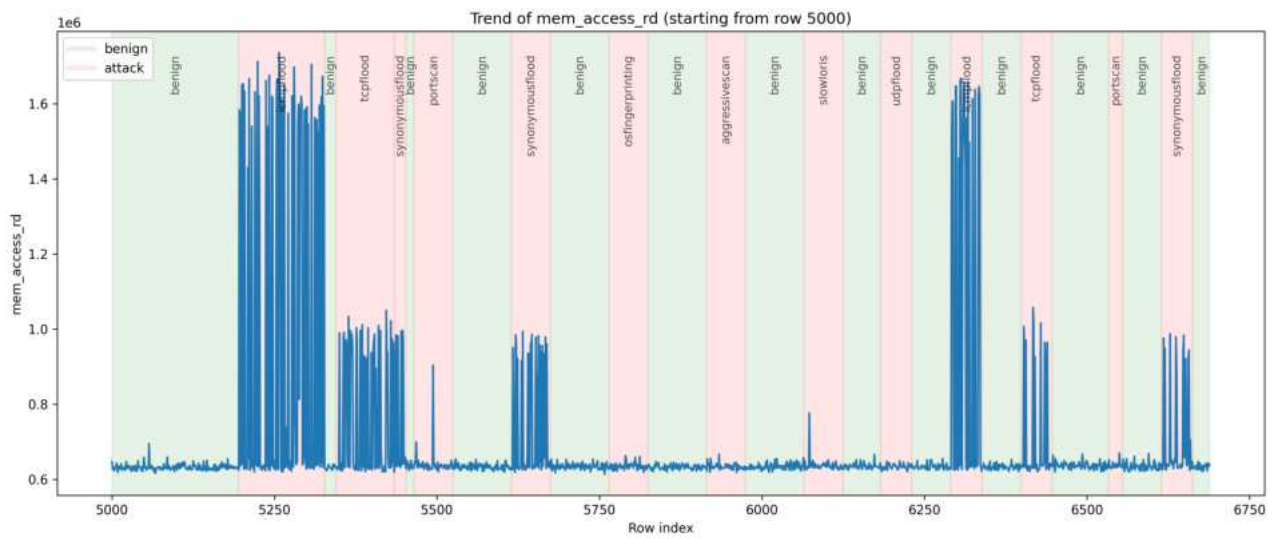


Figure 108: mem_access_rd timeline

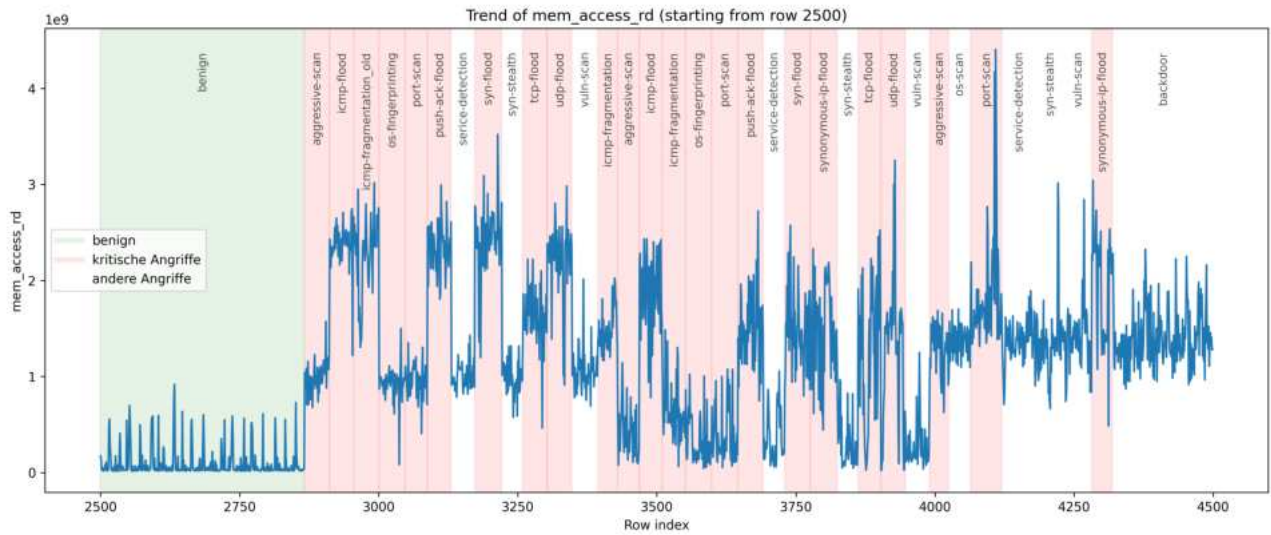


Figure 109: mem_access_rd timeline (CICEVSE2024)

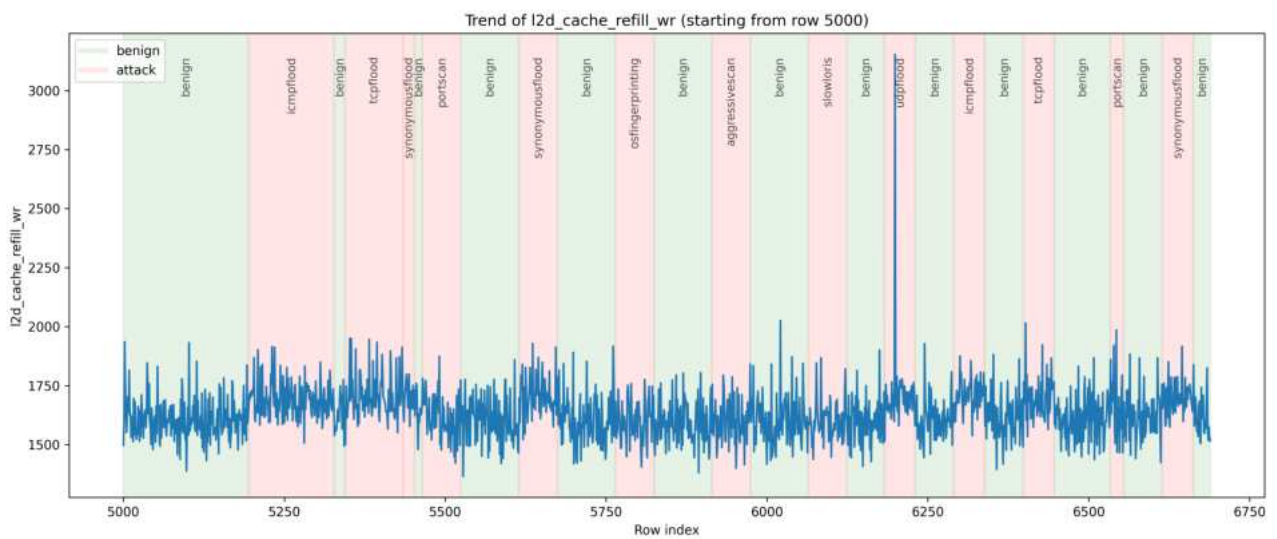


Figure 110: l2d_cache_refill_wr timeline

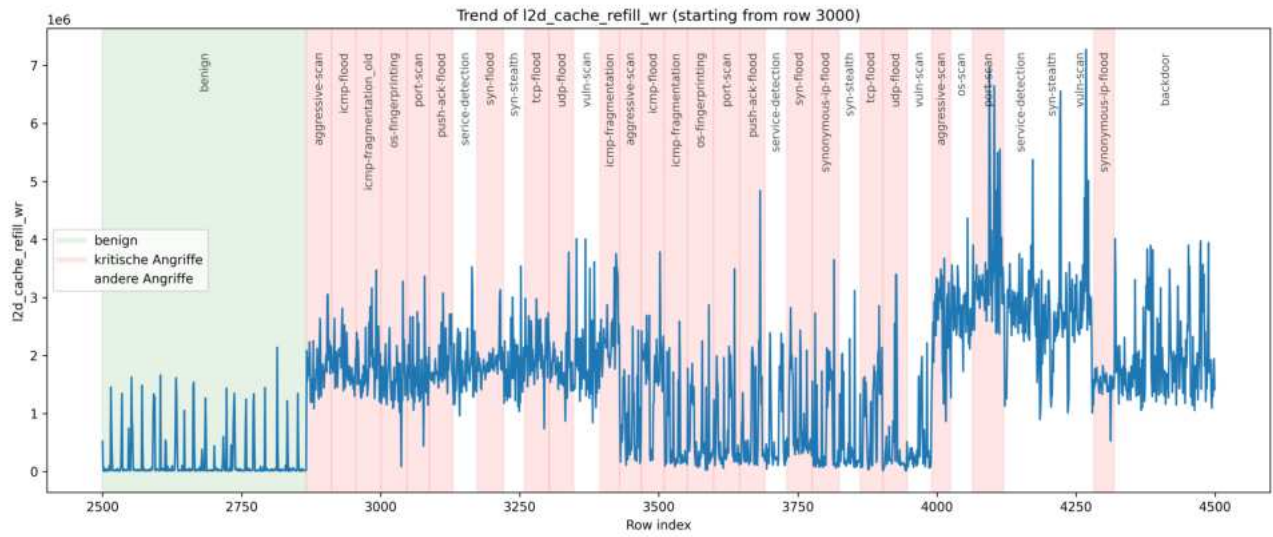


Figure 111: l2d_cache_refill_wr timeline (CICEVSE2024)

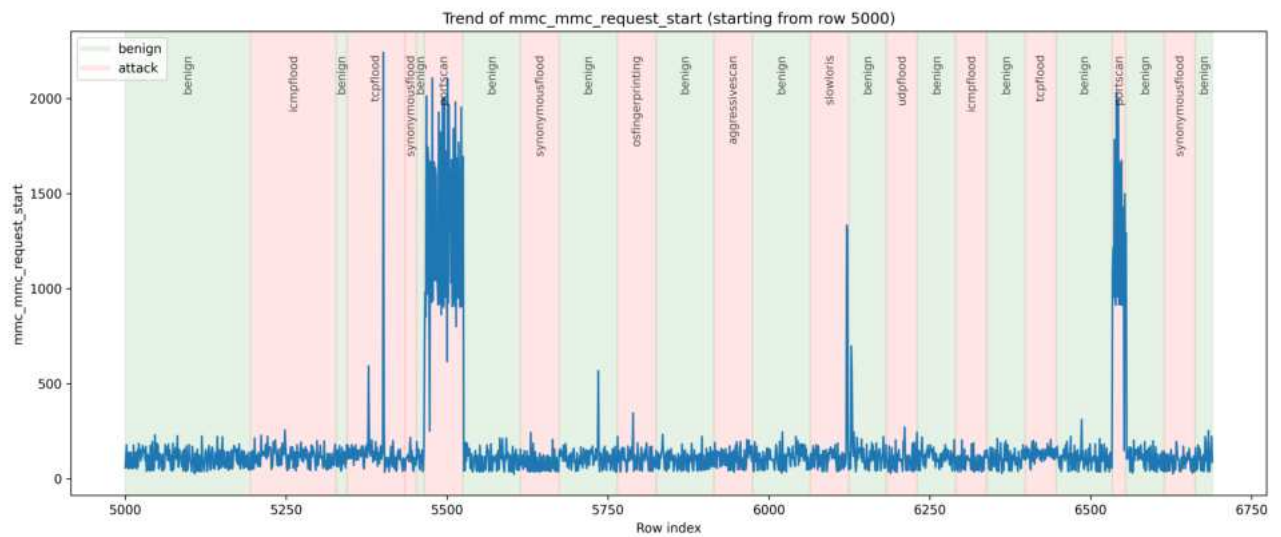


Figure 112: mmc_mmc_request_start timeline

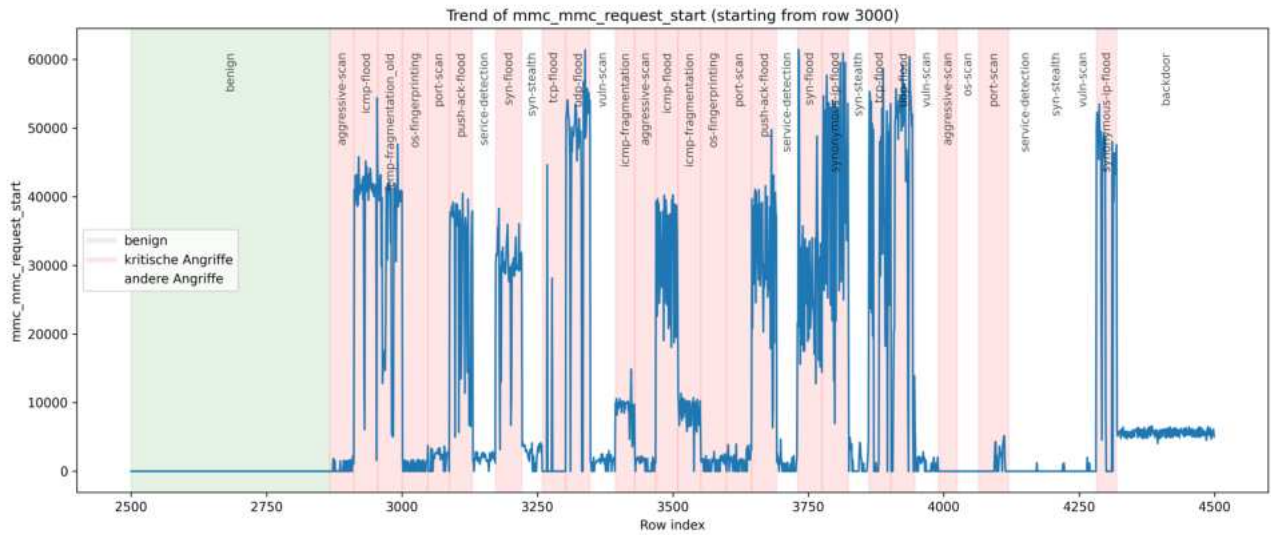


Figure 113: mmc_mmc_request_start timeline (CICEVSE2024)

Contents of the enclosed storage medium:

- Bachelor thesis in PDF format.
- Source code of the Python and Go scripts.